



BANCO DE DADOS



VAGNER ERENO QUINCOZES



ACESSE AQUI ESTE
MATERIAL DIGITAL!

EXPEDIENTE

Coordenador(a) de Conteúdo

Flavia Lumi Matuzawa

Projeto Gráfico e Capa

Arthur Cantareli Silva

Editoração

Alexandre Donzelli

Juliana Duenha

Design Educacional

Giovana Vieira

Revisão Textual

Cindy Luca

Ilustração

André Azevedo

Eduardo Aparecido Alves

Geison Odlevati Ferreira

Fotos

Shutterstock e Envato

FICHA CATALOGRÁFICA

N964 Núcleo de Educação a Distância. **QUINCOZES**, Vagner Ereno.

Banco de Dados / Vagner Ereno Quincozes, - Florianópolis, SC:
Arqué, 2025.

222 p.

ISBN papel 978-65-279-0690-2

ISBN digital 978-65-279-0691-9

1. Banco 2. Dados 3. EaD. I. Título.

CDD - 005.74

Bibliotecária: Leila Regina do Nascimento - CRB- 9/1722.

Ficha catalográfica elaborada de acordo com os dados fornecidos pelo(a) autor(a).

Impresso por:

Ano de Impressão:

AVALIE ESTE LIVRO!



CRIAR MOMENTOS DE APRENDIZAGENS INESQUECÍVEIS É O NOSSO OBJETIVO E POR ISSO, **GOSTARIAMOS DE SABER COMO FOI SUA EXPERIÊNCIA.**

Conta para nós! leva menos de 2 minutos. Vamos lá?!

DIGITE O CÓDIGO

 02511901

 Aa

RESPONDA A PESQUISA









RECURSOS DE IMERSÃO



PENSANDO JUNTOS

Este item corresponde a uma proposta de reflexão que pode ser apresentada por meio de uma frase, um trecho breve ou uma pergunta.



APROFUNDANDO

Utilizado para temas, assuntos ou conceitos avançados, levando ao aprofundamento do que está sendo trabalhado naquele momento do texto.



EU INDICO

Utilizado para agregar um conteúdo externo.



ZOOM NO CONHECIMENTO

Utilizado para desmistificar pontos que possam gerar confusão sobre o tema. Após o texto trazer a explicação, essa interlocução pode trazer pontos adicionais que contribuam para que o estudante não fique com dúvidas sobre o tema.

PRODUTOS AUDIOVISUAIS

Os elementos abaixo possuem recursos audiovisuais. Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.



PLAY NO CONHECIMENTO

Professores especialistas e convidados, ampliando as discussões sobre os temas por meio de fantásticos podcasts.



EM FOCO

Utilizado para aprofundar o conhecimento em conteúdos relevantes utilizando uma linguagem audiovisual.



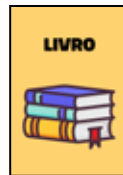
INDICAÇÃO DE FILME

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de filmes que se conectam com o tema do conteúdo.



INDICAÇÃO DE LIVRO

Uma dose extra de conhecimento é sempre bem-vinda. Aqui você terá indicações de livros que agregarão muito na sua vida profissional.



CAMINHOS DE APRENDIZAGEM

7

UNIDADE 1

INTRODUÇÃO AOS BANCOS DE DADOS RELACIONAIS	8
--	---

29

UNIDADE 2

TRANSAÇÕES E CONTROLE DE CONCORRÊNCIA	30
MODELAGEM DE DADOS.	46

67

UNIDADE 3

NORMALIZAÇÃO: PRIMEIRA, SEGUNDA, TERCEIRA E QUARTA FORMA NORMAL . . .	68
INTRODUÇÃO À LINGUAGEM SQL.	90

113

UNIDADE 4

JOINS EM SQL.	114
SQL: SUBCONSULTAS E JUNÇÕES AVANÇADAS	138

163

UNIDADE 5

ADMINISTRAÇÃO DE BANCOS DE DADOS	164
ESTUDOS DE CASO E EXEMPLOS PRÁTICOS.	192

unidade





TEMA DE APRENDIZAGEM 1

INTRODUÇÃO AOS BANCOS DE DADOS RELACIONAIS

MINHAS METAS

- Conhecer os bancos de dados relacionais e não relacionais.
- Diferenciar bancos de dados relacionais dos não relacionais.
- Explorar a evolução dos bancos de dados relacionais.
- Entender os conceitos básicos dos bancos de dados relacionais.
- Avaliar as vantagens e as desvantagens dos bancos de dados relacionais.
- Identificar os principais bancos relacionais do mercado.
- Aplicar os conceitos fundamentais em cenários de banco de dados.

INICIE SUA JORNADA

Suponha que você trabalha no departamento de Tecnologia da Informação (TI) de uma grande empresa de comércio eletrônico. Durante uma reunião com a equipe de vendas, surge a seguinte questão: como os dados dos clientes estão sendo armazenados e acessados para gerar relatórios? Um problema importante é identificado: as consultas ao banco de dados estão cada vez mais lentas, o que está afetando a agilidade na geração de relatórios estratégicos. A equipe percebe que o banco de dados atual está mal estruturado, o que dificulta a obtenção de informações de maneira eficiente. Como você pode ajudar a melhorar essa situação?

Resolver esse problema é crucial, pois a lentidão no acesso aos dados pode impactar negativamente várias áreas da empresa, desde o atendimento ao cliente até a tomada de decisões por parte da liderança. Uma estrutura de banco de dados bem-organizada é essencial para garantir que as operações comerciais possam continuar sem interrupções. Logo, uma solução eficiente pode não apenas aumentar a produtividade, mas também melhorar a experiência do cliente e a competitividade da empresa.

Como parte do time de TI, você teria que desenvolver uma solução para resolver esse problema. Isso envolveria repensar a estrutura das tabelas, otimizar a organização dos dados e melhorar a relação entre eles, buscando formas de garantir um acesso mais rápido e eficiente. A escolha de ferramentas, como MySQL ou PostgreSQL, pode ser fundamental, o que permite simular e implementar melhorias que afetarão diretamente o desempenho do sistema.

Ao refletir sobre essa questão, é importante considerar que os bancos de dados mal estruturados são um dos principais gargalos em qualquer sistema de TI. No entanto, com uma base sólida de conhecimento sobre bancos de dados relacionais, você estará preparado(a) para identificar problemas, propor melhorias e, acima de tudo, garantir que as informações fluam de maneira eficiente. Afinal, dados organizados são a chave para o sucesso de qualquer organização moderna.

**PLAY NO CONHECIMENTO**

Ouçá o nosso podcast, no qual exploramos as tendências modernas em bancos de dados multimodelo. É discutido o modo como eles integram diferentes tipos de dados e se tornaram uma solução flexível para as demandas atuais do mercado. Não perca essa oportunidade de aprender algo novo e complementar! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Que tal rever um conceito fundamental? Assista ao vídeo O que é um Banco de Dados? e descubra como os dados são organizados e gerenciados de maneira eficiente no mundo digital. <https://youtu.be/P6uiLQ6F7Mo?si=XedJMWADqaea7soF>



DESENVOLVA SEU POTENCIAL

EVOLUÇÃO E HISTÓRIA DOS BANCOS DE DADOS RELACIONAIS

Para entender completamente os bancos de dados relacionais, é essencial conhecer o contexto histórico em que eles surgiram e como evoluíram ao longo do tempo. Esse conhecimento proporciona uma visão mais clara sobre os problemas que o modelo relacional deve resolver e como ele se tornou um padrão amplamente adotado no mercado.

Histórico dos bancos de dados relacionais

Os bancos de dados relacionais foram propostos pela primeira vez por **Edgar F. Codd**, em 1970, enquanto ele trabalhava na IBM. Codd propôs um modelo matemático chamado **modelo relacional**, que organizava dados em tabelas (relações) e permitia a realização de operações sobre elas por meio de uma linguagem declarativa conhecida como **SQL** (*Structured Query Language*) (Cood, 2021).

Antes disso, os sistemas de banco de dados existentes, como os **bancos de dados hierárquicos** e **em rede**, organizavam os dados de forma rígida, limitando a flexibilidade e a escalabilidade. O modelo relacional de Codd revolucionou a forma como os dados eram organizados e acessados, separando o armazenamento físico dos dados de sua organização lógica.

Vários eventos importantes marcaram a evolução e a popularização dos bancos de dados relacionais (Ricardo; Urban; Davis, 2022):



1970S - MODELO RELACIONAL PROPOSTO:

Edgar Codd publica um famoso artigo, A Relational Model of Data for Large Shared Data Banks, introduzindo o conceito de bancos de dados relacionais.

1974 - CRIAÇÃO DA LINGUAGEM SQL:

a International Business Machines Corporation (IBM) começa a desenvolver a linguagem SQL baseada no modelo relacional de Codd, tornando-se a linguagem padrão para interação com bancos de dados relacionais.

1980S - COMERCIALIZAÇÃO DOS BANCOS RELACIONAIS:

empresas, como Oracle e IBM, lançam as primeiras versões comerciais de sistemas de gerenciamento de bancos de dados relacionais (SGBDR), como Oracle Database e IBM DB2.

1990S - ADOÇÃO EM MASSA:

com o aumento do uso de sistemas ERP e CRM em grandes empresas, os bancos de dados relacionais se tornaram a espinha dorsal da gestão de dados corporativa.

2000S - AVANÇOS EM ESCALABILIDADE:

com o crescimento da internet e das grandes aplicações web, surgem as necessidades de escalabilidade, que impulsionam o desenvolvimento de bancos de dados distribuídos e abordagens híbridas, misturando bancos de dados relacionais e não relacionais.

Esses marcos históricos destacam a evolução dos bancos de dados relacionais. A seguir, abordaremos os conceitos fundamentais dos bancos de dados relacionais, os quais são indispensáveis para o gerenciamento eficaz de dados e a garantia da integridade das transações.

CONCEITOS FUNDAMENTAIS DOS BANCOS DE DADOS RELACIONAIS

Os bancos de dados relacionais são amplamente utilizados para organizar e gerenciar grandes volumes de dados de maneira eficiente e estruturada. Eles se baseiam no modelo relacional, que utiliza tabelas inter-relacionadas para armazenar dados, garantindo **integridade** e **consistência**.

Nesta seção, vamos explorar os conceitos básicos que formam a base dos bancos de dados relacionais, como tabelas, chaves primárias e chaves estrangeiras, além de abordar as diferenças entre bancos de dados relacionais e não relacionais.

Estrutura e funcionamento dos bancos de dados relacionais

Um banco de dados relacional é composto por diversas **tabelas**, também chamadas de relações, que armazenam os dados em formato tabular. Cada tabela contém **colunas** (ou campos), que definem o tipo de dado que será armazenado, e **linhas** (ou registros), que contêm os dados em si.

Por exemplo, imagine uma tabela chamada “**Clientes**”, que armazena informações sobre os clientes de uma empresa. Essa tabela pode ter colunas, como “ID do Cliente”, “Nome”, “Endereço” e “Telefone”. Cada linha representa um cliente específico, preenchendo as informações em suas respectivas colunas. Essa organização facilita a busca, a filtragem e a atualização dos dados de maneira eficiente.

Esse modelo facilita o acesso e a manipulação dos dados. Para garantir a organização e a integridade dos dados, os bancos de dados relacionais utilizam **chaves primárias** e **chaves estrangeiras**.

Chave Primária (*Primary Key*)

A **chave primária** é o identificador único de um registro em uma tabela. Ela é usada para garantir que não existam duas linhas (registros) com o mesmo valor em uma determinada coluna. Geralmente, essa coluna é chamada de **ID** (abreviação de Identificador), porque é justamente o campo que identifica cada registro de forma única.

Por exemplo, na tabela **Clientes** (tabela 1), o campo **ID do Cliente** serve como chave primária. Isso significa que cada cliente possui um número único, evitando duplicidade de informações. Dessa forma, se dois clientes tivessem o mesmo **ID**, seria impossível distingui-los corretamente.

ID DO CLIENTE	NOME	ENDEREÇO	TELEFONE
1	João Silva	Rua A, 123	(11) 99999-9999
2	Maria Oliveira	Rua B, 456	(21) 98888-8888
3	Pedro Souza	Rua C, 789	(31) 97777-7777

Tabela 1 - Exemplo de tabela Clientes de um banco de dados / Fonte: o autor.

Portanto, o **ID do Cliente** é uma maneira prática e eficaz de identificar individualmente cada cliente.

Chave Estrangeira (Foreign Key)

A **chave estrangeira** é um campo em uma tabela que faz referência à chave primária de outra tabela, criando uma relação entre elas. O objetivo é que um registro em uma tabela possa “apontar” para um registro relacionado em outra tabela, conectando os dados de forma lógica.

Por exemplo, na tabela **Pedidos** (Tabela 2), existe um campo chamado **ID do Cliente**, que se refere à chave primária da tabela **Clientes**. Isso permite associar cada pedido a um cliente específico. No exemplo a seguir, podemos perceber que os pedidos 101 e 103 foram feitos pelo cliente com **ID 1** (João Silva), enquanto o pedido 102 foi feito pelo cliente com **ID 2** (Maria Oliveira).

ID DO PEDIDO	ID DO CLIENTE	PRODUTO	VALOR TOTAL
101	1	Pizza	120
102	2	Hambúrguer	40,5
103	1	Sushi	150

Tabela 2 – Tabela Pedidos de um banco de dados

Essa relação entre as tabelas é fundamental para garantir a **integridade referencial**. Isso significa que os dados de diferentes tabelas estão corretamente relacionados e mantidos de forma consistente.

Por exemplo, um pedido não pode ser registrado para um cliente que não exista na tabela **Clientes**.

Ao entender os conceitos de **chave primária** e **chave estrangeira**, você pode visualizar como os dados são organizados e relacionados em um banco de dados relacional. Essas chaves garantem a **integridade** e a **consistência** dos dados, permitindo que diferentes conjuntos de informações estejam conectados de forma eficaz.

Esse modelo de organização é essencial para o bom funcionamento de sistemas que lidam com grandes volumes de dados, como sistemas de gestão empresarial ou plataformas de *e-commerce*.

BANCOS DE DADOS RELACIONAIS E NÃO RELACIONAIS

Agora que entendemos como os bancos de dados relacionais funcionam, é importante compararmos brevemente os bancos de dados relacionais com os bancos de dados não relacionais (NoSQL), apenas para destacar as diferenças.

Os **bancos de dados relacionais**, como vimos, organizam as informações em tabelas estruturadas, permitindo a criação de relacionamentos bem definidos entre os dados. Essa estrutura é ideal para situações em que os dados são bem-organizados e estruturados, com relações claras entre diferentes entidades. Um exemplo típico é um sistema de vendas, em que você pode ter tabelas separadas para **Clientes**, **Produtos** e **Pedidos**, todas conectadas por chaves estrangeiras que garantem a integridade referencial.

Por outro lado, os **bancos de dados não relacionais**, também conhecidos como **NoSQL**, adotam uma abordagem diferente para o armazenamento e o gerenciamento de dados. Eles são mais adequados para dados **não estruturados** ou **semi-estruturados**, como documentos, imagens ou grandes volumes de dados dinâmicos que mudam frequentemente (Khan *et al.*, 2023).

Ao contrário dos bancos relacionais, os bancos NoSQL não dependem de tabelas e relações fixas. Eles usam modelos mais flexíveis, como documentos, colunas largas, grafos ou pares chave-valor, permitindo maior escalabilidade e flexibilidade em certos cenários (Khan *et al.*, 2023).

Por exemplo, considere o armazenamento de *posts* de uma rede social. Cada *post* pode ter uma estrutura diferente: alguns contêm apenas texto, enquanto outros incluem imagens, links ou metadados adicionais.

Ao contrário de exigir que todos esses dados se encaixem em uma tabela rígida, um banco de dados não relacional pode armazená-los como documentos **JSON**, em que a estrutura pode variar conforme necessário. Esse modelo oferece maior flexibilidade, permitindo armazenar dados dinâmicos sem a necessidade de reformular o esquema do banco de dados.



Agora, imagine que você está armazenando os dados de uma loja on-line. Em um banco de dados **relacional**, você teria uma tabela para Produtos e outra para Pedidos, com uma chave estrangeira para relacionar os produtos aos pedidos. Já em um banco de dados **não relacional**, ao contrário de dividir essas informações em diferentes tabelas, você poderia armazenar todos os detalhes do pedido em um único documento. Esse documento conteria todos os dados relacionados ao pedido de forma encapsulada, assim como é mostrado no exemplo de **documento JSON** da Figura 1.

```
{
  "id_pedido": 103,
  "cliente": {
    "id_cliente": 1,
    "nome": "João Silva",
    "endereco": "Rua A, 123"
  },
  "produtos": [
    {
      "id_produto": 201,
      "nome_produto": "Teclado Mecânico",
      "quantidade": 1,
      "preco_unitario": 150.0
    },
    {
      "id_produto": 202,
      "nome_produto": "Mouse Óptico",
      "quantidade": 2,
      "preco_unitario": 50.0
    }
  ],
  "data_pedido": "2024-10-12",
  "valor_total": 250.0
}
```

Figura 1 - Documento JSON com dados de um Pedido / Fonte: o autor.

Descrição da Imagem: é exibida uma estrutura de dados no formato JSON contendo informações de um pedido. Os principais campos incluem "id_pedido", "cliente" (com detalhes do cliente, como "id_cliente", "nome" e "endereco"), "produtos" (uma lista que contém o "id_produto", "nome_produto", "quantidade" e "preco_unitario" para cada item comprado), além de "data_pedido" e "valor_total". Fim da descrição.

Vantagens e desvantagens dos bancos de dados relacionais

Cada tipo de banco de dados tem as próprias vantagens, dependendo das necessidades do sistema (Khan *et al.*, 2023). Entre os principais benefícios dos bancos de dados relacionais, estão:

CONSISTÊNCIA DOS DADOS:

as relações entre dados são mantidas com precisão, evitando inconsistências e duplicidade.

INTEGRIDADE REFERENCIAL:

o uso de chaves primárias e estrangeiras assegura que os dados estejam corretamente relacionados entre diferentes tabelas.

SUORTE A TRANSAÇÕES ACID:

há transações seguras e confiáveis, garantindo que todas as operações sejam concluídas ou revertidas corretamente.

AMPLO SUPORTE NO MERCADO:

existe uma vasta gama de ferramentas e sistemas de gerenciamento de bancos de dados SQL amplamente suportados no mercado.

ESTRUTURA BEM DEFINIDA:

a organização dos dados em tabelas estruturadas facilita a manutenção e o acesso aos dados, sendo ideal para sistemas estáveis e que requerem alto controle de integridade.

Apesar das vantagens, os bancos de dados relacionais apresentam algumas limitações (Khan *et al.*, 2023). Eles enfrentam desafios na **escalabilidade horizontal**, que se refere à capacidade de aumentar o desempenho distribuindo a carga entre vários servidores. Esse modelo pode não ser o mais eficiente para sistemas que precisam lidar com grandes volumes de dados em plataformas distribuídas ou aplicações web de grande escala.

Além disso, a **rigidez do esquema** pode ser uma desvantagem em cenários em que os dados mudam frequentemente ou têm estruturas variáveis. Nos bancos de dados relacionais, é necessário definir previamente a estrutura das tabelas (colunas e tipos de dados), o que torna o gerenciamento de dados menos flexível em sistemas que exigem adaptação contínua.

Por fim, esses bancos de dados não são ideais para gerenciar grandes volumes de dados não estruturados, como vídeos, imagens ou documentos. Dados não estruturados não se encaixam bem no modelo tabular tradicional dos bancos SQL, sendo mais adequados para bancos NoSQL, que oferecem maior flexibilidade de armazenamento.

Vantagens e desvantagens dos bancos de dados não relacionais

Os bancos de dados não relacionais (NoSQL) também têm vantagens e desvantagens (Khan *et al.*, 2023). Algumas das principais vantagens incluem:

ESCALABILIDADE HORIZONTAL:

facilitam a distribuição de dados em múltiplos servidores, permitindo que o sistema cresça facilmente conforme a demanda aumenta.

FLEXIBILIDADE NO MODELO DE DADOS:

adaptam-se bem a dados não estruturados ou semi-estruturados, como documentos JSON, imagens, vídeos e metadados, sem a necessidade de um esquema rígido.

DESEMPENHO EM GRANDES VOLUMES:

são otimizados para manipular grandes quantidades de dados com baixa latência, tornando-os ideais para sistemas que precisam de respostas rápidas e dados em tempo real.

ARMAZENAMENTO DINÂMICO:

permitem que diferentes tipos de dados sejam armazenados com estruturas variáveis, sem a necessidade de redefinir o esquema do banco de dados.

ADEQUAÇÃO PARA APLICAÇÕES MODERNAS:

são amplamente utilizados em redes sociais, plataformas de streaming, sistemas de recomendação e outras aplicações web de grande escala.

Embora ofereçam flexibilidade e escalabilidade, os bancos de dados NoSQL também apresentam algumas desvantagens. Eles não garantem a mesma consistência dos bancos de dados relacionais, o que pode ser um problema em aplicações que exigem transações rigorosas e seguras.

Além disso, a **ausência de um esquema rígido** pode resultar em dados desorganizados ou inconsistentes quando não gerenciados adequadamente. Isso pode complicar a governança dos dados em grandes sistemas.

Por fim, a **complexidade de manutenção** de um sistema NoSQL em larga escala pode se tornar um desafio, especialmente quando se trata da integração com ferramentas tradicionais e suporte técnico especializado.



PRINCIPAIS BANCOS DE DADOS USADOS NO MERCADO

Agora que entendemos os fundamentos e as diferenças entre os bancos de dados relacionais e não relacionais, vamos focar nos principais Sistemas de Gerenciamento de Bancos de Dados Relacionais (SGBDR), amplamente utilizados no mercado (Romanowski, 2020).

Os **SGBDRs** são ferramentas que auxiliam na organização, armazenamento e manipulação de dados estruturados em tabelas inter-relacionadas, seguindo o modelo tradicional proposto por Edgar F. Codd. Esses sistemas utilizam a linguagem SQL para gerenciar e consultar os dados de forma eficiente. Os SGBDRs mais populares são:

MYSQL:

banco de dados relacional open-source, popular por sua simplicidade e eficiência. Amplamente usado em aplicações web e oferecido pela Oracle.

POSTGRESQL:

banco de dados open-source robusto, com funcionalidades avançadas. Ideal para empresas que precisam de flexibilidade e escalabilidade.

ORACLE DATABASE:

usado em grandes corporações, é conhecido por confiabilidade, segurança e suporte a grandes volumes de dados, especialmente em finanças.

MICROSOFT SQL SERVER:

amplamente usado em ambientes corporativos Microsoft, oferece integração nativa com outros produtos da empresa e alta confiabilidade.

Dependendo das necessidades específicas do projeto, é possível escolher entre opções, como MySQL, PostgreSQL, Oracle Database e Microsoft SQL Server, cada um com características e vantagens. O conhecimento desses sistemas e da linguagem SQL é essencial para qualquer profissional que deseja trabalhar com a gestão de dados no mercado atual.

**EM FOCO**

Acesse o seu Ambiente Virtual de Aprendizagem e confira a aula referente a este tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

Ao longo deste conteúdo, você teve a oportunidade de explorar a fundo o conceito de bancos de dados relacionais e como eles se tornaram a base para a gestão eficiente de informações em diversas organizações. Agora que estamos nos aproximando do fim desta jornada, é o momento de conectar a teoria à prática, destacando como esses conhecimentos se aplicam diretamente ao ambiente profissional que você encontrará em sua carreira.

No mercado de trabalho, a habilidade de gerenciar grandes volumes de dados de forma organizada é crucial para a tomada de decisões em tempo real. Empresas que operam em setores, como e-commerce, finanças e telecomunicações, dependem de bancos de dados relacionais para garantir que as informações estejam disponíveis de maneira rápida e precisa.

Os desafios enfrentados, como os problemas de lentidão ou a necessidade de escalabilidade, são comuns em qualquer infraestrutura de TI, e é nesse contexto que a sua capacidade de aplicar o que aprendeu fará toda a diferença. Identificar gargalos de performance e propor soluções eficientes é uma competência valiosa e muito procurada.

Além disso, com o avanço das tecnologias e o aumento da complexidade dos dados, é cada vez mais importante que os profissionais saibam quando utilizar bancos de dados relacionais e quando adotar soluções não relacionais, como NoSQL. Em muitas situações, a solução mais eficaz não será aquela que segue um modelo único, mas uma combinação de diferentes abordagens. O mercado exige profissionais que consigam avaliar as necessidades de cada projeto e escolher a tecnologia mais adequada, garantindo flexibilidade e otimização.

Com base no que você aprendeu, agora, é hora de pensar em como você pode começar a aplicar essas habilidades em seu futuro profissional. Quais problemas você será capaz de resolver? Como as empresas poderão se beneficiar do seu conhecimento? Esses são os desafios que você encontrará, mas, com a base teórica e prática que adquiriu, estará preparado(a) para enfrentá-los com confiança.



VAMOS PRATICAR

1. Inovações importantes moldaram o desenvolvimento e a popularização dos bancos de dados relacionais (SQL). O surgimento de uma linguagem padrão para interagir com esses bancos, além do início da comercialização dos sistemas de gerenciamento, foram eventos que consolidaram o uso dessa tecnologia em larga escala.

Com base nos eventos históricos sobre a evolução dos bancos de dados relacionais, assinale a alternativa que descreve corretamente um marco importante dessa trajetória:

- a) A comercialização de sistemas de bancos de dados relacionados, como Oracle Database e IBM DB2, que ocorreu na década de 1970.
 - b) Em 1980, Edgar Codd publicou um artigo sobre o modelo relacional, revolucionando a maneira como os dados eram acessados.
 - c) A linguagem SQL foi desenvolvida pela IBM em 1974, sendo amplamente exigida como a linguagem padrão para bancos de dados relacionais.
 - d) O modelo relacional foi implementado pela primeira vez em 1990, quando a Oracle lançou o primeiro sistema comercial de banco de dados.
 - e) A separação entre a organização lógica e o armazenamento físico dos dados foi possível devido ao surgimento dos bancos de dados hierárquicos.
2. Nos bancos de dados relacionais, as informações são armazenadas em tabelas compostas por colunas e linhas. Para garantir a integridade e a organização dos dados, são utilizadas chaves primárias e chaves estrangeiras. Cada tabela tem uma estrutura específica, facilitando o gerenciamento eficiente dos dados e as operações de busca e manipulação.

Com base no funcionamento dos bancos de dados relacionais, assinale a alternativa que interprete corretamente o uso de tabelas, colunas, chaves primárias e estrangeiras, e a respectiva importância na organização dos dados:

- a) Uma tabela pode conter várias chaves primárias para garantir que cada coluna tenha dados exclusivos e individuais de cada registro.
- b) As chaves primárias e estrangeiras são responsáveis por conectar diferentes tabelas dentro de um banco de dados relacional.
- c) Uma chave estrangeira é utilizada para garantir que os valores de uma coluna sejam únicos em cada linha, identificando exclusivamente os registros.
- d) As linhas em uma tabela relacional são responsáveis por definir o tipo de dado armazenado, enquanto as colunas contêm os dados individuais de cada registro.
- e) No modelo relacional, as colunas definem os tipos de dados armazenados, como "Nome", enquanto as linhas contêm os dados correspondentes de cada registro.

VAMOS PRATICAR

3. Nos sistemas de bancos de dados relacionais, diferentes mecanismos garantem a integridade e a consistência dos dados, especialmente quando há múltiplas tabelas conectadas. A forma como essas tabelas se relaciona entre si, garantindo que os dados sejam mantidos de maneira lógica e organizada, é essencial para o bom funcionamento de sistemas que lidam com grandes volumes de informações.

Com base no papel das chaves primárias e estrangeiras em um banco de dados relacional, assinale a alternativa que interpreta corretamente a função da chave estrangeira:

- I - A chave estrangeira garante a exclusividade dos valores na coluna onde é aplicada.
- II - A chave estrangeira mantém a integridade referencial, vinculando registros entre duas tabelas.
- III - A chave estrangeira permite que os registros sejam duplicados em uma tabela, desde que as chaves primárias sejam diferentes.
- IV - A chave estrangeira é responsável por referenciar corretamente a chave primária de outra tabela, criando uma conexão entre ambas.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

REFERÊNCIAS

CODD, E. F. A Relational Model of Large Shared Data Banks (1970). *In*: LEWIS, H. R. (ed.). **Ideas That Created the Future**. Classic Papers of Computer Science. Massachusetts: The MIT Press, 2021.

KHAN, W. *et al.* SQL and NoSQL Database Software Architecture Performance Analysis and Assessments — A Systematic Literature Review. **Big Data and Cognitive Computing**, v. 7, n. 2, 2023.

RICARDO, C. M.; URBAN, S. D.; DAVIS, K. C. **Databases illuminated**. Burlington: Jones & Bartlett Learning, 2022.

ROMANOWSKI, J. The Most Popular Databases in 2020. **LearnSQL**, 2020. Disponível em: <https://learnsql.com/blog/most-popular-sql-databases-2020/>. Acesso em: 6 nov. 2024.

CONFIRA SUAS RESPOSTAS

1. Opção C.

A comercialização de sistemas de bancos de dados relacionados começou na década de 1980, e não nos anos de 1970. O artigo de Edgar Codd foi publicado na década de 1970, e não em 1980. A linguagem SQL foi desenvolvida em 1974 pela IBM para interagir com bancos de dados relacionados. O modelo relacional foi comercializado na década de 1980, e não em 1990. A separação entre a organização lógica e o armazenamento físico foi uma funcionalidade modificada pelo modelo relacional, e não pelos bancos de dados hierárquicos.

2. Opção E.

Uma tabela pode ter apenas uma chave primária, que é a responsável por identificar unicamente cada registro. Apenas a chave estrangeira conecta tabelas diferentes, enquanto a chave primária identifica apenas os registros dentro de uma mesma tabela. A chave estrangeira relaciona tabelas diferentes, e não unicidade: essa função pertence à chave primária. As linhas armazenam os dados dos registros, enquanto as colunas definem os tipos de dados, como "Nome" ou "Telefone". As colunas definem o tipo de dado, e cada linha armazena os dados de um registro específico.

3. Opção B.

A exclusividade dos valores é uma função da chave primária. A chave estrangeira garante os dados entre duas tabelas relacionadas de maneira consistente. A chave estrangeira não tem relação com a duplicação de registros, mas com a integridade referencial. A chave estrangeira estabelece uma relação com a chave primária de outra tabela, garantindo a conexão entre os dados.



unidade



TEMA DE APRENDIZAGEM 2

TRANSAÇÕES E CONTROLE DE CONCORRÊNCIA

MINHAS METAS

- Compreender o conceito de transações em bancos de dados.
- Explorar as propriedades ACID em uma transação.
- Diferenciar atomicidade, consistência, isolamento e durabilidade.
- Identificar problemas de concorrência em bancos de dados.
- Aplicar o controle de concorrência em transações.
- Entender o isolamento de transações e os respectivos níveis.
- Analisar exemplos de uso de transações em sistemas reais.

INICIE SUA JORNADA

Suponha que você trabalha no departamento de Tecnologia da Informação (TI) de uma empresa de *e-commerce* de grande porte. Durante a **Black Friday**, o número de acessos ao sistema atinge **picos inesperados**, com milhares de pedidos sendo processados simultaneamente. De repente, começam a surgir problemas: um cliente relata que foi cobrado duas vezes por um pedido, enquanto outro afirma que o pedido desapareceu do sistema. A equipe percebe que os problemas estão relacionados ao controle de transações e à concorrência no acesso aos dados. O que você faria para solucionar essa situação?

Garantir que o sistema possa lidar com **múltiplas transações** ao mesmo tempo, sem comprometer a integridade dos dados, é **essencial** para a operação de uma empresa em momentos de alta demanda. Controlar a concorrência entre transações e assegurar que as propriedades ACID sejam seguidas pode evitar falhas, como cobranças duplicadas ou pedidos perdidos. Resolver esse tipo de problema protege a experiência do cliente e assegura que a empresa não perca vendas importantes, minimizando o risco de impactos financeiros negativos e mantendo o sistema operacional estável.

Como parte da equipe de TI, o seu desafio será **entender a fundo e implementar mecanismos eficientes de controle de transações e concorrência**. Isso inclui a garantia de que todas as operações dentro de uma transação ocorram de maneira atômica, mantendo a consistência dos dados, mesmo em cenários de falhas ou acessos concorrentes. Você também precisará aplicar técnicas de controle de concorrência, como bloqueios e níveis de isolamento, para evitar que múltiplas transações causem conflitos ou dados inconsistentes. A adoção de um bom planejamento de controle de concorrência pode garantir que o sistema funcione corretamente mesmo em picos de uso intenso, evitando falhas críticas e mantendo a confiança dos clientes na plataforma.

Ao refletir sobre essa questão, você perceberá que, em ambientes de alta concorrência, a gestão de transações é fundamental para evitar erros críticos. Dominar esses conceitos e aplicá-los de maneira eficaz não só resolverá problemas técnicos, mas também será uma habilidade valiosa para garantir a estabilidade e o desempenho de grandes sistemas em tempo real.

**PLAY NO CONHECIMENTO**

Para aprofundar ainda mais o seu entendimento sobre controle de concorrência, que tal ouvir o nosso podcast? Nele, discutimos técnicas avançadas, como MVCC e *Timestamp Ordering*, incluindo vantagens, desafios e aplicações práticas. Não perca essa oportunidade de complementar os seus estudos! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Que tal revisar um conceito importante antes de avançar? Assista ao vídeo *Você sabe a diferença entre Banco de Dados e SGBD?*, do canal CreateSe, e entenda as distinções fundamentais entre um banco de dados e um Sistema de Gerenciamento de Banco de Dados (SGBD) que são essenciais para uma compreensão completa do tema que será abordado. https://br.freepik.com/vetores-gratis/conceito-de-criptomoeda-desenhado-a-mao-com-cadeado_20059349.htm#fromView=search&page=1&position=23&uuid=5098241f-85e1-4984-bcod-037c14009591

DESENVOLVA SEU POTENCIAL

CONCEITO DE TRANSAÇÕES

Em um banco de dados, uma transação é uma unidade lógica de trabalho que executa uma ou mais operações de leitura ou escrita em dados. O conceito de transação é fundamental para garantir a integridade dos dados e a execução correta das operações em sistemas de bancos de dados, especialmente em ambientes concorrentes, onde múltiplos usuários ou processos acessam o mesmo conjunto de dados ao mesmo tempo (Silberschatz; Korth; Sudarshan, 2020).

Nos próximos tópicos, exploraremos com mais detalhes o que é uma transação, a importância dela para a manutenção da integridade dos dados e o modo como as transações são utilizadas em sistemas reais.

Definição de transação em banco de dados

Uma transação em um banco de dados pode ser definida como uma sequência de operações realizadas como uma única unidade de trabalho que deve ser completamente executada ou completamente revertida. Em outras palavras, uma transação deve ser **atômica**, ou seja, ou todas as operações da transação são concluídas com sucesso, ou nenhuma delas será aplicada, deixando o banco de dados no estado anterior à transação (Silberschatz; Korth; Sudarshan, 2020).

As transações são essenciais, porque garantem que o banco de dados permaneça em um estado **consistente**, mesmo em situações de falhas ou interrupções no sistema. Elas permitem que grupos de operações, como **inserções**, **atualizações** ou **exclusões** de dados, sejam tratadas como uma única operação. Isso evita que alterações parciais sejam aplicadas, o que poderia corromper os dados (Elmasri; Navathe, 2019).

Importância das transações para a integridade dos dados

A principal função das transações é garantir que os dados de um banco de dados permaneçam consistentes e íntegros, mesmo em casos de falhas no sistema, quedas de energia ou múltiplos acessos concorrentes (Ricardo; Urban; Davis, 2022). Transações são particularmente importantes em sistemas críticos, como bancos ou *e-commerce*, já que são vitais para que todas as operações sejam realizadas corretamente.

Por exemplo, considere uma transação que envolva a **transferência de fundos** entre duas contas bancárias. As operações de débito e crédito devem acontecer de maneira atômica — ou ambas são realizadas, ou nenhuma. Se o sistema falhar após debitar a conta de origem, antes de creditar a conta de destino, sem o uso de transações, os dados ficariam **corrompidos**. Portanto, as transações garantem que, em caso de falha, o banco de dados volte ao estado original, preservando a **integridade** dos dados (Elmasri; Navathe, 2019).

Além disso, as transações garantem que as operações sigam as propriedades **ACID** (que serão abordadas na seção seguinte), garantindo confiabilidade no tratamento dos dados.

Exemplos de uso de transações em sistemas

Transações são amplamente usadas em sistemas de todos os tipos, especialmente em aplicações financeiras, *e-commerce* e gestão de inventários. Veja a seguir alguns exemplos de uso prático:

TRANSFERÊNCIA BANCÁRIA:

quando um cliente transfere dinheiro de uma conta para outra, o sistema realiza uma transação que debita o valor da conta de origem e o credita na conta de destino. Se qualquer uma dessas etapas falhar, toda a transação é desfeita.

PROCESSAMENTO DE PEDIDOS EM *E-COMMERCE*:

em uma plataforma de vendas on-line, uma transação é usada para garantir que o estoque seja atualizado corretamente ao processar um pedido. Se o pagamento for aprovado, o sistema deve reduzir o estoque e confirmar o pedido. Caso qualquer parte do processo falhe, a transação é revertida.

ATUALIZAÇÃO DE INVENTÁRIO:

empresas que lidam com grandes volumes de estoque usam transações para garantir que as entradas e saídas de produtos sejam atualizadas corretamente. Isso evita inconsistências, como a contagem errada de produtos em casos de falhas durante a atualização do banco de dados.

Transações, portanto, não só garantem a integridade dos dados, mas também melhoram a confiabilidade e a robustez dos sistemas, proporcionando uma experiência mais segura e previsível para os usuários finais.

PROPRIEDADES ACID

As transações em bancos de dados são regidas por quatro propriedades fundamentais, conhecidas pela sigla ACID: **Atomicidade**, **Consistência**, **Isolamento** e **Durabilidade**. A Figura 1 ilustra essas propriedades.

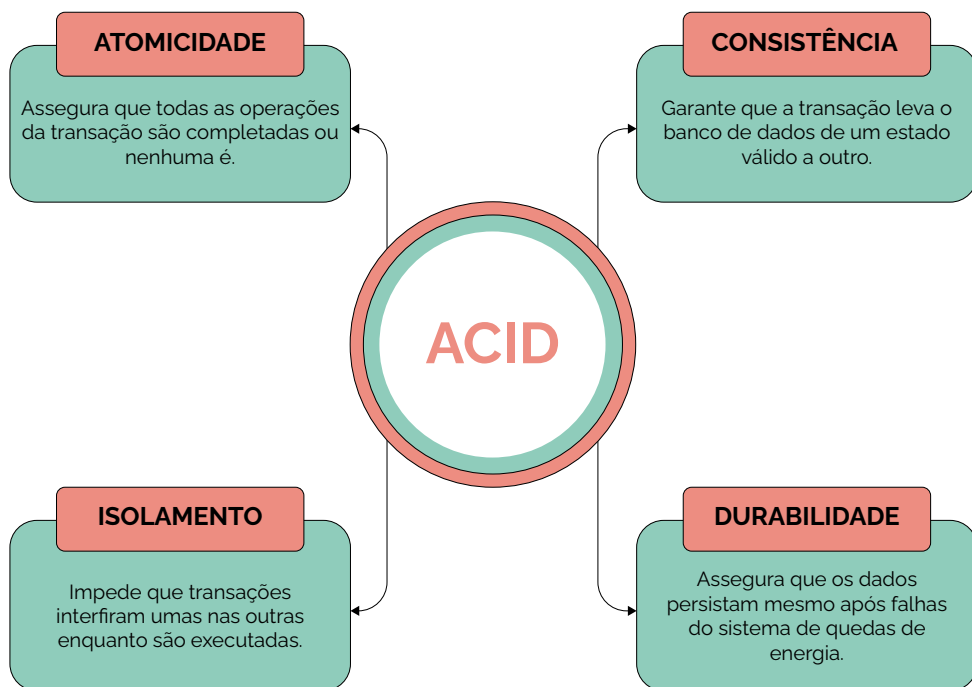


Figura 1 - Propriedades ACID / Fonte: o autor.

Descrição da Imagem: é ilustrado um diagrama das quatro propriedades fundamentais das transações em bancos de dados, conhecidas como ACID. Cada uma das propriedades – Atomicidade, Consistência, Isolamento e Durabilidade – está representada com uma definição resumida. Essas propriedades, juntas, garantem a confiabilidade e a integridade das transações em sistemas de bancos de dados. Fim da descrição.

Essas propriedades asseguram que as transações sejam processadas de maneira confiável e que o banco de dados mantenha a integridade, mesmo diante de falhas ou concorrência entre múltiplos usuários.

Vamos entender o que cada uma dessas propriedades significa e como elas são aplicadas no gerenciamento de transações em um banco de dados.

Atomicidade

A **atomicidade** garante que todas as operações em uma transação sejam tratadas como uma única unidade. Em outras palavras, ou todas as operações da transação são concluídas com sucesso, ou nenhuma delas será aplicada ao banco de dados.

Se ocorrer uma falha durante a execução da transação, ela precisa ser **desfeita** por completo, retornando o banco de dados ao estado anterior.

Por exemplo, em uma transação de transferência bancária, o débito de uma conta e o crédito em outra devem ser feitos juntos. Se o sistema falhar após o débito, mas antes do crédito, a **atomicidade** garante que o débito também seja revertido, evitando inconsistências nos dados.

Em resumo, a atomicidade **protege** o banco de dados contra **transações incompletas**, garantindo que nenhuma alteração parcial seja aplicada.

A atomicidade protege o banco de dados contra transações incompletas

Consistência

A propriedade de **consistência** assegura que a transação leve o banco de dados de um estado consistente para outro estado consistente, respeitando todas as **regras e restrições** definidas, como integridade referencial, validações de tipos de dados e outras restrições de integridade.

Por exemplo, se um **sistema de inventário** tem a regra de que não pode haver produtos com **quantidade negativa**, a consistência garante que, após uma transação, essa regra permaneça válida. Se uma transação tentasse violar essa regra (como ao permitir que o estoque ficasse com um valor negativo), a transação falharia, e o banco de dados não seria alterado.

A consistência, portanto, **protege** o banco de dados contra **alterações inválidas**, mantendo a integridade dos dados mesmo diante de transações falhas.

Isolamento e durabilidade

A propriedade de **isolamento** garante que as transações ocorram de forma independente umas das outras. Em outras palavras, se duas transações estão sendo executadas ao mesmo tempo, os resultados de uma não devem interferir na outra. O isolamento evita que os efeitos de uma transação parcial fiquem visíveis para outras transações em andamento.

Por exemplo, em um **sistema de reservas de passagens**, se dois usuários tentarem comprar o último bilhete ao mesmo tempo, o isolamento garante que

apenas uma transação seja concluída com sucesso. A outra será revertida, e o usuário será informado que o bilhete já foi vendido.

Por outro lado, a **durabilidade** garante que, uma vez que uma transação seja confirmada (ou **committed**), as alterações dela no banco de dados persistam, mesmo em caso de falhas no sistema, como quedas de energia. Isso significa que, após a confirmação, os dados são permanentemente gravados e protegidos contra falhas futuras.

Se pensarmos em um **sistema de pagamento on-line**, a durabilidade garante que, depois que um pagamento é aprovado, essa informação será **salva** e **permanecerá acessível** mesmo se houver uma interrupção no serviço logo após a transação.

CONTROLE DE CONCORRÊNCIA

Em ambientes de banco de dados onde **múltiplos usuários** ou **processos** realizam transações simultaneamente, pode haver situações de conflito ou inconsistência nos dados.

O **controle de concorrência** refere-se ao conjunto de técnicas e mecanismos que garantem que várias transações possam ocorrer ao mesmo tempo sem comprometer a integridade e a consistência do banco de dados (MariaDB, 2024).

Nesta seção, vamos explorar os problemas comuns causados pela concorrência e as principais técnicas utilizadas para gerenciar transações simultâneas de maneira eficiente.

Problemas de concorrência em bancos de dados

Quando várias transações ocorrem simultaneamente, elas podem acessar ou modificar os mesmos dados ao mesmo tempo, gerando problemas de concorrência. Alguns dos problemas mais comuns incluem:

LEITURA SUJA (*DIRTY READ*):

ocorre quando uma transação lê dados que foram modificados por outra transação que ainda não foi confirmada. Se essa outra transação falhar ou for revertida, os dados lidos estarão incorretos.

LEITURA NÃO REPETÍVEL (*NON-REPEATABLE READ*):

acontece quando uma transação lê os mesmos dados duas vezes e obtém resultados diferentes, porque outra transação os modificou no meio do processo.

FANTASMAS (*PHANTOM READ*):

esse problema surge quando uma transação executa uma consulta e outra transação insere, deleta ou modifica linhas que afetariam o resultado da consulta original.

Esses problemas de concorrência podem comprometer a integridade dos dados, levando a resultados inconsistentes e operações falhas. Felizmente, existem soluções, como os níveis de isolamento e técnicas específicas de controle de concorrência, que ajudam a coordenar o acesso simultâneo aos dados, garantindo a consistência das operações.

Técnicas de controle de concorrência

Para evitar esses problemas, os sistemas de banco de dados implementam várias técnicas de controle de concorrência. As principais incluem:

1. **Bloqueios (*Locks*):** o mecanismo de bloqueio impede que duas transações acessem o mesmo dado simultaneamente. Existem dois tipos principais de bloqueios:
 - **Bloqueio Exclusivo (*Exclusive Lock*):** permite que apenas uma transação modifique os dados, impedindo que outras transações os leiam ou modifiquem ao mesmo tempo.
 - **Bloqueio Compartilhado (*Shared Lock*):** permite que várias transações leiam os dados ao mesmo tempo, mas impede que qualquer uma delas os modifique.

2. **Controle de Concorrência Otimista:** ao contrário de bloquear o acesso aos dados, essa técnica permite que várias transações acessem os mesmos dados simultaneamente, mas verifica se houve conflitos ao final da transação. Se um conflito for detectado, a transação é revertida.
3. **Controle de Concorrência Baseado em *Timestamp*:** cada transação recebe um *timestamp* (marca temporal) ao ser iniciada. O banco de dados usa esses *timestamps* para garantir que as transações sejam executadas de acordo com a ordem cronológica, evitando conflitos.

Essas técnicas garantem que as transações concorrentes sejam gerenciadas de forma eficiente, preservando a consistência dos dados e evitando problemas, como leituras sujas ou não repetíveis.

Isolamento de transações

O **isolamento** é uma das propriedades ACID que já discutimos e desempenha um papel crucial no controle de concorrência. O nível de isolamento determina até que ponto uma transação pode ser “vista” por outras transações enquanto ainda está em andamento.

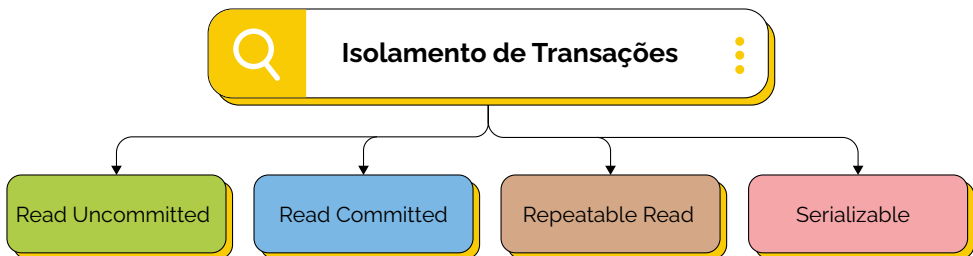


Figura 2 - Isolamento de transações / Fonte: o autor.

Descrição da Imagem: é ilustrado um diagrama dos níveis de isolamento de transações em bancos de dados. Cada nível define o grau de isolamento entre as transações concorrentes e como elas podem interferir umas nas outras. Os quatro níveis representados são: Read Uncommitted, Read Committed, Repeatable Read e Serializable. Fim da descrição.

Existem quatro níveis principais de isolamento, cada um equilibrando desempenho e consistência de diferentes maneiras:

1. ***Read Uncommitted (Leitura Não Confirmada)***: o nível mais baixo de isolamento, no qual as transações podem ler dados que ainda não foram confirmados por outras transações, permitindo leituras sujas. Esse nível oferece o melhor desempenho, mas com riscos de inconsistência, sendo mais comum em sistemas nos quais a precisão dos dados não é crítica.
2. ***Read Committed (Leitura Confirmada)***: garante que uma transação só possa ler dados confirmados, evitando leituras sujas, mas permitindo leituras não repetíveis. Isso mantém a consistência em um nível aceitável, sendo amplamente utilizado em muitos sistemas de *e-commerce* e transações financeiras.
3. ***Repeatable Read (Leitura Repetível)***: garante que uma transação sempre leia os mesmos dados, evitando leituras não repetíveis, mas ainda permitindo o problema de fantasmas. É comum em sistemas nos quais é importante que as leituras permaneçam consistentes durante a execução, como em sistemas de inventário.
4. ***Serializable (Serializável)***: o nível mais alto de isolamento, no qual as transações são executadas de forma que os resultados sejam equivalentes a uma execução serial (uma transação após a outra), eliminando todos os problemas de concorrência. Embora ofereça a maior consistência, tem um custo maior de desempenho, sendo normalmente usado em transações de sistemas financeiros críticos.

A escolha do nível de isolamento depende das necessidades específicas do sistema. Em muitos casos, há um equilíbrio entre garantir a consistência dos dados e maximizar o desempenho.

Além dos níveis de isolamento, técnicas mais avançadas, como o MVCC (*Multiversion Concurrency Control*) e o *Timestamp Ordering*, são alternativas amplamente utilizadas em bancos de dados modernos para gerenciar transações concorrentes de forma ainda mais eficiente.

Essas técnicas permitem que várias transações acessem os mesmos dados simultaneamente, sem depender exclusivamente de bloqueios, e garantem consistência sem sacrificar o desempenho.

 EM FOCO

Confira a aula referente a este tema. Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.

NOVOS DESAFIOS

Ao longo deste tema de aprendizagem, exploramos o conceito de transações e o papel crucial que elas desempenham para garantir a integridade e a consistência dos dados em um sistema de banco de dados. Também mergulhamos nas propriedades ACID, as quais asseguram que as transações sejam processadas de forma confiável, e nas técnicas de controle de concorrência, que ajudam a gerenciar transações simultâneas em ambientes multiusuário. Agora que você compreendeu esses fundamentos, é o momento de refletir sobre como tudo isso se conecta ao seu futuro profissional.

No mercado de trabalho, a capacidade de lidar com múltiplas transações simultâneas sem comprometer a integridade dos dados é um dos grandes desafios em ambientes de TI. Sistemas, como os utilizados em bancos, *e-commerces* e outros setores críticos, dependem de profissionais que saibam implementar mecanismos de controle de concorrência de forma eficiente, garantindo que os dados permaneçam seguros e consistentes, mesmo em condições de alto volume de acessos. Resolver problemas, como conflitos de transações ou garantir a durabilidade dos dados após falhas, é uma habilidade valiosa e altamente requisitada no setor.

Além disso, com a crescente demanda por sistemas escaláveis e distribuídos, entender como as propriedades ACID podem ser aplicadas (ou adaptadas) em diferentes contextos, como em bancos de dados distribuídos ou em ambientes de grande volume de dados, é fundamental para a sua carreira. Ser capaz de identificar e solucionar problemas relacionados às transações e à concorrência será essencial para desenvolver sistemas robustos que suportem muitos usuários sem perder a eficiência ou a confiabilidade.

Agora que você domina os conceitos apresentados, como você acha que pode aplicá-los em cenários reais? O gerenciamento eficiente de transações, aliado ao controle preciso da concorrência e à garantia da integridade dos dados, será um diferencial significativo em qualquer área que você decidir atuar. Esses conhecimentos e habilidades são essenciais para desenvolver sistemas robustos e confiáveis, que operem sem falhas mesmo em ambientes desafiadores e de grande escala.

VAMOS PRATICAR

1. Em sistemas críticos, como os utilizados por bancos ou *e-commerce*, é essencial garantir que os dados permaneçam íntegros e consistentes, mesmo diante de falhas no sistema ou intermitentes. As transações em um banco de dados têm um papel crucial para garantir que as operações, como transferências bancárias, sejam concluídas com segurança, sem o risco de corromper os dados, especialmente em caso de falhas.

Com base nas funções das transações em banco de dados, selecione a alternativa que melhor explica a importância das transações para garantir a integridade dos dados:

- a) Uma transação só se aplica em sistemas distribuídos e não é necessária em operações simples, como transferências bancárias.
 - b) As transações garantem que os dados sejam sempre salvos, mesmo em casos de falhas graves, sem necessidade de reversão.
 - c) Uma transação permite que múltiplas transações sejam realizadas, mas, se uma etapa falhar, o sistema mantém as alterações já aplicadas.
 - d) As transações garantem que apenas a operação mais importante dentro de um conjunto de operações seja concluída, independentemente das outras.
 - e) Uma transação é uma operação única e atômica que garante que todas as etapas sejam concluídas, ou o sistema reverte qualquer alteração realizada.
2. As transações em sistemas de bancos de dados são amplamente utilizadas para garantir a integridade e a consistência dos dados, especialmente em setores críticos, como bancos e *e-commerce*. Essas transações garantem que várias operações sejam concluídas de forma segura, mesmo em casos de falhas, evitando a corrupção dos dados.

Considerando o modo como as transações funcionam em sistemas financeiros ou de *e-commerce*, analise as afirmativas a seguir:

- I - Em uma transação de *e-commerce*, se o estoque for atualizado, mas o pagamento não for processado, o pedido será mantido pendente até que uma falha seja corrigida.
- II - Em um sistema de *e-commerce*, se o pagamento falhar, o pedido não será concluído, e o estoque não será atualizado. A transação será revertida para garantir a integridade dos dados.
- III - Em uma transação bancária, o valor só é transferido da conta de origem para a conta de destino se ambas as operações (débito e crédito) forem realizadas com sucesso. Se houver falha em qualquer uma das etapas, a transação será cancelada.

É correto o que se afirma em:

VAMOS PRATICAR

- a) I, apenas.
 - b) III, apenas.
 - c) I e II, apenas.
 - d) II e III, apenas.
 - e) I, II e III.
3. Em sistemas de banco de dados, a atomicidade é um dos princípios fundamentais que garante que todas as operações de uma transação sejam tratadas como uma única unidade de trabalho. Isso significa que, se qualquer parte da transação falhar, nenhuma das operações será aplicada ao banco de dados, e o sistema retornará ao estado anterior à transação. Esse princípio é essencial para garantir a integridade dos dados, especialmente em operações críticas, como transferências bancárias.

Com base no princípio da atomicidade, assinale a alternativa que exhibe o modo como uma transação bancária deve ser tratada em caso de falha durante a operação:

- a) Se o débito em uma conta for realizado, mas o crédito na conta de destino falhar, a transação será revertida, e o valor debitado será restaurado.
- b) Se ocorrer uma falha durante a execução de uma transação, apenas as operações que foram concluídas com sucesso até aquele ponto serão aplicadas.
- c) A atomicidade permite que partes de uma transação sejam aplicadas ao banco de dados, desde que a falha ocorra após uma operação importante, como o subsídio.
- d) Em uma transferência bancária, se o débito for bem-sucedido, ele será suspenso, mesmo que o crédito falhe, e o sistema aguardará até que o crédito possa ser processado.
- e) No caso de uma transferência bancária, se o crédito não puder ser realizado imediatamente, a atomicidade permite que o débito seja confirmado e o crédito seja processado assim que possível.

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2019.

MARIADB. ACID: Concurrency Control with Transactions. **MariaDB Knowledge Base**, 2024. Disponível em: <https://mariadb.com/kb/en/acid-concurrency-control-with-transactions/>. Acesso em: 7 nov. 2024.

RICARDO, C. M.; URBAN, S. D.; DAVIS, K. C. **Databases Illuminated**. 4. ed. Burlington: Jones & Bartlett Learning, 2022.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistemas de banco de dados**. 7. ed. Rio de Janeiro: Elsevier, 2020.

CONFIRA SUAS RESPOSTAS

1. Opção E.

As transações são fundamentais tanto em sistemas distribuídos quanto em operações críticas, como transferências bancárias, para garantir a consistência dos dados. As transações garantem que, em caso de falhas, os dados retornem ao estado anterior à transação, revertendo quaisquer alterações realizadas. Uma transação deve garantir que, em caso de falha, todas as alterações sejam revertidas, não permitindo que alterações parciais sejam mantidas. Todas as operações dentro de uma transação são tratadas igualmente e devem ser concluídas para que a transação seja bem concluída.

2. Opção D.

Uma transação precisa ser completamente bem-sucedida. Se o pagamento falhar, a atualização do estoque e a criação do pedido também serão desfeitas. Se o pagamento falhar, a transação inteira, incluindo a atualização do estoque e a confirmação do pedido, será cancelada. Além disso, tanto o subsídio quanto o crédito precisam ser bem-sucedidos para que a transação seja concluída. Caso contrário, ela é revertida.

3. Opção A.

A atomicidade garante que, se uma operação falhar, todas as alterações anteriores sejam revertidas, preservando a consistência dos dados. O princípio da atomicidade garante que, em caso de falha, nenhuma operação parcial seja mantida. Ou todas as operações são concluídas, ou nenhuma delas é aplicada. A atomicidade não permite a aplicação parcial de operações. Se qualquer transação ocorrer, ela será totalmente revertida. A atomicidade garante que o subsídio e o crédito ocorram juntos, ou nenhum deles será realizado. Não há confirmação de uma parte enquanto se aguarda a conclusão da outra.



TEMA DE APRENDIZAGEM 3

MODELAGEM DE DADOS

MINHAS METAS

- Compreender os fundamentos das modelagens conceitual, lógica e física.
- Identificar e definir entidades e os respectivos atributos.
- Diferenciar os tipos de relacionamentos entre entidades.
- Aplicar corretamente os tipos de relacionamento em modelos.
- Desenvolver diagramas de entidade-relacionamento (DER).
- Construir modelos entidade-relacionamento (MER) detalhados.
- Analisar a adequação dos modelos à estrutura do banco de dados.

INICIE SUA JORNADA

Imagine que você acaba de iniciar o trabalho em uma *startup* como desenvolvedor(a) júnior. Em seu primeiro projeto, a empresa está criando uma plataforma para gerenciar pedidos e entregas. O sistema atual, baseado em planilhas, não atende mais às necessidades da equipe, dificultando o acesso a informações cruciais, como histórico de pedidos e perfil dos clientes. Essa desorganização afeta a qualidade do atendimento e gera insatisfação. O seu gerente pede uma solução eficiente e estruturada para organizar todos esses dados. Como você lidaria com esse desafio?

Para resolver o problema, é essencial entender a modelagem de dados. Sem uma estrutura adequada, a equipe corre o risco de se perder em dados duplicados ou inconsistentes, deixando passar erros que impactam diretamente a experiência dos clientes. Com a modelagem de dados, **você consegue organizar e estruturar informações de forma prática e intuitiva**, facilitando a tomada de decisões. Com essa habilidade, você contribuirá para a eficiência e a confiabilidade de projetos, influenciando a produtividade da equipe e garantindo dados sempre acessíveis e precisos.

Agora, vamos explorar um estudo de caso: imagine um sistema hospitalar que precisa gerenciar dados de pacientes, médicos e consultas. Atualmente, não há uma organização clara dessas informações, e os registros são feitos manualmente. Como desenvolvedor(a) júnior, **seu desafio é estruturar o banco de dados inicial para resolver esse problema**. Nesse sentido: (i) identifique as principais entidades que compõem o sistema (tais como paciente, médico, consulta); (ii) liste os atributos que seriam essenciais para cada entidade (tais como nome, especialidade e data de consulta); e (iii) proponha como as entidades poderiam se relacionar entre si (por exemplo, um médico pode atender vários pacientes).

Com base no estudo de caso, **reflita sobre os desafios encontrados ao organizar as informações e definir relacionamentos entre entidades**. Quais critérios você usou para escolher os atributos essenciais e como evitou problemas de duplicidade ou inconsistência? Considere como um banco de dados bem estruturado facilita consultas rápidas e elimina erros, enquanto decisões inadequadas podem gerar retrabalho e comprometer a eficiência. Essas reflexões serão aprofundadas ao longo do tema, o que te preparará para aplicar a modelagem de dados em diversos cenários reais.

**PLAY NO CONHECIMENTO**

Eu te convido a ouvir o nosso podcast sobre tipos de dados! Tipos de dados são os alicerces de uma boa modelagem de banco de dados, definindo como armazenamos e organizamos cada informação. Neste episódio, vamos explorar as principais categorias de tipos de dados e como elas impactam os respectivos projetos. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Para que você tenha uma compreensão mais sólida sobre modelagem de dados, sugiro que assista ao vídeo a seguir. Ele aborda de forma clara os conceitos dos modelos conceitual, lógico e físico, essenciais para entender as etapas e os níveis de abstração na modelagem de dados. <https://youtu.be/8CkMX2qXgdY?si=6lvE1YgCxEl-f6-->

DESENVOLVA SEU POTENCIAL

INTRODUÇÃO À MODELAGEM DE DADOS

A modelagem de dados é o ponto de partida para a construção de bancos de dados eficientes e organizados, essenciais para gerenciar e processar informações de maneira estruturada. Nos ambientes corporativo e acadêmico, a modelagem de dados permite a criação de uma representação lógica dos dados que um sistema precisará armazenar, processar e disponibilizar, o que auxilia na organização de informações e facilita o desenvolvimento de sistemas robustos e escaláveis (Ricardo; Urban; Davis, 2022).

Um modelo de dados é uma representação simplificada e abstrata da realidade. Nele, objetos, eventos e processos são traduzidos em entidades, atributos e relacionamentos. Esse processo é crucial para garantir que as necessidades de armazenamento e recuperação de dados sejam bem atendidas, proporcionando uma base sólida para a criação de sistemas de banco de dados que suportem a evolução das demandas e das necessidades dos usuários (Elmasri; Navathe, 2019). Quando bem estruturados, esses modelos são fundamentais para assegurar a integridade dos dados e evitar redundâncias e inconsistências.

A modelagem de dados permite que desenvolvedores e analistas visualizem o modo como os dados estarão organizados e relacionados entre si, promovendo uma visão clara da estrutura e do funcionamento do banco de dados antes mesmo da devida implementação. Além disso, é um processo que facilita a comunicação entre os membros da equipe de desenvolvimento e os *stakeholders*, permitindo que todas as partes compreendam a estrutura dos dados que suportarão o sistema (Elmasri; Navathe, 2019).

Neste tópico, exploraremos a importância da modelagem de dados e o modo como ela impacta o desenvolvimento de sistemas eficientes. Veremos os diferentes tipos de modelagem (conceitual, lógica e física) e como cada um contribui para um entendimento mais detalhado e prático da estrutura de dados necessária para cada aplicação.

INTRODUÇÃO À MODELAGEM DE DADOS

Ao desenvolver um sistema de banco de dados, a modelagem de dados passa por três etapas fundamentais: modelagem conceitual, modelagem lógica e modelagem física. Cada uma dessas fases representa um nível de detalhamento e abstração que contribui para a construção de uma estrutura organizada e eficiente, capaz de atender tanto às necessidades dos usuários quanto às limitações técnicas do sistema de banco de dados.

MODELAGEM CONCEITUAL

A modelagem conceitual é a primeira etapa do processo, focada na representação de alto nível dos dados e das respectivas relações, sem considerar a implementação prática. O objetivo é organizar as informações com base na compreensão inicial do domínio de negócios ou do sistema, criando uma visão geral que inclui entidades (ou objetos) e seus relacionamentos, mas sem detalhar como serão armazenados (Ricardo; Urban; Davis, 2022). São usados diagramas, como o diagrama de entidade-relacionamento (DER), para representar graficamente as entidades e os relacionamentos. Essa fase reúne requisitos essenciais, facilitando a comunicação entre a equipe técnica e os *stakeholders*, garantindo uma compreensão clara das necessidades e funções dos dados no sistema.

MODELAGEM LÓGICA

A modelagem lógica é o próximo passo. Ela transforma a visão conceitual em uma estrutura lógica, detalhando as entidades e os respectivos relacionamentos. Esse nível elimina ambiguidades, define chaves primárias e estrangeiras, e aplica restrições para garantir a integridade dos dados. Aqui, organiza-se como os dados serão estruturados, sem considerar ainda o Sistema de Gerenciamento de Banco de Dados (SGBD) específico (Ricardo; Urban; Davis, 2022). Por ser independente da plataforma, o modelo lógico pode ser aplicado em qualquer sistema de banco de dados. A função dele é preparar a informação para a estrutura física, definindo atributos, relações e regras de integridade referencial, além de identificar redundâncias e corrigir problemas estruturais antes da implementação.

MODELAGEM FÍSICA

A modelagem física é a fase final, traduzindo o modelo lógico para o SGBD específico. Envolve decisões sobre o armazenamento real dos dados, como criação de tabelas, colunas, índices e tipos de dados. Leva em consideração as características técnicas e limitações do SGBD, adaptando o modelo às necessidades de desempenho e segurança (Silberschatz; Korth; Sudarshan, 2020). É nessa fase que são definidas as configurações de armazenamento, métodos de acesso e otimizações para garantir a eficiência e a escalabilidade. Por exemplo, pode-se usar índices para acelerar consultas, definir tipos de dados precisos para cada atributo e configurar políticas de segurança e *backup*. Assim, a modelagem física transforma o planejamento em uma estrutura prática e pronta para a operação.

ENTIDADES, ATRIBUTOS E RELACIONAMENTOS

Agora, exploraremos os componentes fundamentais de qualquer modelo de dados: as entidades, os atributos que as descrevem e os relacionamentos que as conectam. Esses elementos formam a base para a estruturação de um banco de dados, permitindo a organização e a integração de informações de forma eficiente e prática.

Identificação de entidades e atributos

No processo de modelagem de dados, as entidades e os respectivos atributos são elementos essenciais para representar o que o sistema precisa armazenar (Silberschatz; Korth; Sudarshan, 2020).

Uma **entidade** representa um objeto ou um conceito no domínio do sistema, sendo algo com relevância e existência própria no contexto da aplicação. Em um sistema de *e-commerce*, por exemplo, entidades podem ser “**Cliente**”, “**Produto**” e “**Pedido**”. Cada entidade tem características próprias e desempenha um papel distinto dentro do sistema.

Identificar corretamente as entidades é o primeiro passo na criação de um modelo de dados eficaz. Para isso, é importante analisar as informações que o sistema irá gerenciar e definir quais objetos precisam ser representados de forma independente. Esse processo envolve entender as necessidades de negócios e os requisitos do sistema, a fim de garantir que todas as entidades relevantes sejam incluídas no modelo.

Os **atributos** representam as propriedades ou as características das entidades. Eles descrevem as informações específicas que precisamos armazenar sobre cada entidade. Por exemplo, a entidade “**Cliente**” pode ter atributos, como “**nome**”, “**endereço**” e “**telefone**”. Cada atributo deve ser único para a entidade que representa, e a definição correta dos atributos é essencial para assegurar a precisão e a relevância dos dados.

Os atributos podem ser classificados em diferentes tipos, como:

Entidade representa um objeto ou um conceito no domínio do sistema

Atributos representam as propriedades ou as características das entidades

SIMPLES:

valores únicos e indivisíveis, como "nome".

COMPOSTOS:

valores que podem ser divididos em subcomponentes, como "endereço" (que inclui rua, cidade e CEP).

DERIVADOS:

atributos calculados a partir de outros valores, como "idade", derivada da "data de nascimento".

MULTIVALORADOS:

atributos que podem ter múltiplos valores para uma entidade, como "telefone", que pode armazenar diferentes números para um cliente.

Identificar e classificar os atributos de maneira correta ajuda a garantir que o modelo de dados seja abrangente e evite redundâncias, promovendo a eficiência e a integridade do banco de dados.

Tipos de relacionamentos

Após definir as entidades e os respectivos atributos, é necessário estabelecer como essas entidades se relacionam dentro do sistema. Os relacionamentos representam as interações e as conexões entre entidades, possibilitando a criação de uma estrutura integrada de dados (Elmasri; Navathe, 2019).

RELACIONAMENTO UM-PARA-UM (1:1)

Um relacionamento um-para-um ocorre quando uma ocorrência de uma entidade está associada, no máximo, a uma ocorrência de outra entidade, e vice-versa. Esse tipo de relacionamento é menos comum, mas pode ser útil para representar dados com dependência específica. Um exemplo é a relação entre "Pessoa" e "Carteira de Identidade", em que cada pessoa possui uma carteira exclusiva e cada carteira pertence a uma única pessoa.

RELACIONAMENTO UM-PARA-MUITOS (1:N)

No relacionamento um-para-muitos, uma ocorrência de uma entidade está associada a várias ocorrências de outra entidade, enquanto uma ocorrência na segunda entidade se relaciona, no máximo, a uma ocorrência da primeira. Esse é um dos relacionamentos mais comuns em sistemas. Por exemplo, um "Cliente" pode fazer vários "Pedidos", mas cada "Pedido" pertence a apenas um "Cliente".

RELACIONAMENTO MUITOS-PARA-MUITOS (N:N)

Um relacionamento muitos-para-muitos ocorre quando várias ocorrências de uma entidade estão associadas a várias ocorrências de outra entidade. Em sistemas de banco de dados, esse tipo de relacionamento é geralmente implementado mediante uma tabela intermediária. Um exemplo é a relação entre "Estudante" e "Curso" em uma universidade, em que cada estudante pode se matricular em vários cursos, e cada curso pode ter vários estudantes.

Identificar corretamente os tipos de relacionamentos entre entidades é fundamental para garantir que o modelo de dados represente adequadamente as interações no sistema. Esse processo envolve não apenas definir os relacionamentos, mas também assegurar que eles sejam implementados de forma a promover a integridade dos dados e a eficiência do banco de dados.

DIAGRAMAS DE ENTIDADE-RELACIONAMENTO (DER)

Os Diagramas de Entidade-Relacionamento (DER) são representações gráficas que mostram as entidades, os atributos e os relacionamentos de um banco de dados. Utilizados amplamente na fase de modelagem conceitual, os DERs ajudam a transformar as informações coletadas em uma estrutura visual fácil de interpretar, facilitando o entendimento e a comunicação entre desenvolvedores, analistas e *stakeholders*.

Um DER é composto por três elementos principais: entidades, atributos e relacionamentos. Cada um desses componentes é representado de uma forma específica:

Entidades são representadas por retângulos e indicam os objetos ou os conceitos que compõem o sistema.

Atributos são representados por elipses e conectados às entidades para indicar as respectivas características.

Relacionamentos são mostrados por **losangos**, ligando duas ou mais entidades para ilustrar a interação entre elas.

Esses elementos visuais permitem uma representação clara das estruturas de dados, ajudando a identificar rapidamente a organização e a relação entre as partes do sistema.

Construção de um DER

Para construir um DER eficaz, é importante seguir uma sequência de etapas que parte da identificação das entidades e dos atributos, passando pela definição dos relacionamentos:

- 1. Identificação das entidades:** liste todas as entidades principais e considere o escopo do sistema para garantir que as mais relevantes sejam incluídas.
- 2. Definição dos atributos:** associe a cada entidade os atributos essenciais, que representam as informações a serem armazenadas.
- 3. Estabelecimento dos relacionamentos:** defina os tipos de relacionamento entre as entidades, como um-para-um, um-para-muitos e muitos-para-muitos.

4. Representação gráfica: desenhe o diagrama usando as formas corretas para entidades, atributos e relacionamentos, mantendo uma disposição clara e legível.

Esses passos contribuem para um DER bem estruturado, que servirá como referência para a construção dos modelos lógico e físico do banco de dados.

Exemplos práticos de DER

Para ilustrar a criação de um DER, considere um sistema de gerenciamento de pedidos em uma loja on-line. As principais entidades podem ser “Cliente”, “Pedido” e “Produto”. O relacionamento entre “Cliente” e “Pedido” é um-para-muitos, enquanto “Pedido” e “Produto” têm um relacionamento muitos-para-muitos, pois um pedido pode incluir vários produtos e cada produto pode estar presente em diferentes pedidos. Considerando esses requisitos, a modelagem do diagrama DER pode ser elaborada conforme a Figura 1.

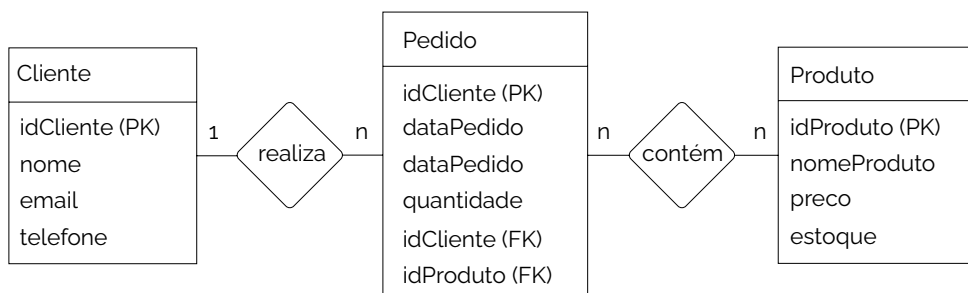


Figura 1 - DER de um Sistema de Gerenciamento de Pedidos / Fonte: o autor.

Descrição da Imagem: trata-se de um diagrama de entidade-relacionamento (DER) em preto e branco. Ele é composto por três retângulos com os títulos “Cliente”, “Pedido” e “Produto”. À esquerda, o retângulo “Cliente” lista os campos: “idCliente (PK)”, “nome”, “email” e “telefone”. No centro, o retângulo “Pedido” contém: “idPedido (PK)”, “dataPedido”, “quantidade”, “idCliente (FK)” e “idProduto (FK)”. À direita, o retângulo “Produto” apresenta: “idProduto (PK)”, “nomeProduto”, “preço” e “estoque”. Os retângulos são conectados por losangos que indicam os relacionamentos. O losango entre “Cliente” e “Pedido” possui o texto “realiza”, com os números “1” e “n” nos lados respectivos. O losango entre “Pedido” e “Produto” apresenta o texto “contém”, com “n” em ambos os lados. Fim da descrição.

Ao desenvolver exemplos como esse, é possível visualizar o processo de construção do DER e compreender como ele traduz as necessidades do sistema em uma estrutura organizada.

MODELO ENTIDADE-RELACIONAMENTO (MER)

O Modelo Entidade-Relacionamento (MER) é uma representação mais detalhada do que foi estruturado no DER, incorporando aspectos adicionais que facilitam o desenvolvimento e a implementação do banco de dados.

O MER é projetado para traduzir as necessidades do sistema em uma estrutura que possa ser adaptada ao modelo lógico e posteriormente implementada em um banco de dados. Ele permite visualizar com clareza as entidades, os relacionamentos e as regras de integridade que precisam ser mantidas. O principal objetivo do MER é definir um modelo que seja, ao mesmo tempo, intuitivo para os desenvolvedores e suficiente para suportar a complexidade dos dados exigida pelo sistema.

Componentes do MER

Os principais elementos que compõem o MER são:

Entidades e tabelas: representam objetos ou conceitos essenciais para o sistema, como "Cliente", "Produto" ou "Pedido".

Atributos: são os detalhes específicos de cada entidade, como "nome" do cliente, "preço" do produto ou "data" do pedido. No MER, cada atributo recebe um tipo de dado (como VARCHAR para texto ou INT para números inteiros) que define o formato dos dados armazenados.

Chaves primárias: atributos únicos que identificam cada instância de uma entidade, como «ID do cliente» ou «ID do pedido».

Chaves estrangeiras: atributos que conectam diferentes entidades, estabelecendo os relacionamentos entre elas.

Relacionamentos: conexões entre as entidades, detalhando as interações e as regras de integridade que definem como elas se relacionam.

Esses componentes formam uma base completa e coerente, permitindo ao banco de dados armazenar e recuperar as informações de forma precisa e segura. Agora, vamos explorar o processo de construção de um MER usando uma ferramenta amplamente adotada no mercado.

Construindo um MER completo

A criação de um MER detalhado envolve algumas etapas fundamentais. Para tornar essa prática mais acessível, vamos usar o **MySQL Workbench** como exemplo, uma ferramenta comum e visualmente intuitiva que facilita o desenvolvimento de modelos de banco de dados (Romanowski, 2024).

 **EU INDICO**

Para construir o seu próprio MER com o MySQL Workbench, você pode baixar a ferramenta gratuitamente no site oficial da MySQL neste link. A instalação é compatível com os principais sistemas operacionais, incluindo Windows, macOS e Linux. <https://www.mysql.com/products/workbench/>

Com o MySQL Workbench instalado, vamos construir um exemplo de modelo de dados simples para um Sistema de Gerenciamento de Produtos, que gerencia pedidos de clientes, para dar continuidade no DER que construímos nos tópicos anteriores. Esse MER incluirá as seguintes entidades e atributos:

ENTIDADE	ATRIBUTO
Cliente	ID Cliente (chave primária)
	Nome
	E-mail
	Telefone

ENTIDADE	ATRIBUTO
Produto	ID Produto (chave primária)
	Nome do Produto
	Preço
	Estoque
Pedido	ID Pedido (chave primária)
	Data do Pedido
	ID Cliente (chave estrangeira para conectar com "Cliente")
	ID Produto (chave estrangeira para conectar com "Produto")
	Quantidade

Quadro 1 - Entidades e atributos para a construção do modelo de dados / Fonte: o autor.

Essas entidades e os respectivos atributos formam a base do sistema. Você pode expandi-las conforme as necessidades do projeto, adicionando entidades e atributos.



Figura 2 - MER de uma plataforma de e-commerce / Fonte: o autor.

Descrição da Imagem: trata-se de um MER criado no MySQL Workbench. Ele é composto por três tabelas retangulares: "Cliente", "Pedido" e "Produto", dispostas horizontalmente, com bordas azuis e fundo branco. A tabela "Cliente", à esquerda, contém os campos: "idCliente INT" (chave primária, ícone de chave dourada), "nome VARCHAR(45)", "email VARCHAR(45)" e "telefone VARCHAR(45)". No centro, a tabela "Pedido" inclui: "idPedido INT" (chave primária), "dataPedido DATE", "idCliente INT" e "idProduto INT" (ambos chaves estrangeiras), e "quantidade INT". À direita, a tabela "Produto" possui: "idProduto INT" (chave primária), "nomeProduto VARCHAR(45)", "preco VARCHAR(45)" e "estoque VARCHAR(45)". Linhas conectam as tabelas: "idCliente INT" em "Cliente" é chave estrangeira em "Pedido", assim como "idProduto INT" em "Produto". Fim da descrição.

Após criar as tabelas e adicionar os relacionamentos no MySQL Workbench, o MER deve se assemelhar ao exemplo ilustrado na Figura 2, representando as conexões e as regras de integridade entre cliente, produto e pedido.

Boas práticas na construção do MER

Para criar um MER claro e funcional, siga algumas boas práticas. Use nomes descritivos e consistentes para entidades e atributos; prefira nomes no singular, como "Cliente", ao contrário de "Clientes"; e atribua nomes que reflitam claramente o propósito dos atributos, como "nomeCliente" ou "dataPedido".

No MySQL Workbench, as chaves primárias aparecem com um ícone de chave amarela e identificam cada instância de uma entidade (como "ID Cliente" e "ID Produto"). As chaves estrangeiras conectam entidades e são visualizadas no diagrama como linhas entre elas, indicando relacionamentos que garantem a integridade dos dados.

Escolha tipos de dados apropriados para cada atributo: "ID Cliente" pode ser do tipo **INT** (número inteiro), enquanto "Nome" e "Email" podem ser **VARCHAR** com limite de caracteres. Essa prática mantém o modelo otimizado e previne inconsistências nos dados.

Por fim, organize o diagrama de maneira clara e legível. Posicionar as entidades de forma que os relacionamentos não se sobreponham ou confundam a visualização facilita a compreensão do modelo, especialmente em sistemas mais complexos.

Essas práticas ajudam a construir um MER sólido e de fácil manutenção, servindo como uma base confiável para o desenvolvimento do banco de dados e permitindo que ele cresça com o sistema ao longo do tempo.

Erros comuns e dicas para evitá-los

Durante a criação de um MER, alguns erros são comuns, e evitá-los é essencial para manter a consistência do banco de dados. Um desses problemas é a **redundância de dados**, que ocorre quando informações são duplicadas entre entidades, o que pode gerar inconsistências e dificultar a manutenção.

Outro erro frequente é a definição inadequada de chaves primárias e estrangeiras: é fundamental garantir que as chaves primárias sejam únicas e que as chaves estrangeiras estejam corretamente relacionadas, assegurando a integridade das conexões entre as entidades.

Além disso, **relacionamentos mal definidos** são outro ponto crítico (Silberschatz; Korth; Sudarshan, 2020). É necessário que o tipo e a cardinalidade dos relacionamentos sejam especificados de forma clara, para que o modelo reflita corretamente as interações esperadas no sistema. Outro cuidado importante envolve a definição de regras de integridade, que estabelecem as restrições essenciais para a proteção dos dados. Essas regras ajudam a garantir que o modelo suporte todas as operações necessárias do sistema de maneira segura e coerente.

A atenção a esses detalhes ajuda a construir um MER confiável e funcional, que irá sustentar o banco de dados ao longo de seu ciclo de vida.

EM FOCO

Estudante, para expandir os seus conhecimentos sobre o assunto abordado, gostaríamos de te indicar a aula que preparamos especialmente para você. Acreditamos que essa aula irá complementar e aprofundar ainda mais o seu entendimento sobre o tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

Chegamos ao fim deste tema, mas o seu aprendizado sobre modelagem de dados é apenas o início de um caminho essencial na área de tecnologia. Você, agora, entende como as modelagens conceitual, lógica e física, a identificação de entidades e atributos, e a criação de relacionamentos e diagramas de entidade-relacionamento se unem para formar uma base sólida para qualquer projeto de banco de dados. Esses conhecimentos não são apenas teóricos; eles representam habilidades práticas que podem ser aplicadas diretamente no mercado de trabalho.

Em um ambiente profissional, a modelagem de dados se torna uma ferramenta poderosa para resolver problemas reais de organização e estruturação de dados, especialmente em cenários em que a eficiência e a precisão dos dados são cruciais. Pense nos setores de e-commerce, saúde, finanças e educação, onde a organização eficaz de dados pode determinar o sucesso ou o fracasso de uma operação. A capacidade de criar uma estrutura robusta e otimizada impacta diretamente a experiência dos usuários e a produtividade da equipe. Logo, para você, será um diferencial competitivo.

Ao retomarmos a situação-problema inicial, quando uma empresa precisava estruturar dados de pedidos e clientes, entendemos como a modelagem fornece soluções práticas para esses desafios. A compreensão dos relacionamentos entre dados e a criação de modelos bem estruturados possibilitam o desenvolvimento de sistemas ágeis e escaláveis, prontos para atender demandas que só aumentam no mercado atual. Além disso, o exercício de projetar e analisar modelos ao longo deste conteúdo deu a você uma visão prática de problemas comuns que profissionais enfrentam.

Por fim, lembre-se de que o aprendizado contínuo é essencial. A modelagem de dados está em constante evolução, e o mercado exige cada vez mais profissionais que saibam não apenas construir um banco de dados, mas também manter e adaptar estruturas conforme surgem novas demandas. Assim, ao entrar no mercado, você encontrará um vasto campo para aplicar e expandir esses conhecimentos, seja aprofundando-se em otimização de banco de dados, migração para novas tecnologias ou até mesmo integrando a modelagem com áreas de inteligência artificial.

VAMOS PRATICAR

1. No desenvolvimento de um sistema de informações, o processo de modelagem de dados busca identificar quais objetos e conceitos precisam ser representados para atender às necessidades do sistema. Para isso, é importante reconhecer as entidades que terão relevância no domínio do sistema e definir as características específicas, que são representadas pelos atributos. Esse cuidado garante que as informações sejam organizadas e armazenadas de maneira eficaz e precisa.

Com base no conceito de entidades e atributos na modelagem de dados, assinale a alternativa que melhor interpreta o papel desses elementos no processo de modelagem:

- a) A identificação de atributos únicos para cada entidade é opcional e pode ser dispensada quando os dados não precisam de organização específica.
 - b) A definição de entidades e atributos não exige uma análise detalhada do domínio do sistema, pois os objetos principais são intuitivamente identificados.
 - c) Os atributos são informações que descrevem as propriedades das entidades, e cada atributo deve ser único em todo o sistema para evitar duplicidade de dados.
 - d) As entidades, em um modelo de dados, representam objetos ou conceitos que possuem uma existência própria no contexto do sistema, e cada uma deve ser identificada exclusivamente pelos atributos.
 - e) Entidades e atributos são definidos para representar dados essenciais que o sistema precisa armazenar; as entidades definem o que será armazenado e os atributos definem as características desses dados.
2. Na modelagem de dados, as relações entre entidades desempenham um papel essencial, ao descreverem o modo como as informações se conectam no sistema. Essas relações variam de acordo com o tipo de associação entre as entidades ou que influenciam a organização e a estrutura do banco de dados. A compreensão dos diferentes tipos de relacionamentos é fundamental para uma modelagem eficaz de um sistema.

Considerando os tipos de relacionamento entre entidades na modelagem de dados, analise as afirmativas a seguir:

- I - Um relacionamento um-para-muitos é o mais raro em sistemas de banco de dados e deve ser evitado para simplificar a modelagem.
- II - Em um relacionamento muitos-para-muitos (N), cada ocorrência de uma entidade está associada, no máximo, a uma ocorrência de outra entidade.
- III - Em um relacionamento muitos-para-muitos (N), uma tabela intercalada é geralmente usada para associar múltiplas ocorrências de uma entidade com diversas ocorrências de outra.

VAMOS PRATICAR

IV - Um relacionamento um-para-muitos (1) permite que uma ocorrência de uma entidade esteja associada a várias ocorrências de outra entidade, enquanto uma ocorrência na segunda entidade se relaciona, no máximo, a uma ocorrência da primeira.

É correto o que se afirma em:

- a) I, apenas.
 - b) II e IV, apenas.
 - c) III e IV, apenas.
 - d) I, II e III, apenas.
 - e) I, II, III e IV.
3. A criação de um Diagrama de Entidade-Relacionamento (DER) requer uma sequência de passos para representar as principais partes de um sistema e como elas se relacionam. Esses passos ajudam a organizar visualmente as informações, facilitando o entendimento e a comunicação entre os envolvidos no projeto.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - Para construir um DER eficaz, é essencial identificar todas as entidades principais, pois isso garante que os objetos ou os conceitos fundamentais do sistema sejam representados corretamente.

PORQUE

II - A definição dos relacionamentos entre as entidades é opcional no DER, porque apenas as entidades são suficientes para ilustrar a estrutura do sistema.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2019.

RICARDO, C. M.; URBAN, S. D.; DAVIS, K. C. **Databases illuminated**. Massachusetts: Jones & Bartlett Learning, 2022.

ROMANOWSKI, J. The Most Popular Databases in 2020. **Learn SQL**, 18 ago. 2020. Disponível em: <https://learnsql.com/blog/most-popular-sql-databases-2020/>. Acesso em: 6 jan. 2025.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistemas de Banco de Dados**. 7. ed. Rio de Janeiro: Elsevier, 2020.

CONFIRA SUAS RESPOSTAS

1. Opção E.

A alternativa A está incorreta, pois a definição correta e precisa dos atributos é essencial para garantir a relevância e a organização dos dados que o sistema precisa armazenar. A alternativa B está incorreta, pois a análise do domínio do sistema é crucial para identificar corretamente as entidades e os respectivos atributos, considerando as necessidades de negócios e requisitos do sistema. A alternativa C está incorreta, pois os atributos devem ser únicos para cada entidade específica. Contudo, eles podem se repetir em diferentes entidades (por exemplo, "nome" pode existir em "Cliente" e "Produto"). A alternativa D está incorreta, pois, embora as entidades representem objetos ou conceitos com existência própria, elas não são identificadas exclusivamente pelos atributos. Os atributos apenas descrevem características das entidades.

2. Opção C.

A afirmativa I está incorreta, pois relacionamentos um-para-muitos (1) são comuns em sistemas de banco de dados e não são evitados; pelo contrário, são amplamente utilizados para modelar associações práticas entre entidades. A afirmativa II está incorreta, pois descreve incorretamente o relacionamento muitos-para-muitos (N). Nesse tipo de relacionamento, múltiplas ocorrências de uma entidade podem se associar a diversas ocorrências de outra, sem limitação a uma única ocorrência. A afirmativa III está correta, pois, em um relacionamento muitos-para-muitos (N), uma tabela comum é geralmente utilizada para permitir a associação de múltiplas ocorrências entre as duas entidades. Exemplo: relação entre "Estudante" e "Curso" em uma universidade. A afirmativa IV está correta, pois descreve o relacionamento um-para-muitos (1), em que uma ocorrência de uma entidade está associada a várias ocorrências de outra entidade, mas cada ocorrência da segunda entidade está relacionada a apenas uma ocorrência da primeira. Exemplo: um "Cliente" tem vários «Pedidos», mas cada «Pedido» pertence a um único «Cliente».

3. Opção C.

A asserção I é verdadeira, pois identificar todas as entidades principais é crucial para um DER eficaz. No entanto, a asserção II é falsa, já que a definição dos relacionamentos entre as entidades não é opcional; pelo contrário, é uma etapa essencial para entender as interações e as dependências entre as partes do sistema.

unidade





TEMA DE APRENDIZAGEM 4

NORMALIZAÇÃO: PRIMEIRA, SEGUNDA, TERCEIRA E QUARTA FORMA NORMAL

MINHAS METAS

- Entender os princípios da normalização de banco de dados.
- Compreender a Primeira Forma Normal (1FN) e a respectiva aplicação.
- Identificar a Segunda Forma Normal (2FN) e os respectivos requisitos.
- Aplicar a Terceira Forma Normal (3FN) em estruturas de dados.
- Diferenciar a Quarta Forma Normal (4FN) e as respectivas vantagens.
- Avaliar a importância da normalização na integridade dos dados.
- Identificar erros comuns na normalização e corrigi-los.

INICIE SUA JORNADA

Imagine que você é o(a) responsável pela base de dados de uma pequena empresa. Com o crescimento do negócio, cada vez mais informações são adicionadas ao sistema. Logo, você começa a perceber que alguns dados estão **duplicados e inconsistentes**. Por exemplo, o mesmo cliente aparece em registros diferentes, com pequenas variações de nome ou endereço, e algumas informações parecem estar repetidas em várias tabelas. Esses problemas dificultam a obtenção de relatórios precisos e a gerência das atualizações. Como você resolveria essa situação para garantir que os dados permaneçam organizados, sem redundâncias e confiáveis?

A normalização de dados é fundamental para garantir que um banco de dados funcione de forma organizada e eficiente. Esse processo **elimina redundâncias e melhora a integridade dos dados**, tornando o sistema mais fácil de manter e consultar. Quando a base de dados é bem normalizada, ela se torna mais ágil e confiável, facilitando a atualização e a manutenção das informações. Resolver a problemática referente à duplicidade e à inconsistência de dados não só evita retrabalho, mas também melhora a confiança nas informações, algo essencial para o crescimento de qualquer organização.

Agora, vamos trabalhar com um estudo de caso: imagine um sistema de gestão escolar que precisa armazenar dados de alunos e cursos. As informações estão desorganizadas, com múltiplos registros para o mesmo aluno e duplicidade de dados, como o nome de cursos. A sua tarefa é **propor uma estrutura inicial para normalizar as tabelas**. Nesse sentido: (i) identifique os problemas de redundância e inconsistência nas informações de alunos e cursos; e (ii) proponha uma estrutura para organizar os dados de forma que cada informação seja armazenada apenas uma vez, eliminando redundâncias.

Com base no estudo de caso, reflita sobre os desafios encontrados ao organizar as informações e propor uma solução. Como a normalização ajudou a evitar problemas de duplicidade ou inconsistência? Pense no modo como essas melhorias impactam a confiabilidade e o desempenho do sistema. Em seguida, reflita sobre como as técnicas de normalização podem ser aplicadas a outros contextos profissionais, garantindo a organização e a eficiência de bancos de dados em diferentes cenários.

**PLAY NO CONHECIMENTO**

Quer entender quando e por que desnormalizar um banco de dados? Neste podcast, exploramos as estratégias de desnormalização e o uso de índices para otimizar a performance. Não perca essa oportunidade de descobrir como essas técnicas podem transformar o seu sistema! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Quer entender melhor o conceito de normalização? Confira o vídeo intitulado *Normalização de Banco de Dados - 1FN, 2FN e 3FN*, do canal Daniel Maia, no YouTube! Com ele, você vai aprender as três primeiras formas normais e entenderá como elas ajudam a organizar os dados de maneira mais eficiente. https://youtu.be/c_WGssorPYc?si=eWuQEpPo4c6Tddef

DESENVOLVA SEU POTENCIAL

INTRODUÇÃO À NORMALIZAÇÃO

A normalização é um processo essencial na modelagem de bancos de dados relacionais. Ela consiste em organizar e estruturar os dados de forma a eliminar redundâncias e garantir a integridade e a consistência das informações armazenadas (Elmasri; Navathe, 2019). Em um banco de dados sem normalização, é comum que algumas informações apareçam em mais de um lugar, o que aumenta a possibilidade de erros e inconsistências ao atualizar, excluir ou inserir novos dados.

O conceito de normalização surgiu para lidar com esses problemas e tem como objetivo principal **evitar a duplicidade de dados**, tornando o banco de dados mais eficiente e seguro. Com uma estrutura bem normalizada, cada pedaço de informação é armazenado uma única vez e qualquer atualização ocorre em um único lugar, reduzindo o risco de inconsistências. Além disso, a normalização melhora a performance de consultas e facilita a manutenção

do banco de dados, tornando-o mais fácil de expandir e adaptar conforme as necessidades do sistema crescem.

O processo de normalização

O processo de normalização é uma metodologia que organiza e estrutura os dados dentro de um banco de dados relacional, garantindo que as informações sejam armazenadas de forma eficiente e consistente. A normalização envolve a aplicação de um **conjunto de regras**, as chamadas **Formas Normais**, que impõem restrições sobre a organização das tabelas e as relações entre os dados. Cada Forma Normal é uma etapa que aborda um tipo específico de problema, e a aplicação progressiva dessas etapas resulta em uma estrutura de dados mais otimizada, evitando redundâncias e melhorando a integridade dos dados (Elmasri; Navathe, 2019).

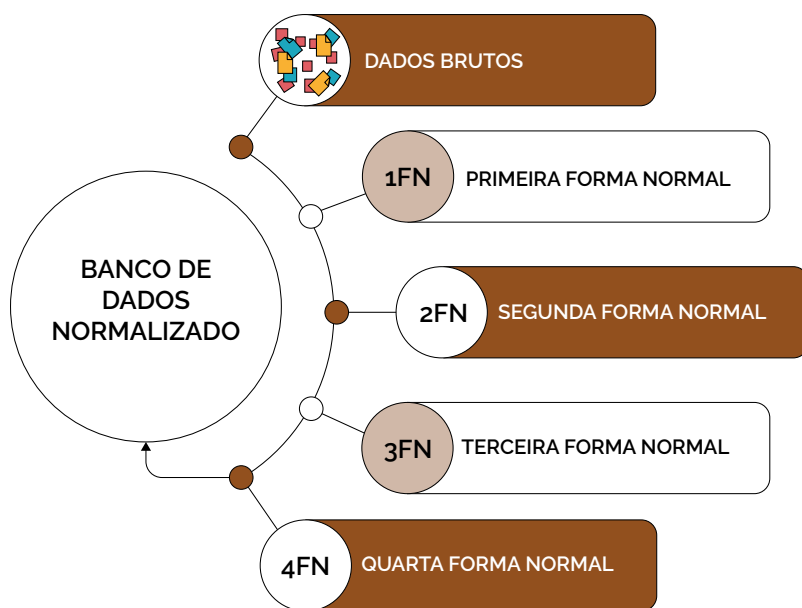


Figura 1 - Processo de normalização de banco de dados: da estrutura bruta ao banco normalizado / Fonte: o autor.

Descrição da Imagem: trata-se de um diagrama colorido que representa o processo de normalização de dados. No centro, há um círculo branco com borda preta contendo o texto "BANCO DE DADOS NORMALIZADO". Acima e à direita, encontra-se um balão marrom com o texto "DADOS BRUTOS". Logo abaixo, há mais quatro balões conectados: "1FN PRIMEIRA FORMA NORMAL" (marrom-claro), "2FN SEGUNDA FORMA NORMAL" (marrom-escuro), "3FN TERCEIRA FORMA NORMAL" (marrom-claro), e "4FN QUARTA FORMA NORMAL" (marrom-escuro). Fim da descrição.

Neste tema de aprendizagem, vamos explorar desde a primeira até a quarta Forma Normal. Entenderemos como cada uma delas contribui para um banco de dados mais eficiente e organizado.

PRIMEIRA FORMA NORMAL (1FN)

A Primeira Forma Normal, ou **1FN**, é o primeiro passo no processo de normalização. Ela impõe uma série de restrições fundamentais para organizar os dados de maneira eficiente (Ricardo; Urban; Davis, 2022). O principal requisito da 1FN é que cada tabela contenha apenas dados atômicos, ou seja, que cada coluna armazene um único valor em cada linha. Em outras palavras, uma tabela em 1FN não deve ter colunas que armazenam conjuntos de valores múltiplos ou listas.

Além disso, para que uma tabela esteja em 1FN, todos os registros devem ser únicos e as colunas não podem conter valores repetidos para identificar uma única instância. Isso significa que cada linha da tabela pode ser identificada de forma exclusiva, geralmente usando uma **chave primária**. Esse requisito ajuda a evitar redundâncias e facilita o uso de consultas, garantindo que as informações estejam extremamente organizadas.

Exemplo prático de Primeira Forma Normal

Para exemplificar, considere uma tabela de clientes que armazena informações de contato, mas em um formato desnormalizado. Essa tabela possui uma coluna “telefones” que armazena múltiplos números de telefone para cada cliente, separados por vírgulas, conforme Quadro 1.

IDCLIENTE	NOME	TELEFONES
1	Maria	(11)1234-5678, (11)9876-5432
2	João	(21)2345-6789
3	Ana	(31)3456-7890, (31)8765-4321

Quadro 1 - Exemplo prático de tabela desnormalizada / Fonte: o autor.

No exemplo anterior, a coluna “telefones” viola a 1FN, pois contém valores múltiplos em uma única célula. Para normalizar essa tabela e colocá-la em 1FN, precisamos **separar** os números de telefone em linhas individuais, onde cada telefone terá o próprio registro. Assim, teríamos uma tabela modificada, conforme é apresentado no Quadro 2.

IDCLIENTE	NOME	TELEFONES
1	Maria	(11)1234-5678
1	Maria	(11)9876-5432
2	João	(21)2345-6789
3	Ana	(31)3456-7890
3	Ana	(31)8765-4321

Quadro 2 - Exemplo prático de tabela normalizada com 1FN / Fonte: o autor.

Agora, a tabela atende aos requisitos da 1FN: cada célula contém apenas um valor, e os registros são únicos. Essa forma organizacional evita confusões e facilita a busca por dados específicos, como consultar um número de telefone associado a um cliente (Ricardo; Urban; Davis, 2022).

A aplicação da 1FN traz uma série de **benefícios**. Ela melhora a clareza dos dados, facilita a consulta de informações específicas e evita duplicidades e inconsistências. Além disso, a 1FN serve como base para as próximas formas normais, garantindo uma estrutura inicial organizada e eficiente.

A Primeira Forma Normal é o primeiro passo para a construção de um banco de dados bem estruturado e é essencial para garantir que a base de dados possa crescer de maneira organizada. Nos próximos tópicos, vamos explorar as etapas subsequentes, que aprofundam o processo de normalização e trazem novos níveis de refinamento.

SEGUNDA FORMA NORMAL (2FN)

A Segunda Forma Normal (2FN) é o segundo estágio do processo de normalização e acrescenta novas regras para garantir a integridade e a organização dos dados. Uma tabela pode ser considerada em 2FN se ela já atende aos requisitos da Primeira Forma Normal (1FN) e elimina qualquer tipo de **dependência parcial**. Isso significa que cada atributo não-chave deve depender totalmente da chave primária da tabela, e não apenas de uma parte dela. Esse conceito é especialmente relevante em tabelas com chaves primárias compostas (formadas por mais de um atributo).

A dependência parcial ocorre quando um atributo depende apenas de parte de uma chave primária composta, e não de toda ela. Esse tipo de dependência cria redundâncias e inconsistências que a 2FN busca resolver. Ao eliminarmos as dependências parciais, garantimos que cada coluna de dados seja funcionalmente dependente de toda a chave primária, fortalecendo a integridade dos dados (Elmasri; Navathe, 2019).

Exemplo prático de Segunda Forma Normal

Para aprender a aplicar a 2FN, considere uma tabela que armazena pedidos de produtos em uma loja. Essa tabela, em 1FN, possui uma chave primária composta pelos atributos **idPedido** e **idProduto**, e armazena informações adicionais sobre o produto e o pedido, conforme Quadro 3.

IDPEDIDO	IDPRODUTO	NOMEPRODUTO	QUANTIDADE	PRECPRODUTO
1	101	Caneta	5	2.00
1	102	Caderno	2	15.00
2	101	Caneta	3	2.00
2	103	Borracha	1	1.50

Quadro 3 – Exemplo prático de tabela na 1FN com dependência parcial / Fonte: o autor.

Nessa tabela, **idPedido** e **idProduto**, juntos, formam a chave primária. No entanto, observe que **nomeProduto** e **precoProduto** dependem apenas de **idProduto**, e não de **idPedido**. Isso cria uma dependência parcial, já que esses atributos não dependem de toda a chave primária composta. A 2FN exige que essas dependências parciais sejam eliminadas, movendo os dados que dependem apenas de **idProduto** para uma tabela separada. Após aplicarmos a 2FN, teríamos as seguintes tabelas, assim como é apresentado nos quadros 4 e 5.

IDPEDIDO	IDPRODUTO	QUANTIDADE
1	101	5
1	102	2
2	101	3
2	103	1

Quadro 4 - Tabela Pedidos: armazena informações específicas de pedidos / Fonte: o autor.

IDPEDIDO	NOMEPRODUTO	PRECOPRODUTO
101	Caneta	2.00
102	Caderno	15.00
103	Borracha	1.50

Quadro 5 - Tabela Produtos: armazena informações específicas de produtos / Fonte: o autor.

Agora, cada atributo depende totalmente da chave primária da própria tabela, atendendo aos requisitos da 2FN. Com essa nova estrutura, evitamos redundâncias, pois o nome e o preço do produto são armazenados uma única vez na tabela de produtos, e qualquer alteração será refletida em todos os pedidos que referenciam esse produto.

**PENSANDO JUNTOS**

A aplicação da 2FN traz melhorias significativas ao banco de dados, eliminando **redundâncias** e **inconsistências** associadas a dependências parciais. Ela facilita a manutenção dos dados, pois informações que se repetiam em várias linhas, agora, são armazenadas apenas uma vez. Além disso, essa estrutura melhora a organização do banco de dados e torna as atualizações mais eficientes, já que dados repetidos foram removidos.

A 2FN é um passo importante no processo de normalização, criando uma estrutura de dados que é mais fácil de manter e menos propensa a erros. Nos próximos tópicos, exploraremos a Terceira Forma Normal (3FN) e compreenderemos como ela nos ajuda a avançar ainda mais no processo de normalização, eliminando dependências transitivas e aprimorando a integridade dos dados.

TERCEIRA FORMA NORMAL (3FN)

A **Terceira Forma Normal (3FN)** é a terceira etapa do processo de normalização e visa eliminar dependências transitivas, um problema que ainda pode existir mesmo após a aplicação da Segunda Forma Normal (2FN). Para que uma tabela esteja na 3FN, ela deve, primeiro, estar na 2FN e não deve haver dependências transitivas entre os atributos (Ricardo; Urban; Davis, 2022).

A dependência transitiva ocorre quando um atributo não-chave depende de outro atributo não-chave, que, por sua vez, depende da chave primária. Essa situação cria uma ligação indireta entre atributos, o que aumenta o risco de inconsistências e dificulta a manutenção dos dados. A 3FN exige que todas essas dependências transitivas sejam eliminadas, garantindo que cada coluna seja exclusivamente dependente da chave primária.

Exemplo prático de Terceira Forma Normal

Vamos considerar uma tabela que armazena informações de pedidos e clientes em uma loja on-line, e que já está em 2FN. Ela inclui informações, como o iden-

tificador do pedido, o identificador do cliente, o nome do cliente e o endereço do cliente, conforme é apresentado no Quadro 6.

IDPEDIDO	IDCLIENTE	NOMECLIENTE	ENDERECOCLIENTE	DATAPEDIDO
1	101	Maria	Rua A, 123	2023-01-10
1	102	João	Rua B, 456	2023-01-15
2	101	Maria	Rua C, 789	2023-01-20

Quadro 6 - Exemplo de tabela com dependência transitiva / Fonte: o autor.

Nessa tabela, o atributo **endereçoCliente** depende de **idCliente**, que não é a chave primária, mas também depende indiretamente de **idPedido** por meio de **idCliente**. Esse é um exemplo de dependência transitiva, pois o endereço do cliente está relacionado ao pedido apenas por meio do identificador de cliente.

Para colocar essa tabela em 3FN, é necessário mover os atributos **nomeCliente** e **endereçoCliente** para uma tabela separada, que armazena apenas as informações dos clientes. Assim, evitamos redundâncias e inconsistências ao mantermos os dados dos clientes em uma única tabela, sem dependências indiretas. A nova estrutura seria (quadros 7 e 8):

IDPEDIDO	IDCLIENTE	DATAPEDIDO
1	101	2023-01-10
2	102	2023-01-15
3	101	2023-01-20

Quadro 7 - Tabela Pedidos: armazena informações específicas de pedidos / Fonte: o autor.

IDCLIENTE	NOMECLIENTE	ENDerecoCLIENTE
101	Maria	Rua A, 123
102	João	Rua B, 456

Quadro 8 – Tabela Clientes: armazena informações específicas de clientes / Fonte: o autor.

Agora, cada tabela está em 3FN, porque todos os atributos dependem diretamente da chave primária da respectiva tabela, e eliminamos as **dependências transitivas**.

Aplicar a 3FN proporciona uma série de benefícios, especialmente em termos de organização e integridade dos dados. Com a 3FN, os dados ficam centralizados e menos sujeitos a erros de redundância, pois cada informação está armazenada uma única vez e em uma tabela específica. Essa estrutura não apenas melhora a eficiência das atualizações, como também torna o banco de dados mais fácil de entender e manter.

A 3FN representa um nível avançado de normalização, garantindo que o banco de dados esteja organizado de maneira lógica e sem dependências desnecessárias. No entanto, em certos casos, formas normais mais avançadas podem ser necessárias, especialmente quando há dependências mais complexas. No próximo tópico, abordaremos a Quarta Forma Normal (4FN), que adiciona regras para eliminar dependências multivaloradas, garantindo uma estrutura ainda mais robusta.

QUARTA FORMA NORMAL (4FN)

A **Quarta Forma Normal (4FN)** é o próximo nível no processo de normalização, introduzindo uma regra específica para eliminar **dependências multivaloradas**. Mesmo após uma tabela estar em 3FN, ela pode conter múltiplos valores associados a uma única chave primária, o que pode gerar redundâncias e inconsistências. A 4FN tem como objetivo resolver esse problema, garantindo que cada tabela armazene apenas uma relação entre atributos, evitando que múltiplos valores de um atributo estejam ligados a uma única chave.

A **dependência multivalorada** ocorre quando um atributo de uma tabela possui vários valores associados a uma chave primária e esses valores são independentes de outros atributos. Em outras palavras, uma dependência multivalorada existe quando uma tabela tenta armazenar múltiplos relacionamentos distintos para uma mesma entidade. Esse tipo de estrutura pode gerar redundâncias e dificultar o acesso e a atualização dos dados.

Exemplo prático de Quarta Forma Normal

Para entender melhor a aplicação da 4FN, considere uma tabela que armazena informações sobre habilidades e projetos de funcionários em uma empresa (Quadro 9). Nessa tabela, cada funcionário pode ter várias habilidades e estar envolvido em múltiplos projetos ao mesmo tempo.

IDFUNCIONARIO	HABILIDADE	PROJETO
1	Programação	Projeto A
1	Banco de Dados	Projeto A
1	Programação	Projeto B
2	Design Gráfico	Projeto A
2	Design Gráfico	Projeto C

Quadro 9 – Exemplo de tabela com dependência multivalorada / Fonte: o autor.

Nessa estrutura, temos uma dependência multivalorada, pois o mesmo funcionário pode ter múltiplas habilidades e estar em múltiplos projetos. Esse formato resulta em duplicação de dados e torna o processo de atualização e consulta mais complexo. Para resolver esse problema e colocar a tabela em 4FN, devemos separar as dependências independentes em **tabelas distintas**. Após aplicarmos a 4FN, teríamos duas tabelas separadas, conforme é apresentado nos quadros 10 e 11.

IDFUNCIONARIO	HABILIDADE
1	Programação
1	Banco de Dados
1	Programação
2	Design Gráfico

Quadro 10 - Tabela de habilidades dos funcionários / Fonte: o autor.

IDFUNCIONARIO	PROJETO
1	Projeto A
1	Projeto A
1	Projeto B
2	Projeto A

Quadro 11 - Tabela de projetos dos funcionários / Fonte: o autor.

Agora, cada tabela armazena uma única relação: uma tabela para habilidades e outra para projetos. Eliminamos a dependência multivalorada, pois cada atributo multivalorado foi **isolado** em sua própria tabela. Com isso, o banco de dados fica mais organizado, com menos redundância e mais fácil de manter.

A aplicação da 4FN permite que o banco de dados seja organizado em estruturas independentes, eliminando redundâncias e facilitando o processo de atualização e consulta. Essa normalização é particularmente útil em sistemas complexos, em que uma mesma entidade pode ter múltiplas associações distintas, como funcionários com várias habilidades ou alunos matriculados em múltiplas disciplinas.

Ao seguirmos a 4FN, garantimos que cada tabela represente uma única relação de dados, simplificando a estrutura e evitando dependências excessivas. Embora nem sempre a 4FN seja necessária para todos os sistemas, ela é extremamente útil em contextos em que múltiplas associações independentes precisam ser armazenadas de forma organizada e eficiente.

QUINTA FORMA NORMAL (5FN)

Até o momento, estudamos as quatro primeiras formas normais, que servem para organizar e estruturar os dados, eliminando redundâncias e dependências desnecessárias. No entanto, é importante mencionar que existe uma Quinta Forma Normal (5FN), também conhecida como Forma Normal de Projeção/Junção, ou *Project-Join Normal Form (PJNF)* em inglês (What [...], 2024).

A 5FN se aplica em casos em que há dependências de junção específicas e complexas entre os dados, isto é, que não podem ser resolvidas pelas etapas anteriores de normalização. Ela garante que as relações entre tabelas possam ser representadas sem redundâncias, eliminando a necessidade de junções adicionais e assegurando a integridade total das relações de dados.

Embora a PJNF seja relevante em algumas situações, a aplicação prática dela é **rara**. Na maioria dos bancos de dados, alcançar a 3FN ou 4FN já assegura uma estrutura organizada e eficiente. A 5FN, então, é mais útil em sistemas extremamente complexos, em que a precisão dos dados é crucial e a eliminação de dependências de junção é indispensável.



EU INDICO

Este tutorial do DataCamp é ideal para aprender os fundamentos da normalização em SQL, com exemplos práticos que facilitam o entendimento das formas normais. Acesse aqui: <https://www.datacamp.com/pt/tutorial/normalization-in-sql>

VANTAGENS E DESAFIOS DA NORMALIZAÇÃO

A normalização traz diversos benefícios para a organização de dados, mas também apresenta alguns desafios práticos, especialmente quando a performance e o acesso rápido aos dados são essenciais. Neste tópico, vamos discutir as principais vantagens da normalização, como a integridade e a consistência dos dados. Não só, mas também conheceremos os possíveis desafios que podem surgir, principalmente quando o sistema precisa equilibrar eficiência e desempenho.

Vantagens da normalização

Os benefícios da normalização vão além da organização dos dados: ela facilita a manutenção, melhora a integridade e torna o sistema mais seguro e preciso (Rolik *et al.*, 2021). A seguir, exploramos algumas das principais vantagens que a normalização pode proporcionar para garantir a consistência e a confiabilidade dos dados.

REDUÇÃO DE REDUNDÂNCIA:

a normalização elimina a duplicidade de dados, armazenando cada informação em um único local. Essa prática reduz o consumo de espaço de armazenamento e torna o banco de dados mais ágil e eficiente. Ao evitar redundâncias, a normalização também diminui o risco de inconsistências, já que qualquer atualização de um dado precisa ser feita em um único lugar.

FACILIDADE DE MANUTENÇÃO:

com uma estrutura bem normalizada, a manutenção do banco de dados se torna mais simples e direta. As tabelas ficam menores e organizadas, facilitando a compreensão da estrutura e tornando o banco de dados mais fácil de expandir. Quando os dados estão organizados de forma lógica, é possível fazer alterações e atualizações com menos risco de erros e de maneira mais rápida.

INTEGRIDADE DOS DADOS:

a normalização melhora a integridade dos dados, ao garantir que cada informação seja armazenada de forma única e em uma tabela adequada. Isso evita que dados inconsistentes se espalhem pelo sistema e garante que as regras de integridade referencial sejam respeitadas.

AGILIDADE NAS CONSULTAS:

em tabelas menores e organizadas, as consultas tendem a ser mais rápidas, já que o banco de dados não precisa processar grandes volumes de dados repetitivos.

Essas vantagens fazem da normalização uma prática fundamental para garantir a eficiência, a integridade e a organização dos dados em sistemas de pequeno a médio porte, promovendo um ambiente de dados confiável e de fácil manutenção.

Desafios da normalização

Embora a normalização seja uma prática importante, ela também apresenta limitações, principalmente quando o sistema precisa lidar com grandes volumes de dados ou consultas frequentes e rápidas (Rolik *et al.*, 2021). Nos tópicos a seguir, abordamos os principais desafios encontrados ao aplicar a normalização em ambientes de grande escala.

COMPLEXIDADE DAS CONSULTAS:

à medida que o banco de dados é normalizado, mais tabelas são criadas para armazenar os dados de forma organizada. Isso pode tornar as consultas mais complexas, pois elas exigem junções entre várias tabelas para reconstruir a informação completa. Em sistemas com alto volume de consultas, essa complexidade pode afetar a performance, exigindo uma otimização cuidadosa das consultas.

IMPACTO NA PERFORMANCE EM SISTEMAS GRANDES:

em sistemas com grandes volumes de dados, a normalização total pode gerar uma sobrecarga na performance. Muitas vezes, bancos de dados de grande escala precisam acessar dados rapidamente, e as junções entre tabelas normalizadas podem ser um obstáculo para esse objetivo. Nesses casos, um equilíbrio entre normalização e des-normalização controlada pode ser necessário para manter o desempenho adequado.

DIFICULDADE DE IMPLEMENTAÇÃO PARA TODOS OS CENÁRIOS:

embora a normalização seja útil em muitos contextos, nem sempre é a solução ideal para todos os tipos de sistemas. Em cenários, como sistemas de análise em tempo real ou grandes bases de dados transacionais, a necessidade de acessar dados rapidamente pode exigir uma estrutura menos normalizada. Por isso, entender o contexto de aplicação é fundamental para decidir até que ponto a normalização será benéfica.

Esses desafios destacam a importância de balancear a normalização com as demandas de performance e complexidade do sistema, permitindo que o banco de dados ofereça uma estrutura eficiente e adaptada às necessidades de cada aplicação.

**EM FOCO**

Estudante, para expandir os seus conhecimentos sobre o assunto abordado, gostaríamos de te indicar a aula que preparamos especialmente para você. Acreditamos que essa aula irá complementar e aprofundar ainda mais o seu entendimento sobre o tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

Ao longo deste tema, você estudou a normalização de dados, um processo essencial para organizar, estruturar e aprimorar a confiabilidade dos bancos de dados. Exploramos as quatro primeiras formas normais (1FN, 2FN, 3FN e 4FN) e entendemos como elas eliminam redundâncias, garantem a integridade dos dados e tornam o sistema mais eficiente. Esse conhecimento é fundamental para criar bases de dados robustas e otimizadas, capazes de suportar tanto operações simples quanto cenários mais complexos.

Resgatando o estudo de caso inicial, em que o sistema de gestão escolar apresentava duplicidades e inconsistências nos dados de alunos e cursos, podemos, agora, propor uma solução prática. O primeiro passo seria identificar as entidades principais: “Aluno”, “Curso” e “Matrícula”. Em seguida, os atributos essenciais para cada entidade seriam definidos, como nome, endereço e CPF para “Aluno”, e nome, carga horária e código para “Curso”. A entidade “Matrícula” conectaria as demais, armazenando informações, como a data da matrícula e o código do curso. Ao aplicarmos a normalização, evitamos duplicidades, como a repetição do nome de alunos ou cursos, garantindo que cada dado seja armazenado de forma única e consistente.

O que você aprendeu aqui é essencial para a sua vida profissional. A normalização não apenas organiza informações, mas também otimiza o desempenho de bancos de dados e facilita a manutenção de sistemas. Saber equilibrar a normalização com a necessidade de performance em cenários reais, como em sistemas grandes ou de alta demanda, é uma habilidade valorizada no mercado de trabalho. Com esse conhecimento, você será capaz de projetar bases de dados que sejam, ao mesmo tempo, flexíveis e confiáveis, atendendo às necessidades específicas de cada projeto com eficiência.

VAMOS PRATICAR

1. O processo de normalização no modelo de dados relacionais inclui diversas etapas para organizar e garantir a integridade das informações em uma tabela. A Segunda Forma Normal é uma dessas etapas, estabelecendo regras adicionais para aprimorar a estrutura e o relacionamento entre dados. Cada estágio na normalização tem um objetivo específico e busca resolver problemas comuns que afetam a consistência dos dados em tabelas com chaves primárias simples ou compostas.

Com base no conceito de Segunda Forma Normal, assinale a alternativa que melhor representa como a 2FN ajuda a fortalecer a integridade dos dados:

- a) A 2FN é aplicável somente a tabelas com chaves primárias simples, pois chaves compostas não apresentam problemas de dependência parcial.
 - b) A 2FN elimina redundâncias em tabelas, permitindo que cada coluna seja independente dos valores da chave primária composta, se necessário.
 - c) Uma tabela em 2FN permite que atributos não-chave dependam apenas de parte da chave primária composta, desde que estejam organizados de forma atômica.
 - d) A aplicação da 2FN permite que tabelas com chaves primárias compostas tenham integridade, garantindo que cada atributo não dependa de apenas parte da chave, mas da chave completa.
 - e) A Segunda Forma Normal se aplica apenas em tabelas que não têm chaves primárias compostas, pois não são necessárias verificações adicionais para a dependência parcial.
2. O processo de normalização em bancos de dados relacionais é dividido em várias etapas, cada uma com regras específicas para organizar e estruturar os dados de forma mais eficiente. A Terceira Forma Normal (3FN) é uma dessas etapas e acrescenta requisitos importantes para evitar problemas de integridade nas tabelas. Cada uma dessas regras é aplicada para reduzir as possibilidades de inconsistências e facilitar a manutenção das informações no banco de dados.

Com base no conceito de Terceira Forma Normal, analise as afirmativas a seguir:

- I - A dependência transitiva ocorre quando um atributo não-chave depende diretamente da chave primária da tabela.
- II - A aplicação da 3FN permite dependências transitivas desde que a integridade da chave primária não seja afetada.
- III - A Terceira Forma Normal exige que uma tabela esteja na 2FN e que todas as dependências transitivas sejam eliminadas.
- IV - A 3FN organiza uma tabela de forma que cada atributo seja diretamente dependente da chave primária, sem dependências indiretas.

VAMOS PRATICAR

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

3. O processo de normalização em bancos de dados relacionados é composto por diferentes níveis, cada um com regras específicas para reduzir redundâncias e melhorar a organização dos dados. Em níveis mais avançados, como a Quarta Forma Normal (4FN), surgem novas abordagens para lidar com questões que podem impactar a estrutura e o gerenciamento dos dados armazenados.

Com base no conceito de Quarta Forma Normal, analise as afirmativas a seguir:

- I - A Quarta Forma Normal elimina dependências multivaloradas em uma tabela.
- II - A 4FN é uma etapa posterior à 3FN e resolve dependências transitivas para evitar redundâncias de dados.
- III - A aplicação da 4FN permite que uma tabela armazene relacionamentos múltiplos distintos para uma mesma chave primária.
- IV - A dependência multivalorada ocorre quando um atributo depende exclusivamente da chave primária, sem relação com os outros atributos.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2019.

RICARDO, C. M.; URBAN, S. D.; DAVIS, K. C. **Databases Illuminated**. 4. ed. Burlington: Jones & Bartlett Learning, 2022.

ROLIK, O. *et al.* Increase efficiency of relational databases using instruments of second normal form. *In*: INTERNATIONAL CONFERENCE ON ADVANCED TRENDS IN INFORMATION THEORY, 2021, [s. l.]. **Proceedings** [...]. [S. l.: s. n.], 2021. p. 221-225.

WHAT is PJNF (Project-Join Normal Form)? **GeeksforGeeks**, 6 out. 2024. Disponível em: <https://www.geeksforgeeks.org/what-is-pjnfproject-join-normal-form/>. Acesso em: 6 jan. 2025.

CONFIRA SUAS RESPOSTAS

1. Opção D.

A alternativa A está incorreta, pois a 2FN é especialmente relevante para as chaves primárias compostas, em que as dependências parciais são mais comuns. A alternativa B está incorreta, pois a 2FN exige que atributos não-chave sejam dependentes da chave primária completa, e não independentes dela. A alternativa C está incorreta, pois a 2FN não permite dependências parciais; todos os atributos não-chave devem depender totalmente da chave primária. A alternativa D está correta, pois a 2FN busca eliminar dependências parciais, garantindo que os atributos não-chave dependam da chave primária em sua totalidade. A alternativa E está incorreta, pois a 2FN é particularmente importante para tabelas com chaves primárias compostas, em que as dependências parciais são um problema comum.

2. Opção C.

A afirmativa I está incorreta, pois a dependência transitiva ocorre quando um atributo não-chave depende de outro atributo não-chave, que depende da chave primária. A afirmativa II está incorreta, pois a 3FN elimina todas as dependências transitivas, independentemente da supervisão da chave primária. A afirmativa III está correta, pois a 3FN requer que a tabela esteja em 2FN e que não haja dependências transitivas entre os atributos. A afirmativa IV está correta, pois a 3FN elimina dependências transitivas, garantindo que os atributos dependam diretamente da chave primária.

3. Opção A.

A afirmativa I está correta, pois a 4FN elimina dependências multivaloradas, garantindo uma organização mais eficiente dos dados. A afirmativa II está incorreta, pois a 4FN trata de dependências multivaloradas, e não de dependências transitivas, que são eliminadas na 3FN. A afirmativa III está incorreta, pois a 4FN visa eliminar relacionamentos múltiplos para a mesma chave, garantindo que cada tabela armazene apenas uma relação entre os atributos. A afirmativa IV está incorreta, pois a dependência multivalorada ocorre quando um atributo tem múltiplos valores associados a uma chave primária e esses valores são independentes de outros atributos.



TEMA DE APRENDIZAGEM 5

INTRODUÇÃO À LINGUAGEM SQL

MINHAS METAS

- Compreender a função dos comandos DDL na criação de tabelas.
- Utilizar o comando CREATE para criar tabelas em um banco de dados.
- Modificar a estrutura de tabelas usando o comando ALTER.
- Excluir tabelas de um banco de dados com o comando DROP.
- Aplicar o comando SELECT para consultar dados de forma eficiente.
- Aprender a inserir novos registros em tabelas com o comando INSERT.
- Aprender a atualizar e a excluir registros em tabelas.

INICIE SUA JORNADA

Imagine que você foi contratado(a) por uma pequena loja para organizar e analisar os dados do estoque e das vendas. Atualmente, todas as informações estão registradas em planilhas separadas e pouco organizadas, o que torna difícil extrair *insights* ou criar relatórios úteis para a tomada de decisão. O gerente, percebendo essa dificuldade, pediu a sua ajuda para criar uma solução que centralize os dados e facilite o acesso a informações importantes, como produtos mais vendidos, quantidades em estoque e histórico de vendas. Como você utilizaria a linguagem SQL para resolver esse problema e tornar os dados mais acessíveis e organizados?

A linguagem SQL (que significa “Linguagem de Consulta Estruturada”, do inglês, *Structured Query Language*) é a base para trabalhar com bancos de dados relacionais e é amplamente utilizada para criar, gerenciar e consultar informações. Com comandos simples, você pode criar tabelas, inserir dados, consultar informações e aplicar filtros para obter os resultados desejados. Aprender SQL permite que você organize grandes volumes de dados e crie sistemas que atendam a diferentes necessidades, seja no comércio, na saúde, na tecnologia ou em qualquer outra área. É uma habilidade essencial para qualquer profissional que lida com informações digitais.

Agora, considere o seguinte estudo de caso: imagine que você está criando o banco de dados para a loja mencionada. A sua tarefa inicial é planejar as tabelas básicas do sistema. Nesse sentido: (i) liste os dados essenciais que precisariam ser armazenados no banco, como produtos, vendas e clientes; (ii) pense nos principais atributos para cada conjunto de dados (tais como nome do cliente, produto, quantidade vendida etc.); e (iii) proponha uma forma de relacionar essas informações para que elas possam ser consultadas e analisadas de maneira integrada.

Com base no estudo de caso, reflita sobre os desafios encontrados na organização das tabelas e na definição dos atributos. Como o uso do SQL pode ajudar a superar esses desafios e garantir que os dados sejam armazenados e consultados de forma eficiente? Pense também em como essas habilidades poderão ser aplicadas em diferentes contextos profissionais, te ajudando a criar soluções robustas e centradas na análise de dados.

**PLAY NO CONHECIMENTO**

Venha ouvir o podcast sobre a criação de bancos de dados! Descubra as melhores práticas, evite armadilhas comuns e aprenda a estruturar tabelas de forma eficiente. Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.

VAMOS RECORDAR?

Quer entender melhor os fundamentos da linguagem SQL? Assista ao vídeo *SQL // Dicionário do Programador*, do Código Fonte TV, e aprenda de forma simples e prática! <https://youtu.be/kMznyi7r2Tc?si=jH2TdRYf5R9Xi-c6>

DESENVOLVA SEU POTENCIAL

INTRODUÇÃO À LINGUAGEM SQL

A linguagem SQL (*Structured Query Language*) é o padrão universal para interação com bancos de dados relacionais. Criada nos anos de 1970, ela se tornou a base para a manipulação de dados em sistemas que organizam informações em tabelas relacionadas. Desde a criação de estruturas básicas até consultas avançadas, o SQL é indispensável para gerenciar, organizar e extrair informações de bancos de dados (Allen; Owen, 2021).

SQL não é apenas uma ferramenta para desenvolvedores ou administradores de bancos de dados. Profissionais de diversas áreas, como analistas de dados, cientistas de dados e gestores, usam SQL para acessar e entender informações que guiam decisões importantes.

A principal força do SQL está na capacidade dele de manipular **grandes volumes de dados** de forma eficiente. Com comandos simples, você pode criar estruturas de dados, inserir informações, realizar consultas detalhadas e até aplicar filtros para obter exatamente o que precisa. O domínio de SQL permite que você explore todo o potencial desses sistemas.

A seguir, apresentaremos algumas **características** dessa linguagem que tornam isso possível:

LINGUAGEM DECLARATIVA:

Você descreve o que deseja como resultado sem precisar especificar como o banco deve realizar a operação.

PORTABILIDADE:

O SQL é compatível com a maioria dos sistemas de gerenciamento de bancos de dados (SGBDs), como MySQL, PostgreSQL, Oracle e SQL Server.

FLEXIBILIDADE:

Desde tarefas simples, como criar uma tabela, até consultas complexas envolvendo várias tabelas, o SQL oferece ferramentas poderosas.

Neste tema, exploraremos os principais comandos SQL, organizados em categorias específicas, como *Data Definition Language* (DDL) e *Data Manipulation Language* (DML). Esses comandos serão a base para **criar, manipular e consultar** informações de forma prática e eficiente.

COMANDOS DDL (*DATA DEFINITION LANGUAGE*)

Os comandos DDL (*Data Definition Language*) são usados para **definir e modificar** a estrutura de bancos de dados. Eles permitem criar, alterar e excluir tabelas, índices e outros objetos dentro do banco (Allen; Owen, 2021). Esses comandos são essenciais para o planejamento inicial e a organização do banco de dados, pois determinam como as informações serão armazenadas e estruturadas.

A seguir, exploraremos cada um desses comandos em detalhes, com exemplos práticos para facilitar a compreensão.

Comando CREATE

O comando **CREATE** é usado para criar objetos no banco de dados, como **tabelas**, índices e **esquemas**. Ele é o ponto de partida para qualquer estrutura de banco, definindo os atributos das tabelas e as respectivas restrições.

A seguir, é exibida a **estrutura básica** do modo como o comando **CREATE** é usado para criar uma tabela.

O comando CREATE é usado para criar objetos no banco de dados

```
CREATE TABLE nome_da_tabela (  
    coluna1 tipo_dado restrições,  
    coluna2 tipo_dado restrições,  
    ...  
);
```

Nessa sintaxe, o nome da tabela é definido no campo **nome_da_tabela**, que identifica a tabela no banco de dados. As colunas, especificadas como **coluna1**, **coluna2** e assim por diante, representam os campos que compõem a tabela, e cada uma delas deve ser acompanhada pelo **tipo de dado** que armazenará, como números ou texto. Além disso, é possível adicionar **restrições** às colunas, como **PRIMARY KEY**, para garantir valores únicos, ou **NOT NULL**, para impedir valores ausentes (Beaulieu, 2020).

Para exemplificar o uso do comando **CREATE**, vamos criar uma tabela chamada **Clientes** para armazenar informações de uma **loja**. O exemplo a seguir (ilustrado na Figura 1) mostra como isso pode ser feito.

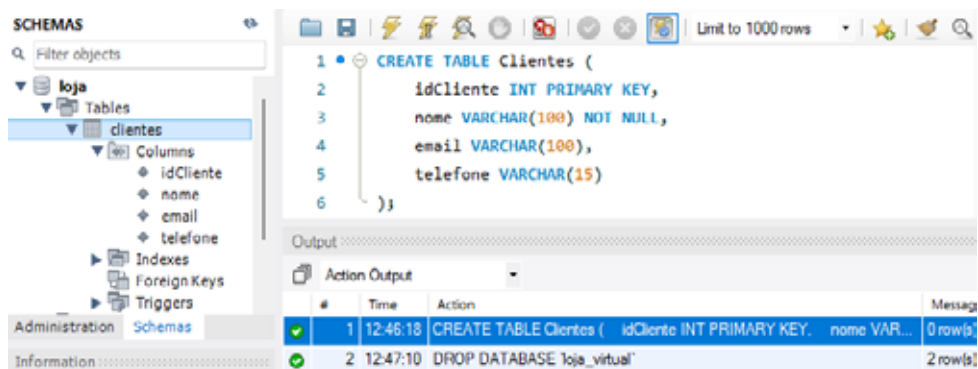


Figura 1 - Comando CREATE TABLE / Fonte: o autor.

Descrição da Imagem: trata-se de uma captura de tela do MySQL Workbench mostrando a interface de criação de tabelas. À esquerda, no painel "SCHEMAS", é selecionado o esquema "loja" com a tabela "clientes" e as respectivas colunas listadas. No centro, há a área de edição de código e, abaixo, a seção "Output" exibe a execução de um comando. No topo, há botões da interface em diferentes cores, como amarelo, vermelho e cinza. Fim da descrição.

Nesse exemplo, a tabela **Clientes** é composta por quatro colunas: a coluna id-Cliente é definida como um número inteiro (INT) e serve como chave primária (PRIMARY KEY), garantindo que cada cliente seja identificado de forma única. A coluna nome armazena texto de até 100 caracteres (VARCHAR(100)) e é obrigatória, conforme a restrição NOT NULL. A coluna email também armazena texto, mas é opcional, já que não tem restrições adicionais. A coluna telefone aceita até 15 caracteres e é opcional (Beaulieu, 2020).

O comando CREATE é a base para construir bancos de dados organizados e eficientes. Ele define a estrutura inicial de tabelas, tipos de dados e restrições que garantem a integridade. Entender a sintaxe, exemplos e boas práticas é o primeiro passo para criar sistemas robustos e funcionais. No próximo tópico, exploraremos o comando ALTER, usado para modificar tabelas existentes.

Comando ALTER

O comando **ALTER** é usado para modificar a estrutura de objetos existentes em um banco de dados. Ele permite alterar tabelas já criadas, adicionando ou removendo colunas, modificando tipos de dados, re-nomeando tabelas e até ajustando restrições. Ao contrário de recriar uma tabela do zero, o ALTER oferece flexibilidade para ajustar a estrutura à medida que novas necessidades surgem no sistema (Elmasri; Navathe, 2019).

A seguir, apresentamos a estrutura básica do comando **ALTER**, que pode variar conforme a operação desejada:

O comando ALTER é usado para modificar a estrutura de objetos existentes

Adicionar uma coluna:

```
ALTER TABLE nome_da_tabela  
ADD nome_da_coluna tipo_dado restrições;
```

Modificar uma coluna existente:

```
ALTER TABLE nome_da_tabela  
MODIFY nome_da_coluna tipo_dado restrições;
```

Remover uma coluna:

```
ALTER TABLE nome_da_tabela  
DROP COLUMN nome_da_coluna;
```

Renomear a tabela:

```
ALTER TABLE nome_da_tabela  
RENAME TO novo_nome_da_tabela;
```

Nessa sintaxe, **nome_da_tabela** refere-se à tabela que será alterada. Dependendo da operação, você pode adicionar, modificar ou excluir colunas, ajustando as características da tabela conforme necessário.

Para exemplificar, vamos aplicar o comando ALTER para modificar a tabela **Clientes** que criamos. Suponha que, ao longo do uso do sistema, foi identificado que faltam algumas informações relevantes e que outras colunas não são mais necessárias (Elmasri; Navathe, 2019).

Para incluir uma coluna chamada **dataCadastro**, que registra a data em que o cliente foi adicionado ao sistema, é preciso fazer o seguinte:

```
ALTER TABLE Clientes
ADD dataCadastro DATE;
```

Agora, a tabela Clientes inclui a coluna dataCadastro, que utiliza o tipo DATE para armazenar informações de data, assim como é ilustrado na Figura 2.

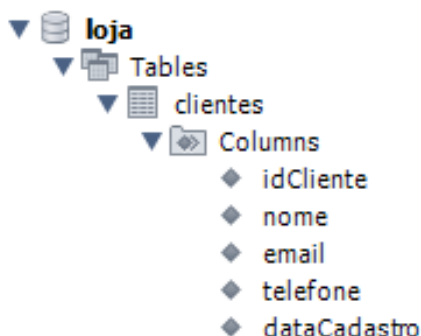


Figura 2 - Inclusão da coluna dataCadastro na tabela Clientes / Fonte: o autor.

Descrição da Imagem: trata-se de uma captura de tela de um painel de navegação de banco de dados no MySQL Workbench. O esquema "loja" está selecionado. Ele contém uma pasta "Tables" que inclui a tabela "clientes". Dentro dessa tabela, estão listadas as colunas: "idCliente", "nome", "email", "telefone" e "dataCadastro", cada uma representada por um ícone de losango cinza claro. O fundo é branco, com textos em preto e cinza. Fim da descrição.

Se for necessário ajustar o tamanho da coluna telefone para permitir números internacionais, podemos aumentar o limite de caracteres:

```
ALTER TABLE Clientes
MODIFY telefone VARCHAR(20);
```

Essa alteração **amplia** o limite de **caracteres** da coluna telefone para 20, garantindo compatibilidade com números maiores.

Caso a coluna email **não seja** mais utilizada no sistema, podemos **excluí-la**:

```
ALTER TABLE Clientes
DROP COLUMN email;
```

Com isso, a coluna email é **permanentemente** removida da tabela.

Se quisermos **renomear** a tabela Clientes para Usuarios, podemos fazer:

```
ALTER TABLE Clientes
RENAME TO Usuarios;
```

A partir desse momento, a tabela passará a ser identificada como Usuarios.

Em resumo, o comando **ALTER** é uma ferramenta que proporciona o ajuste de tabelas existentes sem a necessidade de recriação. Seja para adicionar novas informações, remover campos desnecessários ou ajustar colunas, ele é essencial para a manutenção de sistemas em constante evolução.

Comando DROP

O comando DROP é usado para excluir objetos de um banco de dados, como tabelas, índices ou esquemas. Ele é definitivo e, uma vez executado, o objeto removido não pode ser recuperado. Por isso, o uso do DROP requer cautela. Em geral, ele é utilizado quando uma tabela ou outro objeto não é mais necessário no sistema. A respectiva exclusão pode simplificar a estrutura do banco (Ricardo; Urban; Davis, 2022).

O comando DROP é usado para excluir objetos de um banco de dados

A seguir, encontra-se a estrutura básica para **excluir** uma tabela do banco de dados:

```
DROP TABLE nome_da_tabela;
```

Na sintaxe, **nome_da_tabela** refere-se à tabela que será permanentemente removida do banco de dados.

Vamos aplicar o comando **DROP** na tabela Clientes criada. Suponha que essa tabela tenha sido substituída por uma nova estrutura e não seja mais necessária no banco de dados. Para removê-la:


```
DROP TABLE Clientes;
```

Após a execução desse comando, a tabela Clientes será excluída do banco de dados junto com todos os dados e estrutura.

Além de tabelas, o DROP também pode ser usado para excluir índices ou esquemas. Por exemplo:

Para excluir um índice de uma tabela:

```
DROP INDEX nome_do_indice;
```

Para excluir um esquema inteiro:

```
DROP SCHEMA nome_do_esquema;
```

Esses comandos são úteis para reorganizar o banco de dados e remover objetos que não são mais necessários.

Vale relembrar que é importante usar o comando DROP com cautela, pois a respectiva execução é irreversível. Antes de utilizá-lo, certifique-se de que a exclusão do objeto é realmente necessária e que ele não será mais usado.

COMANDOS DML (DATA MANIPULATION LANGUAGE)

Os comandos DML (*Data Manipulation Language*) são utilizados para manipular os dados armazenados nas tabelas de um banco de dados. Eles permitem a inclusão, modificação, consulta e exclusão de registros, sendo essenciais para interagir com o conteúdo armazenado (Beaulieu, 2020).

Comando SELECT

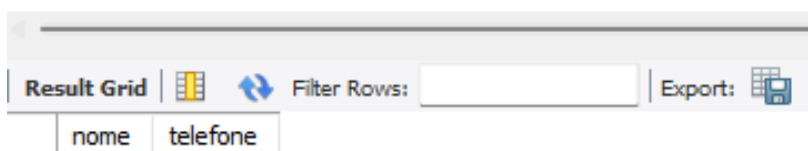
O comando SELECT é um dos mais utilizados em SQL, pois permite recuperar dados armazenados em tabelas de forma eficiente. Ele possibilita **consultar** informações específicas ou gerais, aplicar **filtros**, **ordenar** resultados e realizar **cálculos agregados**. É uma ferramenta indispensável para acessar os dados de maneira prática e organizada. A estrutura básica do SELECT é a seguinte:

```
SELECT colunas
FROM nome_da_tabela
WHERE condição;
```

O **SELECT** especifica quais colunas você deseja visualizar. O **FROM** indica a tabela de onde os dados serão extraídos. O **WHERE** é opcional e permite filtrar os registros com base em uma condição (Silberschatz; Korth; Sudarshan, 2020).

Agora que você já conhece a sintaxe básica, vamos consultar informações da tabela *Cientes* criada:

```
1  SELECT nome, telefone
2  FROM Cientes
3  WHERE telefone IS NOT NULL;
```



nome	telefone
------	----------

Figura 3 - Consulta de informações na tabela *Cientes* / Fonte: o autor.

Descrição da Imagem: é exibido um editor SQL com a execução de uma consulta, e os resultados exibidos abaixo. No editor, há texto em azul e preto. Abaixo, na seção “Result Grid”, há uma tabela com cabeçalhos “nome” e “telefone”. O fundo é branco, com textos em preto e azul. Fim da descrição.

Nesse exemplo, estamos selecionando as colunas **nome** e **telefone**. A condição **telefone IS NOT NULL** filtra apenas os registros onde o telefone foi informado. O resultado será uma lista de clientes com os respectivos números de telefone cadastrados, como ilustrado em “*Result Grid*” da Figura 3.

O comando **SELECT** pode ser combinado com operadores e funções para obter resultados mais específicos:

Filtrar registros com operadores: O exemplo abaixo retorna os clientes cujos nomes começam com a letra "A".

```
SELECT nome, telefone  
FROM Clientes  
WHERE nome LIKE 'A%';
```

Realizar cálculos agregados: O exemplo abaixo retorna o número total de clientes na tabela.

```
SELECT COUNT(*) AS total_clientes  
FROM Clientes;
```

Ordenar resultados: O exemplo abaixo ordena os resultados em ordem alfabética pela coluna nome.

```
SELECT nome, telefone  
FROM Clientes  
ORDER BY nome ASC;
```

O comando **SELECT** é a principal ferramenta para acessar dados em SQL, permitindo consultas personalizadas e análises detalhadas. A flexibilidade dele o torna essencial para qualquer aplicação que exija recuperação de informações.

Comando INSERT

O comando **INSERT** é utilizado para adicionar novos registros a uma tabela no banco de dados. Ele é essencial para alimentar a base com informações iniciais ou incluir dados gerados em tempo real por sistemas e usuários. Com simplicidade e flexibilidade, o **INSERT** permite que você insira dados completos ou parciais de forma eficiente (Allen; Owen, 2021; Beaulieu, 2020). A estrutura básica do comando **INSERT** é a seguinte:

```
INSERT INTO nome_da_tabela (coluna1, coluna2, ...)  
VALUES (valor1, valor2, ...);
```

O **nome_da_tabela** refere-se à tabela onde os dados serão inseridos. As **colunas** especificadas determinam os campos que receberão os valores fornecidos. O **VALUES** lista os valores a serem adicionados na mesma ordem das colunas mencionadas (Allen; Owen, 2021). Agora, vamos adicionar um novo cliente à tabela Clientes que criamos:

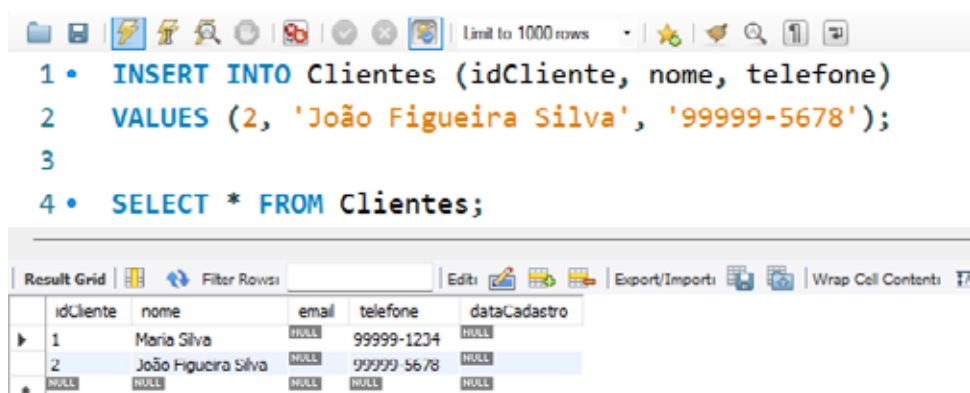


Figura 4 - Adicionando informações na tabela Clientes / Fonte: o autor.

Descrição da Imagem: trata-se de uma captura de tela de um editor SQL no MySQL Workbench que mostra a execução de comandos SQL e os resultados correspondentes. No editor, há texto em azul, laranja e preto. Abaixo, na seção "Result Grid", aparece uma tabela com as colunas "idCliente", "nome", "email", "telefone" e "dataCadastro". Os dados exibidos incluem duas entradas: uma com "idCliente" 1, e outra com "idCliente" 2. O fundo é branco, com elementos da interface em tons de cinza e azul-claro. Fim da descrição.

Assim como é ilustrado na Figura 4, estamos adicionando um registro à tabela Clientes. O identificador do cliente (idCliente) é 2. O nome é "João Figueira Silva" e o telefone é "99999-5678". Se a tabela tiver colunas que aceitem valores nulos ou possuem valores-padrão, como email, elas não precisam ser especificadas no comando.

O comando **INSERT** é indispensável para adicionar dados a um banco de dados, oferecendo flexibilidade para operações simples ou complexas. Ele é o ponto de partida para alimentar as tabelas com informações que serão manipuladas e consultadas posteriormente.

Comando UPDATE e DELETE

Os comandos **UPDATE** e **DELETE** são usados para modificar e remover registros em tabelas, respectivamente. Enquanto o **UPDATE** atualiza valores existentes

no banco de dados, o DELETE elimina registros que não são mais necessários (Beaulieu, 2020). Esses comandos são fundamentais para manter os dados atualizados e organizados, mas devem ser usados com cautela para evitar alterações ou exclusões acidentais (Allen; Owen, 2021).

O comando UPDATE permite modificar os valores de uma ou mais colunas em registros existentes. Ele é frequentemente usado para corrigir informações ou refletir mudanças em dados previamente inseridos (Elmasri; Navathe, 2019). A sintaxe do comando UPDATE é apresentada a seguir:

```
UPDATE nome_da_tabela  
SET coluna1 = valor1, coluna2 = valor2, ...  
WHERE condição;
```

Aqui, **nome_da_tabela** refere-se à tabela onde a atualização será feita. **SET** especifica as colunas e os novos valores que serão atribuídos. Por fim, **WHERE** define quais registros serão atualizados. Sem essa cláusula, todas as linhas da tabela seriam alteradas (Beaulieu, 2020).

O comando **DELETE** é usado para remover registros específicos de uma tabela. É ideal para eliminar **dados desnecessários** ou **inválidos**, mas, como o comando exclui os registros de forma definitiva, deve ser aplicado com atenção. A sintaxe do comando DELETE é descrita a seguir:

```
DELETE FROM nome_da_tabela  
WHERE condição;
```

Na sintaxe, **nome_da_tabela** indica a tabela onde os registros serão removidos e **WHERE** filtra os registros a serem excluídos. Sem essa cláusula, todos os registros da tabela serão apagados (Beaulieu, 2020). Em resumo, os comandos UPDATE e DELETE são indispensáveis para a manutenção dos dados em um banco de dados. Eles permitem corrigir e remover informações, mas exigem cautela para evitar **erros irreversíveis**.

CLÁUSULAS SQL

As cláusulas SQL são ferramentas poderosas que permitem refinar consultas e organizar os dados retornados de um banco. Elas adicionam camadas de filtragem, agrupamento e ordenação aos comandos, tornando os resultados mais específicos e úteis. As cláusulas mais comuns incluem **WHERE**, **GROUP BY**, **HAVING** e **ORDER BY**, que serão exploradas a seguir.

Cláusula WHERE: a cláusula WHERE é usada para filtrar os registros de uma tabela com base em condições específicas. Ela permite restringir os dados retornados, garantindo que apenas as informações relevantes sejam exibidas. Por exemplo:

```
SELECT * FROM Clientes
```

```
WHERE cidade = 'São Paulo';
```

Nesse caso, a consulta retorna apenas os clientes cuja cidade seja “São Paulo”. A

WHERE suporta operadores, como =, >, <, LIKE, BETWEEN, e combinações com AND e OR, possibilitando filtros complexos.

Cláusula GROUP BY: a cláusula GROUP BY organiza os resultados de uma consulta em grupos com base em valores de uma ou mais colunas. Ela é usada principalmente em conjunto com funções agregadas, como SUM, AVG e COUNT. Por exemplo:

```
SELECT cidade, COUNT(*) AS totalClientes
```

```
FROM Clientes
```

```
GROUP BY cidade;
```

Aqui, os resultados são agrupados por cidade, exibindo o número total de clientes em cada uma. Essa cláusula é útil para gerar relatórios ou análises de dados categorizados.

Cláusula HAVING: a cláusula HAVING filtra os grupos criados pelo GROUP BY aplicando condições a eles. Enquanto o WHERE filtra registros individuais, o HAVING é usado para limitar grupos. Por exemplo:

```
SELECT cidade, COUNT(*) AS totalClientes
```

```
FROM Clientes
```

```
GROUP BY cidade
```

```
HAVING COUNT(*) > 10;
```

Essa consulta retorna apenas as cidades com mais de 10 clientes, combinando o agrupamento do GROUP BY com a filtragem do HAVING.

Cláusula ORDER BY: a cláusula ORDER BY organiza os resultados de uma consulta em ordem crescente (ASC) ou decrescente (DESC). Ela é aplicada ao final do comando

SELECT para ordenar os registros retornados. Por exemplo:

```
SELECT nome, cidade  
FROM Clientes  
ORDER BY nome ASC;
```

Nesse exemplo, os clientes são listados em ordem alfabética pelo nome. A ORDER BY pode ser usada com múltiplas colunas; logo, é possível personalizar a ordenação.

As cláusulas SQL, como **WHERE**, **GROUP BY**, **HAVING** e **ORDER BY**, são indispensáveis para personalizar consultas e obter resultados precisos. Elas permitem filtrar, agrupar e ordenar dados, tornando as informações mais organizadas e acessíveis. Dominar essas cláusulas é essencial para trabalhar com bancos de dados de maneira eficiente e produtiva.

Aplicação Prática com Cláusulas SQL

Resgatando o estudo de caso inicial, momento em que foi proposta a criação do banco de dados para uma loja, podemos, agora, planejar uma **solução prática**. A primeira etapa seria definir as tabelas essenciais: “Produtos”, “Clientes” e “Vendas”. Cada tabela deve conter atributos relevantes, como nome e preço para “Produtos”, nome e contato para “Clientes”, e data, quantidade e valor total para “Vendas”.

Em seguida, usaríamos os comandos CREATE para estruturar as tabelas, e INSERT para adicionar registros iniciais. Por exemplo, criaríamos uma tabela “Produtos” com atributos, como idProduto, nomeProduto e preco, e, em seguida, inseriríamos os produtos disponíveis no estoque. Com isso, estaríamos prontos para consultar informações importantes com comandos SELECT e aplicar filtros com a cláusula WHERE, como identificar os produtos mais vendidos ou as vendas em um período específico.

EM FOCO

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema. Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.

NOVOS DESAFIOS

Neste tema, você aprendeu os fundamentos da linguagem SQL e como utilizá-la para criar, gerenciar e consultar bancos de dados. Exploramos os principais comandos DDL e DML, como **CREATE**, **INSERT**, **SELECT**, **UPDATE** e **DELETE**, além de cláusulas importantes, como **WHERE**, **GROUP BY** e **ORDER BY**. Esses conceitos são essenciais para organizar, manipular e analisar dados de maneira eficiente, oferecendo ferramentas indispensáveis para resolver problemas reais no ambiente profissional.

Resgatando o estudo de caso inicial, momento em que foi proposta a criação do banco de dados para uma loja, podemos, agora, planejar uma **solução prática**. A primeira etapa seria definir as tabelas essenciais: “Produtos”, “Clientes” e “Vendas”. Cada tabela deve conter atributos relevantes, como nome e preço para “Produtos”, nome e contato para “Clientes”, e data, quantidade e valor total para “Vendas”.

Resolver problemas utilizando comandos SQL é um passo essencial para consolidar o aprendizado e compreender como a linguagem é aplicada em cenários reais. Criar e manipular tabelas, inserir dados e realizar consultas utilizando filtros e agrupamentos não são ações que apenas desenvolvem a habilidade técnica, mas também simulam situações do mercado profissional. A prática consistente com SQL é indispensável para dominar a construção de bancos de dados e a extração de informações relevantes de forma eficaz.

O que você aprendeu neste tema é extremamente relevante para a sua formação profissional. SQL é uma linguagem universal no mercado de tecnologia e uma das habilidades mais demandadas em áreas que envolvem análise de dados, ciência de dados e desenvolvimento de sistemas. Com SQL, você poderá projetar bancos de dados que não apenas organizam informações, mas também entregam *insights* valiosos para a tomada de decisões. Saber utilizar comandos DDL e DML de forma eficaz te permitirá construir sistemas sólidos e adaptáveis, preparados para os desafios de projetos reais.

VAMOS PRATICAR

1. O comando SELECT é amplamente utilizado para realizar consultas em tabelas de bancos de dados. Ele permite recuperar informações específicas de forma organizada e estruturada, com opções de aplicação de filtros e seleção de colunas. A flexibilidade dele o torna essencial para acessar dados com diferentes níveis de detalhe e precisão.

Considerando o uso do comando SELECT, analise as afirmativas a seguir:

- I - O comando SELECT exclui registros de uma tabela no banco de dados.
- II - O comando SELECT pode ser utilizado para modificar os valores armazenados em uma tabela no banco de dados.
- III - A cláusula WHERE pode ser utilizada em consultas para filtrar registros com base nas condições específicas.
- IV - A cláusula WHERE, usada em conjunto com o comando SELECT, permite filtrar registros com base em condições específicas, como valores iguais, diferentes, nulos ou não nulos.

É correto o que se afirma em:

- a) I, apenas.
 - b) II e IV, apenas.
 - c) III e IV, apenas.
 - d) I, II e III, apenas.
 - e) I, II, III e IV.
-
2. As cláusulas SQL são ferramentas importantes em consultas realizadas em tabelas de bancos de dados. Elas permitem organizar e manipular os dados retornados de diversas maneiras, que são aplicados conforme os objetivos da consulta. As cláusulas SQL oferecem grande flexibilidade e precisão para atender a requisitos específicos de análise e extração de informações.

Considerando o uso das cláusulas SQL, analise as afirmativas a seguir:

- I - A cláusula WHERE filtra os registros antes dos dados serem agrupados.
- II - A cláusula WHERE é obrigatória em consultas que utilizam funções agregadas, como COUNT ou SUM.
- III - A cláusula GROUP BY permite ordenar os registros agrupados em ordem crescente ou decrescente.
- IV - A cláusula GROUP BY é utilizada para filtrar os registros de uma tabela com base nas condições específicas.

É correto o que se afirma em:

VAMOS PRATICAR

- a) I, apenas.
 - b) II e III, apenas.
 - c) III e IV, apenas.
 - d) I, II e III, apenas.
 - e) II, III e IV, apenas.
3. A linguagem Structured Query Language (SQL) é amplamente utilizada para gerenciar bancos de dados relacionais e atender às necessidades de manipulação e consulta de dados. Criada nos anos de 1970, a linguagem SQL se tornou essencial para organizar e extrair informações em sistemas que utilizam tabelas relacionadas.

Com base no texto, qual alternativa descreve corretamente uma ou mais características do SQL e a respectiva aplicação prática?

- a) A característica principal do SQL é a exclusividade dele para sistemas que não utilizam tabelas relacionadas.
- b) O SQL é uma linguagem limitada a administradores de bancos de dados e não é usada por profissionais de outras áreas.
- c) Uma das forças do SQL é a capacidade de manipular dados, mas ele não permite realizar consultas mais complexas.
- d) A portabilidade do SQL significa que ele pode ser utilizado em diferentes sistemas de gerenciamento de bancos de dados.
- e) A característica declarativa do SQL exige que o usuário detalhe todas as etapas de execução de uma operação no banco de dados.

REFERÊNCIAS

ALLEN, G. R.; OWEN, W. A. **SQL in 10 Minutes, Sams Teach Yourself**. 5. ed. [S. l.]: Pearson Education, 2021.

BEAULIEU, A. **Learning SQL: Master SQL Fundamentals**. 3. ed. Sebastopol: O'Reilly Media, 2020.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2019.

RICARDO, C. M.; URBAN, S. D.; DAVIS, K. C. **Databases Illuminated**. 4. ed. Burlington: Jones & Bartlett Learning, 2022.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistemas de Banco de Dados**. 7. ed. Rio de Janeiro: Elsevier, 2020.

CONFIRA SUAS RESPOSTAS

1. Opção C.

I. O comando SELECT exclui registros de uma tabela no banco de dados.

Feedback: incorreta. O SELECT é usado apenas para consultar dados, enquanto a exclusão é realizada com o comando DELETE.

II. O comando SELECT pode ser utilizado para modificar os valores armazenados em uma tabela no banco de dados.

Feedback: incorreta. O SELECT é usado apenas para consultas; comandos, como UPDATE, são usados para modificar dados.

III. A cláusula WHERE pode ser utilizada em consultas para filtrar registros com base nas condições específicas.

Feedback: correta. A cláusula WHERE é opcional e usada para aplicar filtros aos resultados retornados.

IV. A cláusula WHERE, usada em conjunto com o comando SELECT, permite filtrar registros com base em condições específicas, como valores iguais, diferentes, nulos ou não nulos.

Feedback: correta. A cláusula WHERE é uma funcionalidade do comando SELECT que permite aplicar filtros e personalizar os resultados das consultas.

2. Opção A.

I. A cláusula WHERE filtra os registros antes dos dados serem agrupados.

Feedback: correta. Essa é uma das principais funcionalidades da cláusula WHERE, permitindo filtrar registros antes de operações de agrupamento.

II. A cláusula WHERE é obrigatória em consultas que utilizam funções agregadas como COUNT ou SUM.

Feedback: incorreta. O uso WHERE é opcional, mesmo em consultas que utilizam funções agregadas.

III. A cláusula GROUP BY permite ordenar os registros agrupados em ordem crescente ou decrescente.

Feedback: incorreta. A ordenação é realizada com a cláusula ORDER BY, enquanto a GROUP BY organização dos registros em grupos.

IV. A cláusula GROUP BY é utilizada para filtrar os registros de uma tabela com base nas condições específicas.

Feedback: incorreta. A GROUP BY organização de registros em grupos, enquanto a filtragem é feita com a cláusula WHERE.

CONFIRA SUAS RESPOSTAS

3. Opção D.

A alternativa A está incorreta, pois o SQL é projetado para manipular dados em tabelas relacionadas, sendo uma base de sistemas de bancos de dados relacionais.

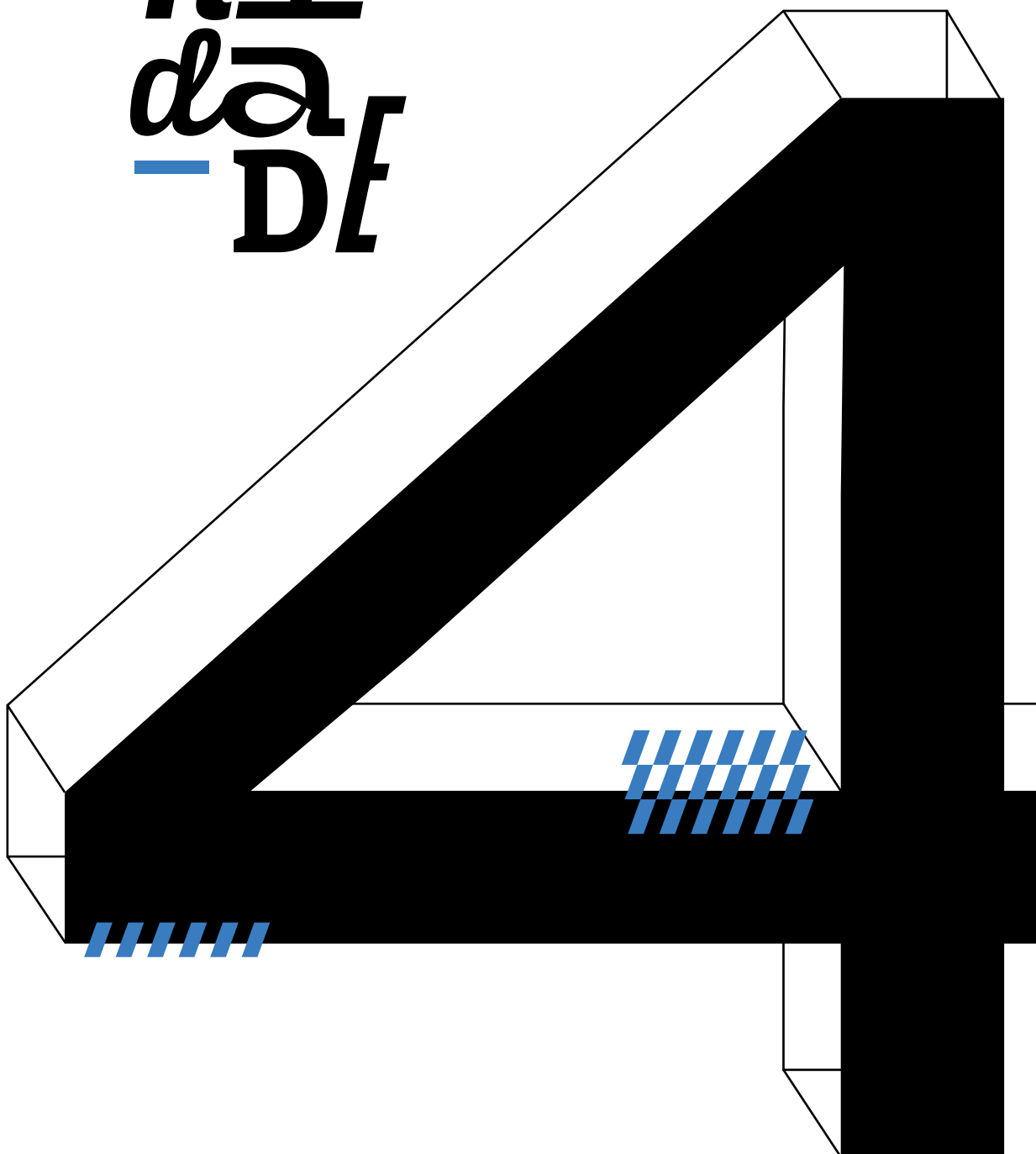
A alternativa B também está incorreta, porque o SQL é amplamente usado por profissionais de diversas áreas, como analistas e gestores.

A alternativa C está incorreta, pois o SQL é capaz de realizar tanto manipulações simples quanto consultas complexas com eficiência.

Já a alternativa D está correta, pois o SQL é compatível com diversos sistemas, como MySQL e PostgreSQL, ou que ampliem a devida aplicação.

Por fim, a alternativa E está incorreta, porque o SQL é declarativo, ou seja, o usuário apenas descreve o resultado esperado, e o sistema executa as operações possíveis.

**u
ni
da
-
De**





TEMA DE APRENDIZAGEM 6

JOINS EM SQL

MINHAS METAS

- Compreender o conceito e a importância dos JOINS em SQL.
- Identificar os diferentes tipos de JOINS e as respectivas aplicações práticas.
- Utilizar INNER JOIN para combinar dados de múltiplas tabelas.
- Aplicar LEFT JOIN e RIGHT JOIN em consultas relacionais.
- Explorar FULL OUTER JOIN para integrar dados de todas as tabelas.
- Construir consultas eficientes usando JOINS e cláusulas SQL.
- Resolver problemas comuns e aplicar boas práticas no uso de JOINS.

INICIE SUA JORNADA

Imagine que você está desenvolvendo o banco de dados para um sistema de reservas de hotel. O sistema precisa integrar informações de hóspedes, quartos e reservas, mas os dados estão armazenados em **tabelas separadas**. O gerente solicita um relatório consolidado que exiba os detalhes de cada reserva, como o nome do hóspede, o tipo do quarto e a data da reserva. Como você consultaria essas informações de forma integrada, garantindo que todas as tabelas sejam utilizadas de maneira eficiente para entregar o relatório solicitado?

O uso de JOINS é essencial para combinar dados de várias tabelas em um banco de dados relacional. Essa técnica permite criar consultas que integram informações de diferentes fontes, proporcionando uma visão completa e detalhada do sistema. Ao dominar os diferentes tipos de JOINS, como INNER JOIN, LEFT JOIN e FULL OUTER JOIN, você será capaz de unir dados de maneira prática e eficiente, resolvendo problemas comuns em sistemas que dependem de múltiplas tabelas. Entender como e quando usar JOINS é uma habilidade essencial para trabalhar com bancos de dados complexos.

Agora, imagine que você foi contratado para criar relatórios integrados no sistema de reservas de hotel mencionado. Para isso: (i) identifique as tabelas principais que precisam ser usadas para gerar o relatório, como “Hóspedes”, “Quartos” e “Reservas”; (ii) defina os atributos mais relevantes de cada tabela, como nome do hóspede, tipo de quarto e data da reserva; e (iii) descreva como as tabelas se relacionam entre si, identificando as chaves primárias e estrangeiras que conectam os dados.

Com base no estudo de caso proposto, reflita sobre os desafios encontrados ao relacionar dados de diferentes tabelas. Como o uso de JOINS pode ajudar a solucionar esses problemas e facilitar a criação de relatórios detalhados? Pense também em como essa habilidade poderá ser aplicada em outros contextos profissionais, ajudando você a lidar com sistemas complexos e a criar consultas que integram informações de forma precisa e eficiente.

**PLAY NO CONHECIMENTO**

Ouçá o nosso podcast sobre conteúdo avançado de SQL! Descubra como dominar JOINS para conectar tabelas, otimizar consultas e resolver problemas complexos em bancos de dados. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Aprenda os diferentes tipos de JOINS em SQL, como INNER, OUTER, FULL e muito mais, com o vídeo Bancos Relacionais! Conheça os JOINS do SQL. Assista agora!

https://youtu.be/165r4qUvp8Q?si=BpouJrDCV_8aeBlg



DESENVOLVA SEU POTENCIAL

CONCEITO E IMPORTÂNCIA DE JOINS

Os JOINS são ferramentas indispensáveis em SQL para conectar dados armazenados em diferentes tabelas de um banco de dados relacional. Essa funcionalidade é essencial, porque os bancos de dados relacionais seguem o princípio da normalização, em que as informações são distribuídas em várias tabelas para evitar redundâncias e garantir a integridade dos dados (Ricardo; Urban; Davis, 2021). Com JOINS, é possível **combinar** essas tabelas, criando uma visão unificada e estruturada, o que facilita a análise e o uso das informações

De forma prática, o JOIN é uma operação que combina registros de duas ou mais tabelas com base em uma relação lógica entre as colunas. Geralmente, essa relação é definida por chaves primárias e estrangeiras, que estabelecem a forma como os dados estão conectados (Silberschatz; Korth; Sudarshan, 2020). Essa operação permite criar consultas que consolidam as informações distribuídas, oferecendo uma visão integrada e simplificada.

Por exemplo, imagine o banco de dados de uma loja on-line. As informações sobre clientes, pedidos e produtos estão armazenadas em tabelas separadas. Com JOINS, é possível criar consultas que integram essas tabelas, exibindo detalhes, como o cliente que fez o pedido, os itens comprados e o valor total. Sem JOINS, seria necessário replicar essas informações em uma única tabela, o que aumentaria os riscos de inconsistência e dificultaria a manutenção do banco (Elmasri; Navathe, 2019).

Os bancos de dados relacionais seguem o princípio da normalização

Importância do JOIN em consultas relacionais

Os JOINS desempenham um papel crucial nas consultas de bancos de dados relacionais, que são projetados para armazenar dados de forma distribuída. Essa organização melhora a integridade dos dados e reduz redundâncias, mas também exige ferramentas eficazes para integrar as informações quando necessário (Allen; Owen, 2021). É aqui que os JOINS se destacam.

Com JOINS, você pode:

GERAR RELATÓRIOS COMPLEXOS E DETALHADOS:

Como uma lista de hóspedes, quartos e reservas em um sistema de hotelaria.

SIMPLIFICAR CONSULTAS:

Ao eliminar a necessidade de consultas manuais que exigiriam múltiplas operações e junções manuais.

MANTER A INTEGRIDADE DAS RELAÇÕES:

Garantindo que os dados combinados estejam corretamente conectados e alinhados.

Por exemplo, em um relatório que combine hóspedes, reservas e quartos, os JOINS garantem que todas as informações sejam exibidas corretamente, respeitando os relacionamentos entre as tabelas.

TIPOS DE JOIN

Existem diferentes tipos de JOINS, cada um projetado para atender a cenários específicos de consulta e integração de dados, assim como é ilustrado na Figura 1.

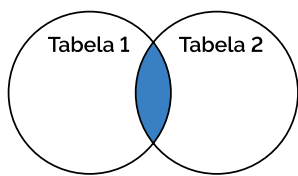
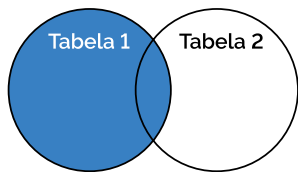
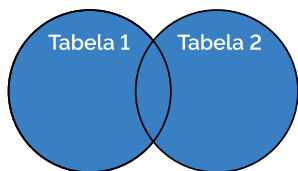
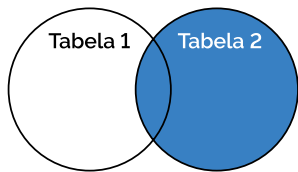
**INNER JOIN****LEFT JOIN****FULL OUTER JOIN****RIGHT JOIN**

Figura 1 - Diferentes tipos de JOINS / Fonte: o autor.

Descrição da Imagem: são exibidos quatro diagramas de Venn, cada um representando um tipo de JOIN (INNER, FULL OUTER, LEFT, RIGHT), com círculos nomeados "Tabela 1" e "Tabela 2". Áreas em cinza indicam as partes consideradas em cada operação. Fim da descrição.

Agora, vamos detalhar cada um desses tipos de JOINS, explicando as características e casos de uso.

INNER JOIN:

retorna apenas os registros que têm correspondência em ambas as tabelas. É o tipo mais comum de JOIN, usado para integrar dados totalmente relacionados.

LEFT JOIN:

inclui todos os registros da tabela à esquerda, mesmo que não tenham correspondência na tabela à direita.

RIGHT JOIN:

funciona de forma similar ao LEFT JOIN, mas prioriza os registros da tabela à direita.

FULL OUTER JOIN:

combina os registros de ambas as tabelas, incluindo aqueles que não têm correspondência em nenhuma das duas.

Cada tipo de JOIN atende a uma necessidade específica, possibilitando consultas que variam desde a integração total de tabelas até análises mais seletivas e específicas (Beaulieu, 2020).

INNER JOIN: conceito, sintaxe e aplicações

O **INNER JOIN** é o tipo de JOIN mais utilizado em SQL e serve para combinar registros de duas ou mais tabelas com base em uma correspondência lógica entre colunas relacionadas. Ele retorna apenas os registros que possuem correspondências em ambas as tabelas, ou seja, aquelas que atendem à condição definida na cláusula **ON** (Beaulieu, 2020).

Essa operação é amplamente usada em bancos de dados relacionais para **integrar dados distribuídos** em diferentes tabelas, garantindo consultas precisas e estruturadas. O **INNER JOIN** é projetado para conectar dados diretamente relacionados entre tabelas. Geralmente, essas conexões são baseadas em chaves primárias e estrangeiras.

Por exemplo, em um banco de dados que gerencia clientes e pedidos, a tabela **Clientes** pode ter uma chave primária **idCliente**, que é referenciada como chave estrangeira na tabela **Pedidos**. Um **INNER JOIN** entre essas tabelas pode combinar o nome do cliente com as informações de pedidos. Essa operação filtra os registros automaticamente, exibindo apenas aqueles que têm correspondência nas tabelas envolvidas (Allen; Owen, 2021).

A **sintaxe** básica do **INNER JOIN** é direta e consiste nos seguintes elementos:

```

SELECT colunas
FROM tabela1
INNER JOIN tabela2
ON tabela1.coluna = tabela2.coluna;

```

Na sintaxe anterior, **tabela1** e **tabela2** são as tabelas que serão combinadas. A **coluna** define a condição de correspondência entre as tabelas, geralmente baseada em chaves primárias e estrangeiras. Por fim, **SELECT** permite selecionar as colunas desejadas de ambas as tabelas.

Agora, vamos analisar um exemplo prático. Considere duas tabelas, as quais são apresentadas nos quadros 1 e 2.

IDCLIENTE	NOME	CIDADE
1	Maria Silva	São Paulo
2	João Pereira	Rio de Janeiro
3	Ana Costa	Curitiba

Quadro 1 - Tabela Clientes / Fonte: o autor.

A tabela apresentada no Quadro 1 armazena informações sobre os clientes, identificados unicamente pelo campo idCliente. Ela contém os nomes dos clientes e as cidades onde residem.

IDPEDIDO	IDCLIENTE	VALOR
101	1	150.00
102	2	200.00
103	1	300.00

Quadro 2 - Tabela Pedidos / Fonte: o autor.

Essa tabela (Quadro 2) registra os pedidos feitos pelos clientes. Cada pedido é identificado pelo campo `idPedido`, enquanto o campo `idCliente` relaciona o pedido ao cliente que o realizou. Além disso, o campo `valor` indica o valor do pedido. Agora, suponha que queremos listar os clientes junto com os valores de seus pedidos. Para isso, usamos o **INNER JOIN**:

```
SELECT Clientes.nome, Pedidos.idPedido, Pedidos.valor
FROM Clientes
INNER JOIN Pedidos
ON Clientes.idCliente = Pedidos.idCliente;
```

O **INNER JOIN** funciona verificando a condição definida na cláusula **ON**, que, no exemplo, é **Clientes.idCliente = Pedidos.idCliente**. Isso significa que ele compara os valores na coluna `idCliente` da tabela `Clientes` com os valores na mesma coluna da tabela `Pedidos`. Quando encontra correspondências, ele combina os registros dessas tabelas em um único resultado. Somente os registros que atendem a essa condição de igualdade são incluídos no resultado final, assim como é apresentado no Quadro 3 (Allen; Owen, 2021).

NOME	IDPEDIDO	VALOR
Maria Silva	101	150.00
João Pereira	102	200.00
Maria Silva	103	300.00

Quadro 3 - Resultado da operação **INNER JOIN** combinando registros das tabelas `Clientes` e `Pedidos`
Fonte: o autor.

No exemplo, o cliente “Maria Silva”, identificado pelo `idCliente = 1`, aparece duas vezes no resultado, porque realizou dois pedidos registrados na tabela `Pedidos`: os pedidos 101 e 103. Esses pedidos têm valores associados (150.00 e 300.00, respectivamente) que são exibidos no resultado. O cliente “João Pereira”, com

idCliente = 2, aparece uma vez, porque realizou apenas um pedido (idPedido 102), também registrado com o valor correspondente. Já a cliente “Ana Costa”, identificada pelo idCliente = 3, não aparece no resultado, pois não possui nenhum registro correspondente na tabela Pedidos. Assim, o INNER JOIN retorna exclusivamente os dados que têm correspondência em ambas as tabelas, ignorando os registros sem relação definida.

LEFT JOIN e RIGHT JOIN: diferenças e exemplos

Os JOINS externos, como **LEFT JOIN** e **RIGHT JOIN**, são utilizados para incluir registros de uma tabela, mesmo que não tenham correspondências na outra tabela. Essa abordagem é útil para análises que exigem a preservação de todos os registros de uma tabela principal, exibindo também as informações relacionadas de outra tabela quando disponíveis. A principal diferença entre esses dois JOINS está na prioridade dada às tabelas: o **LEFT JOIN** prioriza os registros da tabela à esquerda (especificada primeiro), enquanto o **RIGHT JOIN** prioriza os registros da tabela à direita (Allen; Owen, 2021).

O **LEFT JOIN** retorna todos os registros da tabela à esquerda junto com os registros correspondentes da tabela à direita. Caso não haja correspondência, os campos da tabela à direita aparecerão como **NULL**. Ele é útil para manter todos os dados da tabela principal, mesmo quando não há informações relacionadas na tabela secundária (Beaulieu, 2020).

A sintaxe básica do **LEFT JOIN** é:

```
SELECT colunas  
FROM tabela1  
LEFT JOIN tabela2  
ON tabela1.coluna = tabela2.coluna;
```

Aqui, **tabela1** é a tabela principal, cujos registros serão preservados, enquanto **tabela2** fornece as informações relacionadas.

Agora, vamos analisar um exemplo de LEFT JOIN. Considere as tabelas ilustradas nos **quadros 1 e 2, vistos anteriormente**. Agora, queremos listar todos os clientes, incluindo aqueles que não possuem pedidos registrados. Usamos o LEFT JOIN:

```
SELECT Clientes.nome, Pedidos.idPedido, Pedidos.valor
FROM Clientes
LEFT JOIN Pedidos
ON Clientes.idCliente = Pedidos.idCliente;
```

No resultado, o qual é apresentado no Quadro 4, “Ana Costa” aparece com valores NULL nas colunas idPedido e valor, pois ela não tem correspondência na tabela Pedidos.

NOME	IDPEDIDO	VALOR
Maria Silva	101	150.00
Maria Silva	103	300.00
João Pereira	102	200.00
Ana Costa	NULL	NULL

Quadro 4 - Resultado da operação LEFT JOIN combinando registros das tabelas Clientes e Pedidos
Fonte: o autor.

Agora, vamos conhecer o **RIGHT JOIN**. Ele funciona de maneira semelhante ao LEFT JOIN, mas prioriza os registros da tabela à direita. Ele retorna todos os registros da tabela à direita, junto com os correspondentes da tabela à esquerda. Quando não há correspondência, os campos da tabela à esquerda aparecem como NULL (Beaulieu, 2020).

A sintaxe básica do RIGHT JOIN é:

```
SELECT colunas
FROM tabela1
RIGHT JOIN tabela2
ON tabela1.coluna = tabela2.coluna;
```

Aqui, **tabela2** é a tabela cujos registros serão preservados, enquanto **tabela1** fornece as informações relacionadas.

Ainda usando as **tabelas ilustradas anteriormente**, suponha que queremos listar todos os pedidos, incluindo aqueles que não têm clientes associados. Usamos o RIGHT JOIN:

```
SELECT Clientes.nome, Pedidos.idPedido, Pedidos.valor
FROM Clientes
RIGHT JOIN Pedidos
ON Clientes.idCliente = Pedidos.idCliente;
```

Nesse caso, como todos os registros da tabela Pedidos têm correspondências na tabela Clientes, o resultado é idêntico ao **INNER JOIN**. Entretanto, em cenários nos quais a tabela à direita contém registros sem relação, esses apareceriam com valores **NULL** para as colunas da tabela à esquerda.

NOME	IDPEDIDO	VALOR
Maria Silva	101	150.00
Maria Silva	103	300.00
João Pereira	102	200.00
Ana Costa	NULL	NULL

Quadro 5 - Resultado da operação RIGHT JOIN combinando registros das tabelas Clientes e Pedidos
Fonte: o autor.

A principal diferença entre **LEFT JOIN** e **RIGHT JOIN** está na tabela cujos registros são preservados. Enquanto o **LEFT JOIN** garante que todos os registros da tabela à esquerda sejam exibidos, o **RIGHT JOIN** faz o mesmo para a tabela à direita. Ambos os **JOINS** são úteis em análises em que informações completas de uma tabela devem ser mantidas, mesmo quando não há correspondências na outra tabela (Beaulieu, 2020).

FULL OUTER JOIN: uso e casos específicos

O **FULL OUTER JOIN** é uma operação em SQL que combina os resultados de um **LEFT JOIN** e um **RIGHT JOIN**, retornando todos os registros de ambas as tabelas, independentemente de terem correspondências. Para os registros que não têm correspondência na outra tabela, os valores são preenchidos com **NULL**. Essa abordagem é útil em cenários em que é necessário analisar dados completos de duas tabelas, mesmo que nem todos os registros estejam relacionados (Beaulieu, 2020).

Enquanto os **JOINS** internos e externos parciais (como o **LEFT** e o **RIGHT JOIN**) focam em registros correspondentes ou prioritários de uma tabela específica, o **FULL OUTER JOIN** combina tudo. Ele preserva todos os registros das tabelas envolvidas, garantindo que nenhuma informação seja excluída, mesmo quando não há relação entre os dados.

Por exemplo, em um banco de dados que armazena informações de clientes e pedidos, o **FULL OUTER JOIN** pode ser usado para gerar um relatório completo que mostre tanto os clientes que fizeram pedidos quanto aqueles que não realizaram nenhuma compra. Da mesma forma, ele incluirá pedidos que não possuem clientes associados, caso existam.

A sintaxe do **FULL OUTER JOIN** é:

```
SELECT colunas
FROM tabela1
FULL OUTER JOIN tabela2
ON tabela1.coluna = tabela2.coluna;
```

Para entender melhor, vamos analisar um exemplo de **FULL OUTER JOIN**. Considere as tabelas apresentadas nos quadros 6 e 7.

IDCLIENTE	NOME	CIDADE
1	Maria Silva	São Paulo
2	João Pereira	Rio de Janeiro
3	Ana Costa	Curitiba

Quadro 6 - Tabela Clientes / Fonte: o autor.

IDPEDIDO	IDCLIENTE	VALOR
101	1	150.00
102	2	200.00
103	1	300.00
104	4	500.00

Quadro 7 - Tabela Pedidos / Fonte: o autor.

Agora, queremos gerar um relatório que inclua todos os clientes e todos os pedidos, mesmo aqueles que não têm correspondências entre as tabelas. Para isso, usamos o **FULL OUTER JOIN**:

```
SELECT Clientes.nome, Pedidos.idPedido, Pedidos.valor
FROM Clientes
FULL OUTER JOIN Pedidos
ON Clientes.idCliente = Pedidos.idCliente;
```

No resultado, apresentado no Quadro 8, todos os registros das tabelas Clientes e Pedidos são exibidos: “Maria Silva” aparece duas vezes, porque realizou dois

pedidos, conforme as correspondências na tabela Pedidos. “João Pereira” aparece com o único pedido. “Ana Costa” aparece com valores NULL em idPedido e valor, pois não realizou nenhum pedido. O pedido 104 aparece com valores NULL em nome, pois não está associado a nenhum cliente.

NOME	IDPEDIDO	VALOR
Maria Silva	101	150.00
Maria Silva	103	300.00
João Pereira	102	200.00
Ana Costa	NULL	NULL
NULL	104	500.00

Quadro 8 - Resultado da operação FULL OUTER JOIN combinando registros das tabelas Clientes e Pedidos
Fonte: o autor.

O FULL OUTER JOIN é útil em cenários que requerem uma visão abrangente dos dados, como:

AUDITORIAS DE DADOS:

Verificar registros ausentes em tabelas relacionadas, como identificar clientes sem pedidos e pedidos sem clientes associados.

RELATÓRIOS CONSOLIDADOS:

Criar relatórios completos que envolvam todas as informações disponíveis, mesmo quando os registros não estão diretamente conectados.

ANÁLISES DE QUALIDADE DE DADOS:

Detectar inconsistências ou lacunas nos dados armazenados em diferentes tabelas.

Embora o FULL OUTER JOIN ofereça uma visão ampla dos dados, ele pode gerar resultados extensos e incluir muitos valores NULL, o que pode dificultar a interpretação. Por isso, deve ser usado com cautela em consultas que exigem precisão e clareza (Allen; Owen, 2021).

BOAS PRÁTICAS E DESAFIOS NO USO DE JOINS

Os JOINS são ferramentas poderosas para integrar dados em bancos de dados relacionais, mas o uso inadequado deles pode levar a problemas de desempenho, resultados inesperados e até inconsistências nos dados.

Compreender as boas práticas e estar ciente dos desafios mais comuns é essencial para aproveitar ao máximo as operações de JOIN e garantir que as consultas sejam eficientes e confiáveis (Elmasri; Navathe, 2019).

Estratégias para otimizar consultas com JOINS

Otimizar consultas com JOINS é crucial em sistemas que lidam com grandes volumes de dados (Silberschatz; Korth; Sudarshan, 2021). Seguir estratégias específicas pode ajudar a melhorar o desempenho e evitar sobrecargas no banco de dados:

SELECIONE APENAS AS COLUNAS NECESSÁRIAS:

Evite usar SELECT *. Em vez disso, escolha explicitamente as colunas que serão usadas na consulta. Isso reduz a quantidade de dados processados e melhora o desempenho.

USE ÍNDICES NAS COLUNAS DE JUNÇÃO:

Certifique-se de que as colunas usadas na cláusula ON possuem índices adequados, como chaves primárias ou índices criados manualmente. Isso acelera a busca por correspondências entre tabelas.

EVITE JOINS DESNECESSÁRIOS:

Inclua apenas as tabelas e condições essenciais para a consulta. JOINS redundantes podem aumentar o tempo de execução e dificultar a leitura da consulta.

APLIQUE FILTROS PRECOCEMENTE:

Use cláusulas WHERE para filtrar os dados antes de realizar o JOIN, quando possível. Isso reduz a quantidade de dados processados na junção.

VERIFIQUE O IMPACTO DA ORDEM DOS JOINS:

Em consultas complexas com várias tabelas, a ordem dos JOINS pode influenciar o desempenho. Teste diferentes combinações e analise os planos de execução para identificar a abordagem mais eficiente.

Essas práticas ajudam a garantir que os JOINS sejam eficientes, mesmo em bancos de dados com **milhões de registros**, evitando consultas lentas e ineficazes.

Problemas comuns e como evitá-los

Apesar da utilidade, os JOINS podem introduzir desafios, especialmente quando usados de forma inadequada (Ricardo; Urban; Davis, 2021). A seguir, abordamos os problemas mais comuns e como evitá-los:

JOIN CARTESIANOS INDESEJADOS:

ocorrem quando não há uma condição clara de junção entre as tabelas, resultando em um número excessivo de combinações. Para evitar isso, sempre defina condições específicas na cláusula ON ou USING.

REGISTROS DUPLICADOS NO RESULTADO:

JOINS mal configurados podem retornar registros duplicados, especialmente em relações de um para muitos. Analise as relações entre tabelas e use cláusulas, como DISTINCT, ou funções de agregação (GROUP BY) para resolver esse problema.

VALORES NULL INESPERADOS:

JOINS externos, como LEFT JOIN ou FULL OUTER JOIN, podem gerar valores NULL para colunas sem correspondência. Considere usar funções, como COALESCE, para tratar esses valores e tornar os resultados mais legíveis.

FALTA DE COMPREENSÃO DAS RELAÇÕES:

JOINS incorretos, muitas vezes, resultam de um entendimento limitado das relações entre tabelas. Antes de criar a consulta, analise o esquema do banco de dados e compreenda como as tabelas estão relacionadas.

Ao estar ciente desses desafios e seguir as melhores práticas, você estará mais bem preparado para criar consultas JOIN eficientes e evitar problemas comuns que podem comprometer a integridade e o desempenho do banco de dados.

**EM FOCO**

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

Neste tema, você aprendeu os diferentes tipos de JOINS em SQL, incluindo INNER JOIN, LEFT JOIN, RIGHT JOIN e FULL OUTER JOIN, além das respectivas aplicações práticas. Esses conceitos são fundamentais para integrar dados de múltiplas tabelas em bancos de dados relacionais, permitindo consultas mais completas e detalhadas.

Resgatando o estudo de caso inicial, no qual você precisava criar um relatório para o sistema de reservas de hotel, podemos, agora, propor uma solução prática. Para integrar as tabelas “Hóspedes”, “Quartos” e “Reservas”, um INNER JOIN seria usado para combinar dados onde há correspondências entre as tabelas. Por exemplo, você poderia consultar o nome dos hóspedes, o tipo de quarto reservado e a data da reserva, conectando as tabelas por meio de chaves primárias e estrangeiras, como idHospede e idQuarto.

Já um LEFT JOIN poderia ser utilizado para identificar reservas onde informações de hóspedes ou quartos estão ausentes, ajudando a localizar possíveis inconsistências no sistema. Com os comandos apropriados, você pode gerar relatórios que atendam às demandas do gerente, exibindo informações precisas e integradas.

O que você aprendeu neste tema é essencial para lidar com sistemas reais e complexos no mercado de trabalho. As habilidades de integrar e relacionar dados usando JOINS permitem que você desenvolva relatórios avançados e otimize processos de análise de dados. Além disso, compreender as limitações e as boas práticas no uso de JOINS te prepara para solucionar problemas de desempenho e inconsistências em bancos de dados relacionais.

Essa competência é amplamente valorizada em áreas, como desenvolvimento de sistemas, ciência de dados e administração de bancos de dados.

VAMOS PRATICAR

1. Os bancos de dados relacionais organizam informações em tabelas separadas para garantir a eficiência e a integridade dos dados. Para consultas que integram dados distribuídos, utiliza-se operações específicas que conectam as tabelas de forma lógica. Essas operações permitem a combinação de informações de diferentes tabelas, gerando resultados consolidados.

Com base nos tipos de JOINS, analise as afirmativas a seguir:

- I - LEFT JOIN e RIGHT JOIN resultam em resultados idênticos em qualquer tipo de consulta.
- II - O FULL OUTER JOIN combina os registros das tabelas apenas quando há correspondência entre elas.
- III - O LEFT JOIN retorna todos os registros da tabela à esquerda, independentemente de haver correspondência na tabela à direita, e adiciona apenas os registros correspondentes da tabela à direita.

É correto o que se afirma em:

- a) I, apenas.
 - b) III, apenas.
 - c) I e II, apenas.
 - d) II e III, apenas.
 - e) I, II e III.
2. Nos bancos de dados relacionais, as operações específicas são usadas para integrar informações de tabelas específicas com base em condições lógicas. Essas operações permitem realizar consultas que retornem resultados precisos e estruturados de acordo com contratos previamente definidos. Essas ferramentas são fundamentais para o trabalho com dados distribuídos em várias tabelas.

Com base no conceito e na aplicação do INNER JOIN, assinale a alternativa correta:

- a) O INNER JOIN conecta tabelas e retorna registros completos de ambas, mesmo que não tenha relação lógica.
- b) O INNER JOIN combina tabelas e retorna todos os registros, mesmo aqueles que não possuem correspondências.
- c) O INNER JOIN permite combinar registros de tabelas diferentes, mas não exige condições lógicas para essa integração.
- d) O INNER JOIN combina tabelas de forma automática, sem depender de colunas relacionadas ou condições de filtragem.
- e) O INNER JOIN conecta tabelas e retorna somente os registros que possuem correspondência, com base nas condições definidas.

VAMOS PRATICAR

3. Em consultas Structured Query Language (SQL), existem operações específicas que permitem combinar registros de tabelas diferentes de formato amplo. Essas operações são especialmente úteis em análises em que é necessário incluir informações completas de ambas as tabelas, mesmo quando não há correspondência entre os dados.

Com base no conceito do FULL OUTER JOIN, analise as afirmativas a seguir:

- I - O FULL OUTER JOIN inclui todos os registros de ambas as tabelas.
- II - O FULL OUTER JOIN funciona como um LEFT JOIN, mas prioriza os registros da tabela à direita.
- III - O FULL OUTER JOIN retorna apenas os registros que possuem correspondência entre as tabelas envolvidas.
- IV - O FULL OUTER JOIN remove duplicatas e apresenta somente os registros únicos das duas tabelas combinadas.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

VAMOS PRATICAR

ALLEN, G. R.; OWEN, W. A. **SQL in 10 Minutes**: Sams Teach Yourself. 5. ed. [S. l.]: Pearson Education, 2021.

BEAULIEU, A. **Learning SQL**: Master SQL Fundamentals. 3. ed. Sebastopol: O'Reilly Media, 2020.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2019.

RICARDO, C. M.; URBAN, S. D.; DAVIS, K. C. **Databases Illuminated**. 4. ed. Burlington: Jones & Bartlett Learning, 2022.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistemas de Banco de Dados**. 7. ed. Rio de Janeiro: Elsevier, 2020.

CONFIRA SUAS RESPOSTAS

1. Opção B.

I. Incorreta. LEFT JOIN e RIGHT JOIN priorizam tabelas diferentes, resultando em conjuntos distintos de dados.

II. Incorreta. O FULL OUTER JOIN combina todos os registros de ambas as tabelas, incluindo aqueles que não possuem correspondências.

III. Correta. O LEFT JOIN prioriza todos os registros da tabela à esquerda e inclui informações da tabela à direita quando há correspondência.

2. Opção E.

A. Incorreta. O INNER JOIN retorna somente os registros que possuem uma correspondência lógica definida.

B. Incorreta. O INNER JOIN retorna apenas registros que possuem correspondência entre as tabelas envolvidas.

C. Incorreta. O INNER JOIN exige uma condição lógica para determinar os registros que serão combinados.

D. Incorreta. O INNER JOIN depende de colunas relacionadas e condições lógicas para realizar a combinação entre tabelas.

E. Correta. Essa é uma funcionalidade do INNER JOIN, que retorna apenas registros que atendem às condições lógicas matemáticas na consulta.

3. Opção A.

I. Correta. Essa é uma descrição resumida e precisa do funcionamento do FULL OUTER JOIN.

II. Incorreta. O FULL OUTER JOIN combina os registros de ambas as tabelas, enquanto o LEFT JOIN prioriza os registros da tabela à esquerda.

III. Incorreta. O FULL OUTER JOIN não se limita aos registros correspondentes; ele inclui todos os registros de ambas as tabelas.

IV. Incorreta. O FULL OUTER JOIN não remove duplicatas; ele exibe todos os registros de ambas as tabelas.



TEMA DE APRENDIZAGEM 7

SQL: SUBCONSULTAS E JUNÇÕES AVANÇADAS

MINHAS METAS

- Compreender o conceito de subconsultas em SQL.
- Aplicar subconsultas em cláusulas WHERE para filtrar dados.
- Identificar a diferença entre subconsultas simples e correlacionadas.
- Executar junções com mais de duas tabelas em SQL.
- Utilizar SELF JOIN para relacionar registros dentro da mesma tabela.
- Analisar e otimizar consultas complexas para obter um melhor desempenho.
- Implementar índices para melhorar a performance de consultas SQL.

INICIE SUA JORNADA

Em uma empresa de tecnologia, você foi designado para criar relatórios detalhados que auxiliem o departamento de vendas. O gerente precisa de uma análise que mostre os clientes mais importantes, considerando o valor total das compras, mas os dados estão espalhados em três tabelas: **Clientes**, **Pedidos** e **Produtos**. Além disso, ele quer um filtro para exibir apenas as compras de produtos com valores superiores a R\$ 500,00. A tarefa exige que você combine as tabelas de forma eficiente, filtrando os dados relevantes e calculando os valores totais de forma precisa. Como você abordaria esse problema?

A resolução desse tipo de desafio é crucial no mundo profissional, especialmente em áreas, como análise de vendas, gestão de estoque e acompanhamento de desempenho. Utilizar consultas SQL avançadas, como subconsultas e junções, permite transformar dados dispersos em informações estratégicas. Isso não apenas otimiza a tomada de decisões, mas também garante precisão e confiabilidade nas análises. Profissionais capazes de lidar com esses desafios são valorizados pela habilidade em conectar informações e gerar insights úteis para a organização.

Agora, imagine o seguinte cenário prático: em um banco de dados, você tem uma tabela **Clientes** com os nomes e as cidades dos clientes, uma tabela **Pedidos** com informações sobre cada compra (como cliente e produto comprado) e uma tabela **Produtos** contendo os preços e as categorias dos itens. Para atender à demanda do gerente, você precisará combinar esses dados, identificando quais clientes compraram produtos acima de R\$ 500,00 e calculando o valor total gasto por cada um.

Pense nos desdobramentos dessa habilidade: como subconsultas e junções avançadas podem ser usadas para **gerenciar dados** em sistemas complexos, como e-commerces ou ERPs? Quais estratégias são mais eficientes para lidar com grandes volumes de dados? Refletir sobre essas questões ajudará a consolidar o seu aprendizado e a aplicar os conceitos deste tema de aprendizagem no mercado profissional. Vamos explorar essas ferramentas avançadas?

**PLAY NO CONHECIMENTO**

Quer aprender a levar as suas consultas SQL ao próximo nível? Ouça o nosso podcast sobre a otimização de consultas e descubra técnicas que farão a diferença no desempenho dos seus sistemas! **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Assista ao vídeo a seguir para recordar o que são subconsultas SQL e como podemos extrair dados com SQL. <https://www.youtube.com/watch?v=XXT8G0dnLmY>



DESENVOLVA SEU POTENCIAL

CONCEITO E DEFINIÇÃO DE SUBCONSULTAS

Subconsultas são ferramentas poderosas em SQL que permitem criar consultas mais sofisticadas e resolver problemas complexos de forma modular. Vamos explorar o conceito de subconsultas, a aplicação delas em cláusulas, como WHERE, e o funcionamento das subconsultas correlacionadas, que trazem uma camada adicional de complexidade e versatilidade às operações (Allen; Owen, 2021).



INDICAÇÃO DE LIVRO

QL for Data Scientists: A Beginner's Guide for Building Datasets for Analysis

Se você deseja aprimorar as suas habilidades em SQL e trabalhar com ciência de dados, o livro SQL for Data Scientists: A Beginner's Guide for Building Datasets for Analysis, de Renée M. P. Teate, é uma excelente escolha. Ele aborda desde conceitos fundamentais, como a criação de consultas eficientes para explorar, filtrar e ordenar dados, até técnicas avançadas, como o uso de junções para combinar informações de múltiplas tabelas, funções de janela e expressões de tabela comuns.



Subconsultas, também chamadas de **consultas aninhadas**, constituem uma técnica poderosa no SQL que permite incluir uma consulta dentro de outra. Essa funcionalidade é particularmente útil quando há a necessidade de resolver problemas complexos por etapas, permitindo que uma consulta principal utilize os resultados de uma consulta interna como referência (Allen; Owen, 2021).

Conceitualmente, subconsultas oferecem uma abordagem modular para criar consultas sofisticadas. Elas permitem que os dados sejam filtrados, calculados ou organizados em uma consulta separada, cujo resultado é integrado à consulta principal. Por exemplo, em um cenário de banco de dados relacional, subconsultas podem ser usadas para encontrar valores máximos ou mínimos, filtrar registros com base em critérios dinâmicos ou comparar dados de diferentes tabelas de forma eficiente (Beaulieu, 2020).



O papel principal de uma subconsulta é atuar como uma etapa intermediária, simplificando problemas que, de outra forma, exigiriam consultas mais extensas e menos legíveis. Elas podem ser aplicadas em diversas partes de uma consulta SQL, como cláusulas **WHERE**, **FROM**, e até mesmo dentro da cláusula **SELECT**.

O papel principal de uma subconsulta é atuar como uma etapa intermediária

Subconsultas em cláusulas **WHERE**

Subconsultas são amplamente utilizadas na cláusula **WHERE** para criar filtros dinâmicos e precisos em consultas SQL. Essa aplicação permite que a consulta principal dependa de resultados calculados pela subconsulta, criando condições específicas e flexíveis. Esse recurso é indispensável em cenários em que os critérios de seleção não podem ser definidos diretamente, mas dependem de dados calculados em outras tabelas ou agrupamentos.

A sintaxe básica de uma subconsulta na cláusula **WHERE** segue o seguinte formato:

```
SELECT coluna1, coluna2
FROM tabela
WHERE coluna = (SELECT coluna
                FROM tabela_secundaria
                WHERE condicao);
```

Nesse modelo, a subconsulta é processada **primeiro**, retornando um conjunto de resultados que servem como **critério** para filtrar os registros da consulta externa. Isso permite criar filtros complexos sem comprometer a legibilidade ou a eficiência da consulta (Beaulieu, 2020).

Considere um exemplo prático. Suponha que você tenha uma tabela **Clientes** que armazena informações básicas, como nomes e cidades, e uma tabela **Pedidos**, que registra os pedidos realizados, incluindo valores e IDs dos clientes. Para listar os nomes dos clientes que fizeram pedidos acima de R\$ 1.000,00, podemos usar a seguinte consulta:

```
SELECT nome
FROM Clientes
WHERE idCliente IN (SELECT idCliente
                    FROM Pedidos
                    WHERE valor > 1000);
```

Nesse exemplo, a subconsulta **SELECT idCliente FROM Pedidos WHERE valor > 1000** é executada primeiro, retornando todos os IDs de clientes cujos pedidos ultrapassaram o valor especificado. Esses IDs são, então, utilizados pela consulta principal para buscar os nomes correspondentes na tabela **Clientes**.

O uso de subconsultas em cláusulas **WHERE** oferece várias vantagens (Beaulieu, 2020). Elas permitem resolver problemas complexos em etapas, simplificando a lógica da consulta principal. Além disso, subconsultas são essenciais para trabalhar com bancos de dados normalizados, em que as informações estão distribuídas entre várias tabelas para reduzir redundâncias.

Subconsultas correlacionadas

Subconsultas correlacionadas são uma **variação avançada** de subconsultas, em que a consulta interna depende de valores da consulta externa para ser executada. Essa relação dinâmica entre as consultas adiciona uma camada de complexidade, mas também oferece flexibilidade para resolver problemas que não podem ser abordados com subconsultas convencionais.

Diferentemente de uma subconsulta tradicional, que é executada apenas uma vez e fornece um conjunto fixo de resultados, uma subconsulta correlacionada é avaliada repetidamente, uma vez para cada registro processado pela consulta externa. Essa interação permite criar filtros e condições que variam de acordo com os valores de cada linha analisada (Elmasri; Navathe, 2019).

A sintaxe de uma subconsulta correlacionada segue o mesmo padrão, mas utiliza referências diretas a colunas da consulta externa. Observe o exemplo a seguir:

```
SELECT nome
FROM Clientes c
WHERE EXISTS (SELECT 1
               FROM Pedidos p
               WHERE p.idCliente = c.idCliente
               AND p.valor > 1000);
```

Nesse caso, a subconsulta interna acessa a coluna **idCliente** da consulta externa para verificar se existe pelo menos um pedido com valor superior a R\$ 1.000,00 associado a cada cliente. A cláusula **EXISTS** retorna verdadeiro para os clientes que atendem a essa condição.

Para clarificar ainda mais, considere outro exemplo. Vamos revisar as tabelas **Clientes** e **Pedidos**. Suponha que você queira listar os nomes dos clientes que têm um pedido cujo valor é maior que o valor médio de todos os pedidos na tabela **Pedidos**. Uma subconsulta correlacionada é ideal para esse caso, pois avalia a condição dinamicamente para cada cliente. A consulta seria:

```
SELECT nome
FROM Clientes c
WHERE EXISTS (SELECT 1
               FROM Pedidos p
               WHERE p.idCliente = c.idCliente
               AND p.valor > (SELECT AVG(valor) FROM Pedidos
                               ));
```

Aqui, o valor médio dos pedidos é calculado pela subconsulta tradicional (**SELECT AVG(valor) FROM Pedidos**). A subconsulta correlacionada verifica, para cada cliente, se há algum pedido cujo valor exceda essa média. Somente os clientes que atenderem a esse critério aparecerão no resultado.



Reiterando, subconsultas correlacionadas são valiosas para resolver problemas que exigem condições dependentes de registros específicos. No entanto, a natureza dinâmica pode torná-las menos eficientes em bancos de dados grandes, devido à necessidade de reavaliar a subconsulta para cada linha processada pela consulta externa (Elmasri; Navathe, 2019).

Quando possível, considere reestruturar a lógica utilizando JOINS ou outras técnicas para melhorar a performance.

JUNÇÕES AVANÇADAS

Exploraremos, agora, junções avançadas em SQL, incluindo como combinar mais de duas tabelas, aplicar SELF JOIN para relações internas e construir consultas complexas que integram múltiplas fontes de dados.

Junções com mais de duas tabelas

Enquanto as junções tradicionais normalmente envolvem a combinação de duas tabelas, cenários mais complexos frequentemente exigem a integração de dados provenientes de três ou mais tabelas. Junções com múltiplas tabelas são uma aplicação avançada do SQL, em que cada tabela é conectada à próxima por meio de condições lógicas definidas em cláusulas **ON**.

A lógica das junções com múltiplas tabelas é uma extensão direta das junções simples. A consulta é estruturada de forma incremental, em que cada tabela adicional é anexada por meio de um JOIN adicional. Para garantir clareza e eficiência, é essencial que cada relação seja bem definida, evitando ambiguidades ou erros (Silberschatz; Korth; Sudarshan, 2020).

Considere um exemplo prático envolvendo as tabelas **Clientes**, **Pedidos** e **Produtos**. Suponha que queremos **listar** os nomes dos clientes, os itens que eles compraram e os valores desses itens. Essa consulta requer a combinação de três tabelas:

```
SELECT c.nome AS NomeCliente, p.nomeProduto AS NomeProduto, ped.valor AS  
Valor  
FROM Clientes c  
INNER JOIN Pedidos ped ON c.idCliente = ped.idCliente  
INNER JOIN Produtos p ON ped.idProduto = p.idProduto;
```

Nesse exemplo, a tabela **Clientes** é conectada à tabela **Pedidos** por meio da coluna **idCliente**. Em seguida, a tabela **Pedidos** é combinada à tabela **Produtos** utilizando a coluna **idProduto**. Essas conexões permitem integrar informações dispersas e gerar um resultado unificado com o nome do cliente, o produto adquirido e o valor correspondente.

Ao trabalhar com junções envolvendo múltiplas tabelas, é importante assegurar que todas as relações entre tabelas sejam claramente definidas, evitando resultados errôneos ou inesperados, como a duplicação desnecessária de registros (Silberschatz; Korth; Sudarshan, 2020). Além disso, garantir que as tabelas envolvidas possuam índices adequados nas colunas utilizadas nas condições de junção pode melhorar significativamente o desempenho da consulta. Revisar os resultados para confirmar que as combinações estão corretas também é uma prática indispensável.

Uso de SELF JOIN

O **SELF JOIN** é uma técnica especial de junção em SQL que permite unir uma tabela a ela mesma. Essa abordagem é útil em situações em que os da-

dos de uma tabela precisam ser comparados ou relacionados internamente. Apesar de utilizar o mesmo princípio de outras junções, o **SELF JOIN** exige o uso de **alias**es (apelidos) para diferenciar as instâncias da tabela durante a consulta (Ricardo; Urban; Davis, 2022).

O **SELF JOIN** é amplamente aplicado em alguns cenários, como análise de hierarquias, comparações entre registros ou quando há a necessidade de calcular relações internas, tais como conexões entre funcionários e gerentes ou comparações entre transações de diferentes datas (Ricardo; Urban; Davis, 2022).

A sintaxe básica de um **SELF JOIN** é semelhante à de outros JOINS, mas utiliza a mesma tabela com **alias**es para criar duas instâncias virtuais:

```
SELECT t1.coluna1, t2.coluna2
FROM tabela t1
INNER JOIN tabela t2
ON t1.coluna_chave = t2.coluna_chave;
```

Aqui, a tabela é referenciada duas vezes, com alias

t1 e **t2** que permitem diferenciar os registros durante a operação de junção.

Agora, para conhecermos um exemplo prático, considere uma tabela chamada **Funcionarios** que contém informações sobre os funcionários de uma empresa, incluindo IDs, nomes e IDs dos respectivos gerentes, conforme Quadro 1. O objetivo é criar uma consulta que liste os nomes dos funcionários junto com os nomes dos respectivos gerentes. A estrutura da tabela seria algo como:

IDFUNCIONARIO	NOME	IDGERENTE
1	Ana Costa	3
2	João Pereira	3
3	Carlos Silva	NULL

Quadro 1 - Informações dos funcionários de uma empresa / Fonte: o autor.

Para conectar os funcionários aos respectivos gerentes, utilizamos um SELF JOIN:

```
SELECT f1.nome AS NomeFuncionario, f2.nome AS NomeGerente
FROM Funcionarios f1
LEFT JOIN Funcionarios f2 ON f1.idGerente =
f2.idFuncionario;
```

Nesse exemplo, a tabela **Funcionarios** é referenciada duas vezes, usando os aliases **f1** para os funcionários e **f2** para os gerentes. A condição de junção (**f1.idGerente = f2.idFuncionario**) conecta cada funcionário ao registro do gerente correspondente. A cláusula **LEFT JOIN** é usada para garantir que funcionários sem gerente, como Carlos Silva, também sejam incluídos no resultado. O resultado dessa consulta seria algo próximo do exibido a seguir:

NOMEFUNCIONARIO	NOMEGERENTE
Ana Costa	Carlos Silva
João Pereira	Carlos Silva
Carlos Silva	NULL

Quadro 2 - Resultado da consulta SELF JOIN na tabela Funcionários / Fonte: o autor.

O SELF JOIN é um recurso **valioso** para trabalhar com relacionamentos internos em dados tabulares (Ricardo; Urban; Davis, 2022). No entanto, embora o SELF JOIN seja uma ferramenta poderosa, ele pode se tornar ineficiente em **tabelas grandes**, devido à necessidade de comparar registros internamente.

Para otimizar o desempenho, garanta que as colunas utilizadas na junção estejam devidamente indexadas. Além disso, certifique-se de que as condições de junção sejam bem definidas para evitar resultados errôneos ou duplicados.

Exemplos de junções complexas

Junções complexas em SQL permitem integrar dados de múltiplas tabelas para resolver problemas mais sofisticados e gerar relatórios detalhados (Beaulieu, 2020). Vamos imaginar o cenário prático de um e-commerce, em que os dados estão organizados em quatro tabelas: **Clientes**, **Pedidos**, **ItensPedido** e **Produtos**.



Na tabela **Clientes**, temos informações básicas, como o nome de cada cliente e o respectivo identificador único, chamado `idCliente`. Já na tabela **Pedidos**, cada pedido é registrado com o devido identificador (`idPedido`), o cliente que o realizou (`idCliente`) e o valor total do pedido. A tabela **ItensPedido** detalha os produtos adquiridos em cada pedido, armazenando o identificador do pedido (`idPedido`), o produto comprado (`idProduto`) e a quantidade adquirida. Por fim, a tabela **Produtos** contém os nomes dos produtos, os respectivos identificadores (`idProduto`) e os preços unitários (Beaulieu, 2020).

Suponha que você queira listar os nomes dos clientes, os produtos que compraram, a quantidade adquirida e o valor total de cada pedido. Para isso, realizaremos uma junção que conecta as quatro tabelas usando os relacionamentos entre seus identificadores. A consulta pode ser escrita da seguinte maneira:

```
SELECT Clientes.nome AS NomeCliente, Produtos.nomeProduto AS Produto,  
       ItensPedido.quantidade AS Quantidade,  
       (ItensPedido.quantidade * Produtos.preco) AS ValorItem,  
       Pedidos.totalPedido AS TotalPedido  
FROM Clientes  
INNER JOIN Pedidos ON Clientes.idCliente = Pedidos.idCliente  
INNER JOIN ItensPedido ON Pedidos.idPedido = ItensPedido.idPedido  
INNER JOIN Produtos ON ItensPedido.idProduto = Produtos.idProduto;
```

Primeiro, conectamos a tabela **Clientes** à tabela **Pedidos** por meio da coluna `idCliente`, para identificar quais pedidos pertencem a cada cliente. Em seguida, a tabela **Pedidos** é vinculada à tabela **ItensPedido**, que detalha os produtos incluídos em cada pedido. Por último, usamos a coluna `idProduto` para conectar a tabela **ItensPedido** à tabela **Produtos**, o que permite exibir os nomes e os preços dos produtos comprados (Beaulieu, 2020).

Ao executar essa consulta, o resultado mostrará uma **visão integrada** que combina o nome de cada cliente, o produto comprado, a quantidade adquirida, o valor total de cada item e o valor total do pedido. Por exemplo, o cliente Ana, que comprou dois notebooks e um mouse em pedidos diferentes, teria as compras detalhadas com os valores calculados diretamente na consulta.

PERFORMANCE E OTIMIZAÇÃO DE CONSULTAS

Vejamos agora como melhorar o desempenho de consultas SQL abordando técnicas de análise, uso de índices e estratégias práticas de otimização para garantir a eficiência em bancos de dados complexos.

Análise de desempenho de consultas complexas

À medida que os bancos de dados crescem em **tamanho e complexidade**, a eficiência das consultas SQL se torna um fator crítico para a performance do sistema. Consultas mal otimizadas

podem levar a tempos de resposta lentos, especialmente em junções complexas ou com grandes volumes de dados. Portanto, é essencial entender como avaliar o desempenho de uma consulta e identificar oportunidades de melhoria (Ricardo; Urban; Davis, 2022).

Uma das maneiras de analisar a performance de uma consulta é utilizar ferramentas, como o comando **EXPLAIN** ou **EXPLAIN PLAN**, disponíveis em diversos sistemas de gerenciamento de bancos de dados (SGBDs). Esses comandos fornecem uma descrição detalhada de como o banco de dados executará a consulta, incluindo informações sobre índices utilizados, tipos de junções e a ordem de execução das operações. Isso permite identificar gargalos e tomar decisões informadas sobre ajustes (Ricardo; Urban; Davis, 2022).

Suponha que uma consulta envolvendo junções entre três tabelas — **Clientes**, **Pedidos** e **Produtos** — está demorando mais tempo do que o esperado para ser executada. Ao usar o comando **EXPLAIN**,

o SGBD pode revelar que uma das tabelas está sendo lida completamente, pois não tem índices na coluna usada para a junção. Com essa informação, a criação de um índice na coluna adequada pode **reduzir significativamente** o tempo de execução da consulta.



Além de **EXPLAIN**, outra estratégia útil é monitorar métricas, como tempo de execução, uso de memória e número de leituras de disco. Essas informações ajudam a identificar consultas que precisam ser otimizadas, priorizando aquelas com maior impacto no desempenho geral do sistema.

Para melhorar a performance de consultas complexas, algumas práticas incluem (Beaulieu, 2020):

ÍNDICES APROPRIADOS:

garantir que as colunas usadas nas junções e nos filtros possuam índices apropriados

SELEÇÃO OTIMIZADA:

reduzir o número de colunas selecionadas com SELECT utilizando apenas as necessárias.

OPERAÇÕES ESSENCIAIS:

evitar operações desnecessárias, como junções redundantes ou subconsultas complexas que podem ser simplificadas.

TESTES REAIS:

sempre testar consultas em ambientes semelhantes ao de produção para garantir que os ajustes realizados tenham o impacto esperado.

A análise de desempenho é uma habilidade essencial para qualquer profissional que trabalha com bancos de dados.



Índices e o respectivo impacto na performance

Índices são estruturas auxiliares criadas sobre as colunas de uma tabela para acelerar os processos de busca e filtragem de dados. Eles funcionam como atalhos que permitem ao SGBD localizar rapidamente os registros correspondentes a uma consulta, **reduzindo drasticamente** o número de **operações** necessárias para acessar os dados (Ricardo; Urban; Davis, 2022).

Ao usar índices, operações, como buscas em colunas específicas, junções e filtros em consultas SQL, tornam-se muito mais eficientes. Sem índices, o SGBD precisa realizar uma varredura completa na tabela (conhecido em inglês como **full table scan**) para encontrar os registros solicitados, o que pode ser extremamente demorado em tabelas grandes (Ricardo; Urban; Davis, 2022).

VOCÊ SABE RESPONDER?

Quais são os tipos de índices existentes em um banco de dados e como eles ajudam a melhorar a performance das consultas?

Os índices podem ser classificados em diferentes tipos, dependendo da aplicação e da estrutura do banco de dados:

ÍNDICES SIMPLES:

Criados em uma única coluna, são ideais para operações básicas de busca e filtros.

ÍNDICES COMPOSTOS:

Abrangem múltiplas colunas, sendo úteis em consultas que utilizam mais de uma coluna como critério de filtragem.

ÍNDICES EXCLUSIVOS:

Garantem que os valores de uma coluna sejam únicos, como em chaves primárias.

ÍNDICES DE TEXTO COMPLETO:

Otimizados para buscas em colunas contendo grandes volumes de texto, como descrições ou documentos.

Agora, para exemplificar, considere uma tabela **Pedidos** com **milhões** de registros. Suponha que uma consulta está filtrando pedidos com valores acima de **R\$ 1.000,00**:

```
SELECT idPedido, valor  
FROM Pedidos  
WHERE valor > 1000;
```

Sem um índice na coluna valor, o SGBD terá que varrer todos os registros para encontrar aqueles que atendem à condição. Isso resulta em alta utilização de recursos e tempo de resposta lento. Ao criar um índice na coluna valor, o SGBD pode acessar **diretamente** os registros que atendem ao **filtro**:

```
CREATE INDEX idx_valor ON Pedidos(valor);
```

Com o índice **idx_valor**, a consulta se torna muito mais eficiente, reduzindo o tempo de execução de segundos para milissegundos, dependendo do volume de dados.

Embora os índices melhorem a performance de leitura, eles também têm desvantagens. Cada índice ocupa um espaço adicional no disco e aumenta o tempo de execução de operações de escrita, como inserções, atualizações e exclusões, pois o índice precisa ser atualizado junto com os dados da tabela. É importante equilibrar o uso de índices para maximizar os benefícios sem comprometer a performance de gravação (Ricardo; Urban; Davis, 2022).

**EM FOCO**

Acesse o ambiente de aprendizagem e confira a aula referente a este tema.
Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.

NOVOS DESAFIOS

Neste tema, você aprendeu a aplicar subconsultas e junções avançadas, ferramentas essenciais para consolidar dados fragmentados em informações coesas e estratégicas. Exploramos como as subconsultas podem ser utilizadas para criar condições específicas e hierarquias de dados, enquanto as junções avançadas permitem conectar múltiplas tabelas de forma eficiente. Essas técnicas aprimoram a precisão das consultas e otimizam a performance, habilidades indispensáveis em ambientes de trabalho que exigem análise de dados complexos.

Voltando ao desafio inicial, no qual precisávamos identificar os clientes com pedidos acima de R\$ 500,00 e calcular o valor total das compras deles, entendemos como subconsultas e junções avançadas são fundamentais para resolver problemas reais. As subconsultas permitiram filtrar dados específicos, enquanto as junções conectaram as tabelas de clientes, pedidos e produtos, criando um panorama completo para decisões empresariais. Essa abordagem prática demonstrou como a teoria pode ser aplicada diretamente no dia a dia profissional.

No mercado de trabalho, a capacidade de criar consultas robustas é amplamente utilizada em setores, como e-commerce, logística e finanças. Seja desenvolvendo relatórios detalhados para prever tendências de vendas ou otimizando processos em sistemas ERP, essas habilidades são altamente valorizadas. Agora, reflita sobre como você pode aplicar o que aprendeu para otimizar processos reais e impulsionar a sua carreira, desenvolvendo soluções confiáveis e eficientes para os desafios do mundo corporativo.

VAMOS PRATICAR

1. Consultas SQL frequentemente exigem a criação de filtros avançados e flexíveis para lidar com dados dinâmicos ou provenientes de diferentes tabelas. Existem técnicas que permitem integrar resultados intermediários em consultas principais, o que torna possível lidar com cenários mais complexos de maneira eficiente. Esses recursos são amplamente utilizados para criar critérios que dependem de cálculos ou relações específicas.

Considerando o uso de subconsultas na cláusula WHERE, analise as afirmativas a seguir:

- I - Subconsultas aplicadas na cláusula WHERE não podem ser combinadas com outras condições na mesma cláusula.
- II - O uso de subconsultas na cláusula WHERE é limitado a cenários em que não há relacionamentos entre tabelas.
- III - Uma subconsulta na cláusula WHERE pode ser usada para comparar valores calculados dinamicamente com os dados da consulta principal.

É correto o que se afirma em:

- a) I, apenas.
 - b) III, apenas.
 - c) I e II, apenas.
 - d) II e III, apenas.
 - e) I, II e III.
-
2. A eficiência de consultas SQL é um elemento essencial em sistemas de gerenciamento de bancos de dados, especialmente à medida que as bases de dados crescem em volume e complexidade. Ferramentas, como o comando EXPLAIN, fornecem uma visão detalhada sobre como as consultas são realizadas, ajudando a identificar possíveis gargalos de desempenho.

Considerando as estratégias e as ferramentas usadas para análise e otimização de consultas SQL, analise as afirmativas a seguir:

VAMOS PRATICAR

- I - EXPLAIN identifica gargalos e ajuda na otimização de consultas.
- II - EXPLAIN fornece informações sobre a execução de uma consulta, incluindo uso de índices, tipos de junções e ordem das operações, e realiza ajustes automáticos para otimização de consultas.
- III - Uma análise de consultas com ferramentas, como EXPLAIN, permite identificar gargalos de desempenho e ajustar consultas de forma eficiente.
- IV - A criação de índices é uma estratégia que pode melhorar significativamente o desempenho de consultas SQL, mas não é a única solução disponível.

É correto o que se afirma em:

- a) I e III, apenas.
 - b) II e III, apenas.
 - c) III e IV, apenas.
 - d) I, II e III, apenas.
 - e) II, III e IV, apenas.
3. Em bancos de dados, os índices são usados para melhorar o desempenho das consultas SQL, diminuindo o tempo necessário para localizar e filtrar registros. Além do mais, o SGBD pode realizar varreduras completas na tabela, o que pode impactar a eficiência em grandes bases de dados. Existem diferentes tipos de índices, cada um adequado às necessidades específicas, como filtragem de múltiplas colunas, garantia de unicidade ou buscas em grandes volumes de texto.

Considerando os índices e os respectivos impactos no desempenho, analise as afirmativas a seguir:

- I - O uso de índices reduz o tempo de busca em tabelas grandes.
- II - Índices compostos só podem ser criados em tabelas que não possuem índices simples.
- III - Criar índices em todas as colunas de uma tabela melhora o desempenho geral do banco de dados.
- IV - A ausência de índices pode levar o SGBD a realizar varreduras completas nas tabelas, prejudicando o desempenho em grandes tabelas.

É correto o que se afirma em:

- a) I apenas.
- b) II e III, apenas.
- c) I e IV, apenas.
- d) I, II e III, apenas.
- e) II, III e IV, apenas.

REFERÊNCIAS

ALLEN, G. R.; OWEN, W. A. **SQL in 10 Minutes**: Sams Teach Yourself. 5. ed. [S. l.]: Pearson Education, 2021.

BEAULIEU, A. **Learning SQL**: Master SQL Fundamentals. 3. ed. Sebastopol: O'Reilly Media, 2020.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2019.

RICARDO, C. M.; URBAN, S. D.; DAVIS, K. C. **Databases Illuminated**. 4. ed. Burlington: Jones & Bartlett Learning, 2022.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistemas de Banco de Dados**. 7. ed. Rio de Janeiro: Elsevier, 2020.

CONFIRA SUAS RESPOSTAS

1. Opção B.

I. Incorreta. Subconsultas podem ser combinadas com outras condições na cláusula WHERE por meio de operadores, como AND e OR.

II. Incorreta. Subconsultas **são utilizadas para lidar com dados relacionados e cenários que envolvem múltiplas tabelas.**

III. Correta. Subconsultas permitem que valores calculados sejam utilizados como critérios de filtro na consulta principal.

2. Opção A.

I. Correta. Essa ferramenta auxilia na análise e melhoria de desempenho.

II. Incorreta. Apesar de ser útil, o comando EXPLAIN não realiza otimizações automaticamente. Ele apenas auxilia na identificação de gargalos.

III. Correta. Ferramentas, como EXPLAIN, auxiliam na identificação de problemas e na aplicação de ajustes para atualização de consultas.

IV. Incorreta. Outras estratégias, como ajustar a lógica da consulta e monitorar avaliações de uso de recursos, também são recomendadas para a otimização.

3. Opção C.

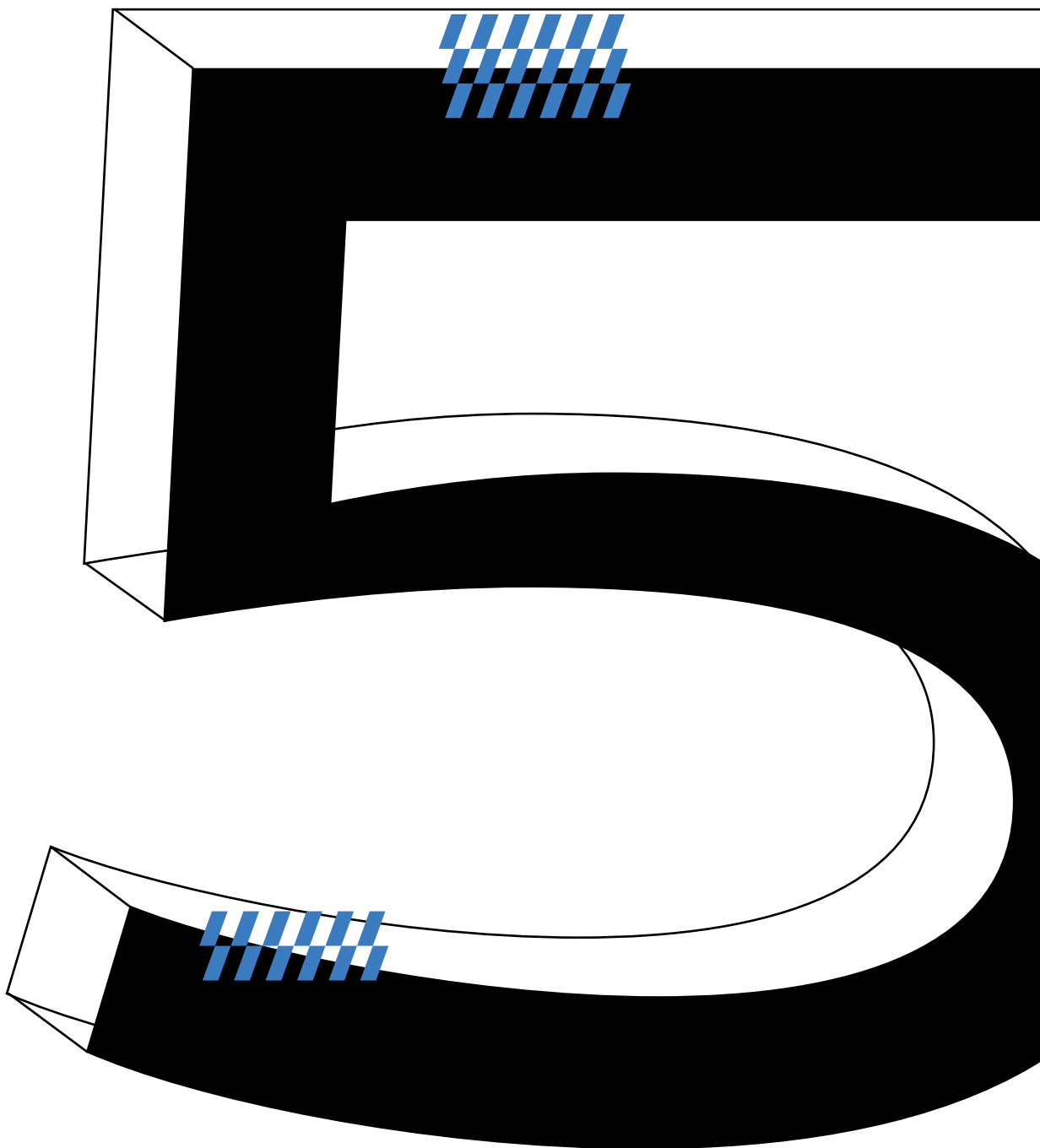
I. Correta. Índices agilizam consultas para evitar varreduras completas na tabela.

II. Incorreta. Esta afirmativa é falsa, porque índices compostos podem coexistir com índices simples. Ambos têm propósitos diferentes: índices compostos ajudam a otimizar consultas envolvendo múltiplas colunas, enquanto índices simples lidam com buscas em colunas individuais.

III. Incorreta. Incorreta. Esta afirmativa é falsa, porque criar índices em todas as colunas pode degradar o desempenho em operações de escrita, como inserções e atualizações, já que o SGBD precisa manter todos os índices sincronizados. Além disso, índices desnecessários consomem mais espaço de armazenamento e podem afetar a performance.

IV. Correta. A ausência de índices força o SGBD a realizar *full table scans*, o que pode ser extremamente ineficiente em tabelas grandes, já que cada registro da tabela precisa ser avaliado para satisfazer a consulta.

 *unidade*





TEMA DE APRENDIZAGEM 8

ADMINISTRAÇÃO DE BANCOS DE DADOS

MINHAS METAS

- Entender a importância de um SGBD na administração de dados.
- Identificar as etapas de instalação e configuração de SGBDs populares.
- Aplicar boas práticas na configuração inicial de um SGBD.
- Reconhecer a importância da segurança em bancos de dados.
- Configurar usuários, permissões e roles em um banco de dados.
- Conectar bancos de dados a sistemas usando drivers e interfaces.
- Implementar o gerenciamento de conexões em sistemas integrados.

INICIE SUA JORNADA

Suponha que você acaba de ser contratado por uma startup de tecnologia que está se expandindo rapidamente. Seu primeiro desafio como administrador de bancos de dados é **configurar e gerenciar** um sistema que suporte o crescimento do negócio. Atualmente, a equipe de desenvolvimento enfrenta problemas de **desempenho e segurança** no banco de dados existente. Além disso, há dificuldades em integrar o banco com novas aplicações e a necessidade de garantir acessos seguros e monitorados. Como você abordaria a instalação, a configuração e a segurança para resolver esses problemas e preparar o sistema para futuras demandas?

A administração de bancos de dados é fundamental para garantir que sistemas de informação funcionem de forma **confiável, segura e escalável**. Um banco de dados mal configurado pode levar a falhas no sistema, vulnerabilidades de segurança e perda de dados importantes. Quando um banco de dados é bem administrado, ele não só suporta o crescimento da empresa, mas também melhora a **eficiência das operações** e protege informações críticas contra acessos não autorizados. Resolver esses desafios contribui diretamente para a confiança no sistema e o sucesso do negócio.

Para experimentar esse contexto, pense em um banco de dados que precise suportar um sistema de **e-commerce**. Imagine configurar o banco desde a **instalação**, definir **permissões** que protejam dados **sensíveis** de clientes e estabelecer conexões **seguras** entre o banco e as aplicações. Além disso, considere a importância de monitorar o desempenho e ajustar configurações para manter a eficiência do sistema em momentos de alta demanda. Quais passos você tomaria para garantir que o sistema fosse seguro, eficiente e capaz de lidar com o crescimento futuro?

Refleta sobre como as práticas de administração de bancos de dados podem ser aplicadas em diferentes áreas, desde startups até grandes corporações. Pense em como a habilidade de gerenciar bancos de dados impacta diretamente a confiabilidade dos sistemas e a proteção das informações críticas. Além disso, considere como o aprendizado obtido pode ser aplicado no **mercado de trabalho**, em que bancos de dados são o **núcleo** de sistemas modernos, exigindo profissionais capazes de resolver problemas e garantir a continuidade das operações.

**PLAY NO CONHECIMENTO**

Quer dominar técnicas essenciais para escalar e gerenciar seus bancos de dados? No podcast de hoje, exploramos sharding, replicação e gerenciamento de conexões, conteúdos indispensáveis para a administração de bancos de dados. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Assista ao vídeo Como funcionam os índices no SQL para relembrar e aprofundar o conceito de índices e a respectiva importância na performance de consultas.

<https://www.youtube.com/watch?v=DLbP5jTJqRA>



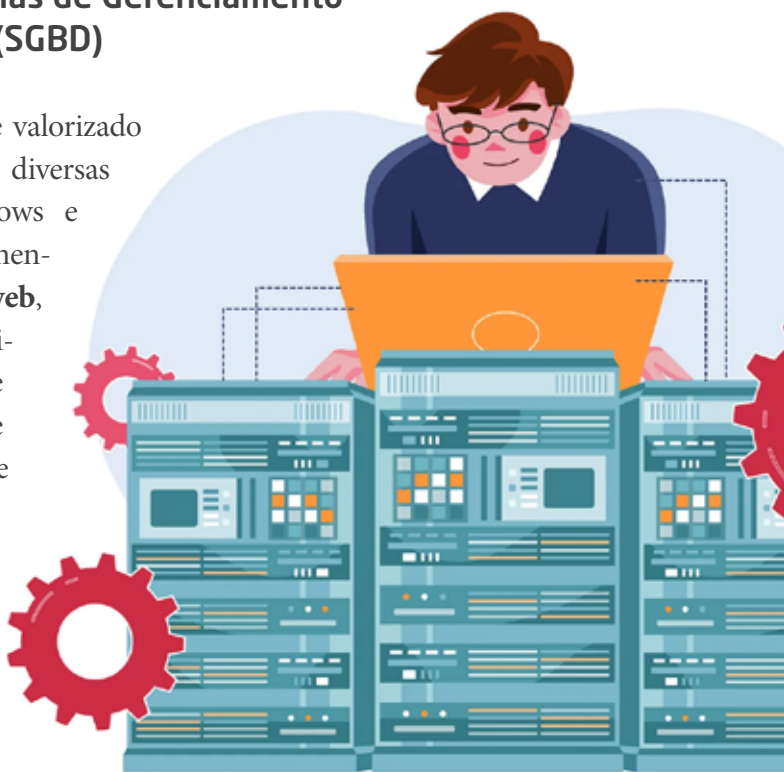
DESENVOLVA SEU POTENCIAL

INSTALAÇÃO E CONFIGURAÇÃO DE SGBD

Sistemas de Gerenciamento de Bancos de Dados (SGBDs) são ferramentas indispensáveis para administrar, organizar e acessar grandes volumes de dados de forma eficiente. Eles fornecem uma **interface** que permite aos usuários e desenvolvedores interagir com os dados, criando, manipulando e consultando informações sem a necessidade de lidar diretamente com detalhes técnicos do armazenamento físico. Entre os SGBDs mais populares, estão **MySQL**, **PostgreSQL**, **SQL Server** e **Oracle**, cada um com características específicas que atendem a diferentes necessidades de projetos (Romanowski, 2020).

Visão geral de Sistemas de Gerenciamento de Bancos de Dados (SGBD)

O MySQL é especialmente valorizado pela compatibilidade com diversas plataformas, como Windows e Linux, além de ser amplamente utilizado em **sistemas web**, como e-commerce e aplicativos baseados em nuvem. Ele oferece uma combinação de desempenho, escalabilidade e facilidade de uso, tornando-se uma escolha popular tanto para **iniciantes** quanto para **profissionais experientes** (Leite Júnior, 2021).



Além de entender a funcionalidade básica de um SGBD, é importante reconhecer o papel dele no ambiente corporativo. Um SGBD bem configurado garante o armazenamento seguro dos dados, otimiza o desempenho do sistema e simplifica tarefas, como backup, recuperação e gerenciamento de usuários. Veremos como instalar e configurar o MySQL no Windows, preparando o ambiente para ser utilizado em projetos reais e compreendendo as práticas recomendadas para iniciar com eficiência.

Instalação de SGBDs Populares (MySQL no Windows)

Instalar um SGBD, como o MySQL, no Windows é um passo essencial para configurar o ambiente de desenvolvimento e aprendizado (LuizTools, 2020). Assim, são exibidas as etapas necessárias para instalar o **MySQL Community Server**, uma versão gratuita e amplamente usada desse sistema.



EU INDICO

Acesse o site oficial do MySQL em <https://dev.mysql.com> e faça o download do MySQL Community Server, a versão gratuita e amplamente utilizada. É importante acessar diretamente o site para garantir que você esteja baixando a versão mais recente e segura. <https://dev.mysql.com>

PASSO 1. BAIXANDO O MYSQL:

Acesse o site oficial do MySQL e clique na opção "Downloads". Selecione "MySQL Community Server" e escolha a versão compatível com o Windows. Durante o download, você será solicitado a criar uma conta, mas pode pular essa etapa clicando em "No thanks, just start my download."

PASSO 2. INICIANDO A INSTALAÇÃO:

Após o download, abra o instalador e escolha o tipo de instalação. Para fins de aprendizado e desenvolvimento, recomendo a opção "Developer Default", que inclui o servidor MySQL, ferramentas de gerenciamento e bibliotecas de conexão. O instalador verificará os pré-requisitos e, caso necessário, orientará a instalação de componentes adicionais, como o Visual C++.

PASSO 3. CONFIGURANDO O SERVIDOR MYSQL:

Durante a configuração, você precisará definir algumas opções importantes:

Configuração do tipo de servidor: escolha "Standalone MySQL Server" para um ambiente local.

- Configurações de rede: mantenha a porta padrão (3306) para facilitar a conexão. Certifique-se de que o firewall está configurado para permitir conexões nessa porta.
- Autenticação e usuário root: escolha o método de autenticação e defina uma senha segura para o usuário root. Adicione outros usuários, se necessário.

PASSO 4. CONCLUINDO A INSTALAÇÃO:

Finalize a instalação e inicie o servidor MySQL. O instalador verificará se tudo está configurado corretamente. Você também terá a opção de abrir o MySQL Workbench, uma ferramenta gráfica que facilita a interação com o banco de dados.

PASSO 5. CONECTANDO-SE AO BANCO DE DADOS:

Após a instalação, abra o MySQL Workbench e conecte-se ao servidor usando as credenciais definidas. Isso permitirá criar, modificar e consultar bancos de dados de maneira intuitiva, sem precisar usar o terminal diretamente.

A instalação do MySQL no Windows é um processo direto, mas requer atenção aos detalhes para garantir que o servidor esteja configurado corretamente. A seguir, vamos explorar as práticas recomendadas para a configuração inicial do MySQL.

Configuração inicial de um SGBD

Após instalar o MySQL no Windows, é essencial realizar algumas configurações iniciais para garantir o bom funcionamento e a segurança do sistema. Esses ajustes ajudam a preparar o ambiente para desenvolvimento ou produção, otimizando o desempenho e protegendo os dados armazenados (LEITE JÚNIOR, 2021).

Durante a instalação, você definiu uma senha para o usuário **root**, que possui privilégios administrativos no servidor MySQL. Certifique-se de que a senha seja forte e evite utilizá-la diretamente em aplicações. Ao contrário disso, crie usuários específicos para cada aplicação ou função e atribua **permissões restritas** com o comando **GRANT**. Por exemplo, para criar um usuário com privilégios limitados, use:

```
CREATE USER 'usuarioApp'@'localhost' IDENTIFIED  
BY 'senhaSegura';  
  
GRANT SELECT, INSERT, UPDATE ON banco.* TO  
'usuarioApp'@'localhost';
```

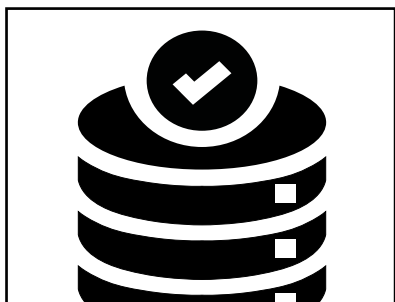
Outro passo importante é executar o script de segurança padrão do MySQL utilizando o comando **mysql_secure_installation** no terminal. Essa ferramenta permite configurar medidas básicas de segurança, como:

Remover usuários anônimos:

Elimina contas sem nome de usuário para evitar acessos não autorizados.

Desabilitar logins remotos para o root:

Restringe o acesso administrativo a conexões locais.

Excluir bancos de teste:

Remove bancos de dados de demonstração para evitar vulnerabilidades.

Reforçar políticas de senha:

Exige senhas fortes para proteger contas contra invasões.

Essas medidas aumentam significativamente a proteção contra acessos não autorizados (Elmasri; Navathe, 2019).

Além dessas medidas de segurança, o arquivo **my.ini**, principal de configuração do MySQL no Windows, permite ajustes que impactam o desempenho do servidor. Configure o **innodb_buffer_pool_size** para otimizar o uso de memória, ajuste o **max_connections** para controlar o número de conexões simultâneas e ative **logs** de erro e de consultas lentas para monitorar e identificar possíveis gargalos de desempenho.

Não menos importante, implemente uma rotina de backup desde o início (Machado, 2020). Ferramentas, como o **mysqldump**, permitem criar backups regulares de bancos de dados.

Um exemplo de comando para gerar um backup seria:

```
mysqldump -u root -p banco > backup.sql
```

Por fim, antes de utilizar o MySQL em um ambiente real, realize testes para validar a configuração. Teste conexões com usuários criados, simule cargas de trabalho no sistema e avalie a performance de consultas frequentes. Essas práticas ajudarão a preparar o servidor MySQL para atender às demandas de sistemas de pequeno ou grande porte, garantindo segurança, eficiência e escalabilidade (Leite Júnior, 2021).

SEGURANÇA E CONTROLE DE ACESSO

A segurança de um banco de dados é uma das áreas mais críticas da administração de SGBDs. Abordaremos as melhores práticas e estratégias para proteger os dados armazenados e garantir que apenas usuários autorizados tenham acesso às informações. Vamos explorar a importância da segurança em bancos de dados, o gerenciamento de controle de acesso e as boas práticas para reduzir riscos de violações de dados.

Importância da segurança em banco de dados

Os bancos de dados são repositórios de informações valiosas e sensíveis, como dados financeiros, pessoais e comerciais (Silberschatz; Korth; Sudarshan, 2020). Garantir a segurança dessas informações não é apenas uma questão técnica, mas também um requisito legal em muitas jurisdições. Uma violação de segurança pode levar à perda de confiança dos clientes, danos financeiros significativos e problemas legais para a organização.



EU INDICO

Para compreender as melhores práticas de segurança de banco de dados, recomendo este recurso da IBM que aborda as complexidades da área, incluindo práticas, políticas e tecnologias essenciais para proteger a confidencialidade, a integridade e a disponibilidade dos dados. Veja aqui: <https://www.ibm.com/br-pt/topics/database-security>

As ameaças à segurança podem incluir desde **ataques externos**, como injeção de SQL e acesso não autorizado, até **riscos internos**, como uso indevido por funcionários. Medidas de segurança robustas ajudam a proteger os dados contra essas ameaças e garantem a integridade e a confidencialidade das informações.

Controle de acesso: usuários, permissões e roles

Controlar quem tem acesso ao banco de dados e o que pode ser feito dentro dele é fundamental para proteger informações e garantir que apenas ações autorizadas sejam realizadas. O controle de acesso em um SGBD, como o MySQL, é baseado em **usuários**, **permissões** e **roles**, que permitem definir regras claras e específicas para cada tipo de operação (Ricardo; Urban; Davis, 2022).

No MySQL, cada usuário é identificado por um nome e um host de conexão. A criação de um usuário requer o comando **CREATE USER** e as respectivas permissões podem ser ajustadas com o comando **GRANT** (LuizTools, 2020). Por exemplo, se você deseja criar um usuário para uma aplicação específica com permissões limitadas, o comando seria:

```
CREATE USER 'usuarioApp'@'localhost' IDENTIFIED BY 'senhaSegura';  
GRANT SELECT, INSERT ON banco.* TO 'usuarioApp'@'localhost';
```

Esse exemplo cria um usuário chamado **usuarioApp**, que pode se conectar apenas do host local e realizar somente operações de leitura e inserção no banco. Restringir permissões é uma forma eficaz de minimizar danos caso um usuário ou credencial seja comprometido.

As roles (funções) permitem agrupar permissões e aplicá-las a vários usuários de maneira mais eficiente. Ao contrário de configurar permissões individualmente para cada usuário, você pode criar uma role e associá-la aos usuários correspondentes (Leite Júnior, 2021). No MySQL, o comando **CREATE ROLE** cria uma role que pode ser atribuída com **GRANT**. Por exemplo:

```
CREATE ROLE 'gerenteDados';  
GRANT SELECT, INSERT, UPDATE ON banco.* TO 'gerenteDados';  
GRANT 'gerenteDados' TO 'usuarioApp'@'localhost';
```

Nesse caso, a role **gerenteDados** concede permissões de leitura, inserção e atualização em todas as tabelas do banco, e é atribuída ao usuário **usuarioApp**. Essa abordagem simplifica a administração de permissões e reduz erros ao gerenciar acessos.

Caso um usuário precise ter o acesso revogado ou as permissões ajustadas, o comando **REVOKE** é utilizado. Além disso, a auditoria de *logs* de acesso pode ajudar a monitorar atividades suspeitas, como tentativas de conexão falhas ou consultas incomuns, permitindo respostas rápidas a possíveis ameaças.

Boas práticas de segurança

A proteção de dados armazenados em bancos de dados vai além do gerenciamento de usuários e permissões. É crucial adotar uma série de **boas práticas** que fortalecem a segurança e ajudam a mitigar riscos tanto internos quanto externos (Silberschatz; Korth; Sudarshan, 2020; Elmasri; Navathe, 2019). Conheça, a seguir, alguns pontos fundamentais para garantir um ambiente seguro:

USO DE CONEXÕES SEGURAS:

Sempre que possível, utilize conexões criptografadas (SSL/TLS) para acessar o banco de dados, especialmente em ambientes distribuídos ou na nuvem. Isso evita que dados sensíveis, como credenciais de login, sejam interceptados durante a transmissão.

LIMITAÇÃO DE PRIVILÉGIOS:

Aplique o princípio do menor privilégio. Conceda aos usuários apenas as permissões estritamente necessárias para realizar as próprias funções. Evite permitir acessos administrativos a usuários que não necessitam deles.

TROCA REGULAR DE SENHAS:

Implemente políticas de senhas que exijam complexidade mínima e renovação periódica. Considere também o uso de autenticação multifator (MFA) para proteger contas sensíveis, como o root.

MONITORAMENTO E AUDITORIA DE LOGS:

Ative o registro de logs de acessos e operações importantes. Esses logs ajudam a identificar atividades suspeitas e a realizar auditorias para detectar falhas ou incidentes de segurança. Ferramentas adicionais podem ser integradas para a análise automatizada de logs.

ATUALIZAÇÕES REGULARES:

Mantenha o SGBD e as respectivas dependências atualizados com os patches mais recentes. Atualizações corrigem vulnerabilidades conhecidas que podem ser exploradas por atacantes.

ISOLAMENTO DE SERVIDORES:

Evite que o banco de dados esteja acessível diretamente da internet. Configure firewalls para permitir conexões apenas de servidores ou redes autorizadas. Em ambientes de produção, utilize sub-redes privadas.

BACKUPS E TESTES DE RECUPERAÇÃO:

Realize backups regulares dos dados e teste periodicamente os processos de recuperação. Certifique-se de armazenar os backups em locais seguros e de usar criptografia para proteger dados sensíveis.

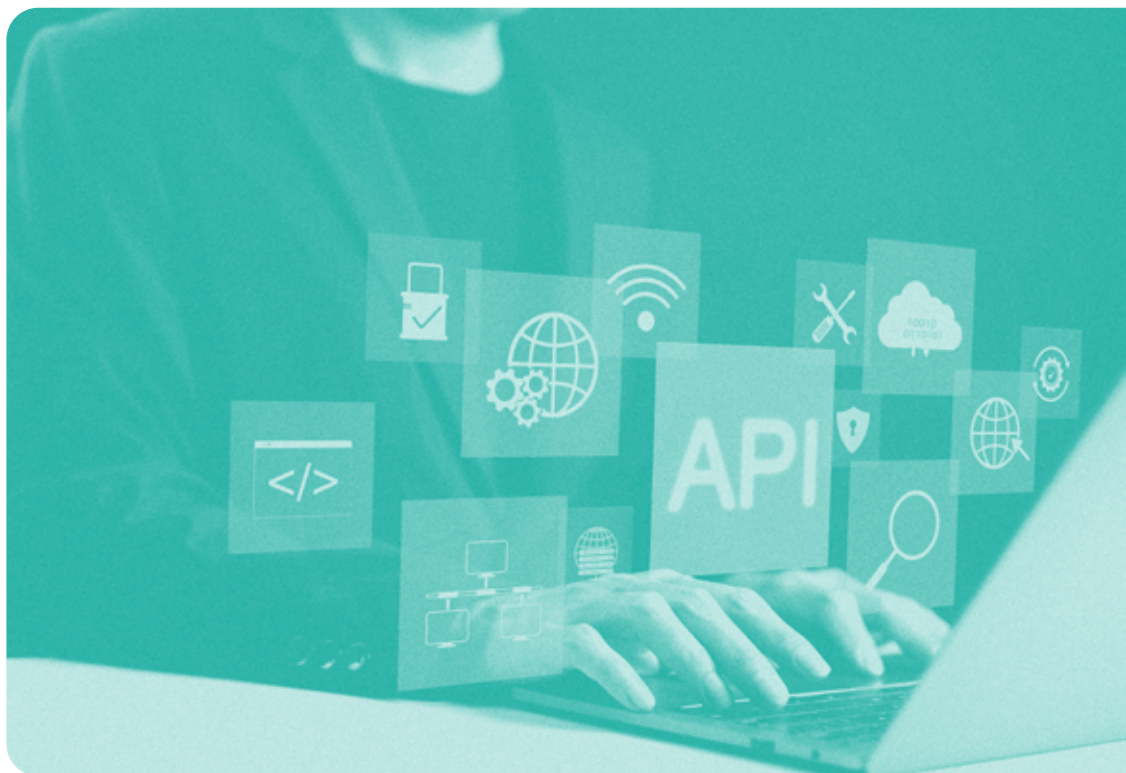
Essas práticas, quando aplicadas de forma consistente, criam camadas de defesa que dificultam acessos não autorizados e garantem maior integridade e confidencialidade dos dados.

CONECTANDO O BANCO DE DADOS A SISTEMAS

A conexão entre um banco de dados e sistemas de software é uma etapa essencial no desenvolvimento de aplicações modernas. Seja para fornecer dados a um site de e-commerce, uma plataforma de gerenciamento empresarial ou um aplicativo móvel, a integração eficaz entre o banco de dados e a aplicação é fundamental para o funcionamento do sistema.

Uso de drivers e interfaces de conexão

A **integração** de um banco de dados com uma aplicação é feita por meio de interfaces de programação (APIs) ou drivers que permitem a comunicação entre o sistema e o banco de dados. Essa conexão permite que a aplicação envie **comandos SQL** ao banco e receba os resultados, seja para exibir dados aos usuários, armazenar informações ou realizar cálculos em tempo real.



Drivers e interfaces de conexão são ferramentas que abstraem a complexidade do protocolo de comunicação e fornecem uma forma padronizada de enviar comandos SQL e receber resultados (Elmasri; Navathe, 2019). Cada sistema de gerenciamento de banco de dados (SGBD) oferece drivers específicos ou compatíveis para diferentes linguagens de programação. Por exemplo:

Python: mysql-connector e PyMySQL.

Java: JDBC (Java Database Connectivity).

Node.js: mysql2 e sequelize.

PHP: PDO (PHP Data Objects).

Esses drivers oferecem funções nativas para estabelecer conexões, executar consultas e gerenciar transações. Ao configurar um driver, você precisa especificar parâmetros, como o endereço do servidor (host), o nome do banco de dados, o nome de usuário e a senha (Elmasri; Navathe, 2019).

Além dos drivers, *frameworks* e bibliotecas modernas incluem interfaces de conexão mais sofisticadas que simplificam ainda mais o gerenciamento de bancos de dados. Esses *frameworks* podem integrar ORM (Object-Relational Mapping), permitindo que você manipule dados como objetos em vez de escrever consultas SQL manualmente. Exemplos incluem **Django ORM** para Python, **Hibernate** para Java e **Sequelize** para Node.js.

Exemplo prático de conexão entre banco de dados e aplicações

Para ilustrar um exemplo prático de como integrar o código de aplicação com um banco de dados MySQL, usaremos **Node.js**, uma plataforma amplamente adotada para desenvolvimento backend (LuizTools, 2020).

**APROFUNDANDO**

O Node.js é uma plataforma de código aberto baseada no motor V8 do Google Chrome projetada para construir aplicações rápidas e escaláveis. Ele é amplamente utilizado para o desenvolvimento back-end e se destaca por permitir a execução de JavaScript fora do navegador, proporcionando alta performance e suporte a operações assíncronas.

Para o exemplo prático que vamos explorar, é essencial ter o Node.js instalado em seu ambiente de desenvolvimento. O MySQL também será necessário, e o processo de instalação foi detalhado anteriormente. Embora utilizemos Node.js nesse exemplo, o mesmo conceito pode ser aplicado com outras linguagens, como Python, Java ou PHP.

**EU INDICO**

Caso ainda não tenha o Node.js, acesse o site oficial do Node.js para baixá-lo e instalá-lo. Certifique-se de escolher a versão recomendada para a maioria dos usuários, garantindo maior estabilidade durante o desenvolvimento.
<https://nodejs.org>

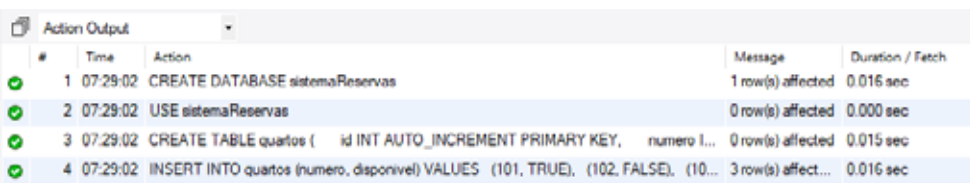
Primeiramente, certifique-se de que o banco de dados está configurado com **permissões adequadas**. Em seguida, crie um banco de dados chamado **sistemaReservas** e uma tabela com os seguintes comandos SQL:

```
CREATE DATABASE sistemaReservas;
USE sistemaReservas;

CREATE TABLE quartos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    numero INT NOT NULL,
    disponivel BOOLEAN NOT NULL
);

INSERT INTO quartos (numero, disponivel) VALUES
(101, TRUE),
(102, FALSE),
(103, TRUE);
```

Esse script cria uma tabela **quartos** com três entradas: dois quartos disponíveis e um indisponível. Certifique-se de executar esse comando no seu banco de dados MySQL antes de prosseguir, conforme Figura 1.



#	Time	Action	Message	Duration / Fetch
1	07:29:02	CREATE DATABASE sistemaReservas	1 row(s) affected	0.016 sec
2	07:29:02	USE sistemaReservas	0 row(s) affected	0.000 sec
3	07:29:02	CREATE TABLE quartos (id INT AUTO_INCREMENT PRIMARY KEY, numero INT NOT NULL, disponivel BOOLEAN NOT NULL)	0 row(s) affected	0.015 sec
4	07:29:02	INSERT INTO quartos (numero, disponivel) VALUES (101, TRUE), (102, FALSE), (103, TRUE)	3 row(s) affected	0.016 sec

Figura 1 – Resultado da execução de script / Fonte: o autor.

Descrição da Imagem: são exibidos quatro comandos SQL executados com sucesso. Eles são indicados por ícones verdes com marcações de verificação à esquerda. Os comandos incluem a criação do banco de dados sistemaReservas, a seleção do banco com USE, a criação da tabela quartos e a inserção de três registros na tabela. Fim da descrição.

Agora, vamos implementar a conexão com o banco de dados no **Node.js**. Para isso, precisaremos configurar um projeto Node.js e instalar o driver **mysql2**, que facilita a comunicação com o MySQL.

Primeiro, crie uma pasta para o projeto em seu computador e navegue até ela no terminal, assim como é apresentado na Figura 2. Em seguida, inicie o projeto com o comando: **npm init -y**. Esse comando cria um arquivo **package.json**, que é necessário para gerenciar as dependências do projeto. Ele permite que o Node.js reconheça o projeto como uma aplicação.

```
PS C:\Users\vagnerereno> mkdir projeto-integracao

Diretório: C:\Users\vagnerereno

Mode                LastWriteTime         Length Name
----                -
d-----          08/01/2025   07:34                projeto-integracao

PS C:\Users\vagnerereno> cd .\projeto-integracao\
PS C:\Users\vagnerereno\projeto-integracao> npm init -y
Wrote to C:\Users\vagnerereno\projeto-integracao\package.json:

{
  "name": "projeto-integracao",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}
```

Figura 2 - Criação de diretório e projeto Node.js Fonte: o autor.

Descrição da Imagem: é exibido um terminal no Windows PowerShell com texto em branco sobre fundo preto. Há comandos executados na sequência: `mkdir projeto-integracao` em amarelo, criando um diretório; `cd .\projeto-integracao\` acessando o diretório; e `npm init -y` inicializando um projeto Node.js. Fim da descrição.

Depois, instale o driver **mysql2** com o seguinte comando: **npm install mysql2**, conforme Figura 3.

```
PS C:\Users\vagnerereno\projeto-integracao> npm install mysql2
added 13 packages, and audited 14 packages in 3s

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Figura 3 - Instalação do driver mysql2 / Fonte: o autor.

Descrição da Imagem: é exibido um terminal no Windows PowerShell com texto em branco sobre fundo preto. O comando executado é `npm install mysql2`, em amarelo. A saída informa que 13 pacotes foram adicionados. Fim da descrição



O próximo passo é conectar a aplicação ao banco de dados. O código da aplicação pode ser escrito em qualquer editor de texto, como o Bloco de Notas do Windows, o Notepad++ ou IDEs mais avançadas, como o VS Code. Salve o arquivo com a extensão **.js** dentro da pasta do projeto, que já contém o arquivo **package.json** (LuizTools, 2020). Por exemplo, salve o arquivo como **conexao.js**. Dentro do arquivo, insira o seguinte código:

```
const mysql = require('mysql2');  
// Configurando a conexão  
const conexao = mysql.createConnection({  
  host: 'localhost',  
  user: 'usuarioApp',  
  password: 'senhaSegura',  
  database: 'sistemaReservas'  
});  
  
// Conectando ao banco  
conexao.connect((err) => {  
  if (err) {  
    console.error('Erro ao conectar ao banco:', err);  
    return;  
  }  
  console.log('Conexão estabelecida com sucesso!');  
});  
  
// Executando uma consulta  
conexao.query('SELECT * FROM quartos WHERE disponivel = TRUE', (err, resultados)  
=> {  
  if (err) {  
    console.error('Erro ao executar a consulta:', err);  
    return;  
  }  
  console.log('Quartos disponiveis:', resultados);  
});  
  
// Fechando a conexão  
conexao.end();
```

Esse código realiza as seguintes etapas (MySQL2, 2025):

IMPORTAÇÃO DA BIBLIOTECA MYSQL2:

a primeira linha do código importa a biblioteca `mysql2`. Essa biblioteca é necessária para usar funções que permitem a comunicação com o MySQL.

CONFIGURAÇÃO DA CONEXÃO:

a função `mysql.createConnection` configura os parâmetros de conexão com o banco de dados. Esses parâmetros incluem:

- O endereço do servidor (host), que, nesse caso, é `localhost` para uso local.
- O nome do usuário (user) que acessará o banco.
- A senha (password) do usuário.
- O banco de dados específico (database) que será acessado.

ESTABELECIMENTO DA CONEXÃO:

o método `connect` verifica se a conexão com o banco de dados foi estabelecida corretamente. Caso ocorra algum erro, ele será exibido no console e a execução do programa será interrompida.

EXECUÇÃO DE CONSULTAS SQL:

a função `query` executa comandos SQL enviados ao banco de dados. No exemplo, a consulta seleciona todos os quartos disponíveis na tabela `quartos`. Os resultados retornados pelo banco são processados na função de callback e exibidos no console.

ENCERRAMENTO DA CONEXÃO:

a função `end` fecha a conexão com o banco de dados após a execução das operações. Isso é fundamental para liberar recursos e evitar sobrecarga no servidor.

Agora, para executar esse código de exemplo, salve o arquivo, abra o terminal, navegue até o diretório onde o arquivo foi salvo e digite: **node conexao.js**.

```
PS C:\Users\vagnerereno\projeto-integracao> node .\conexao.js
Conexão estabelecida com sucesso!
Quartos disponíveis: [
  { id: 1, numero: 101, disponivel: 1 },
  { id: 3, numero: 103, disponivel: 1 }
]
```

Figura 4 - Conexão Node.js com MySQL estabelecida com sucesso/ Fonte: o autor.

Descrição da Imagem: é exibido um terminal no PowerShell com o comando `node .\conexao.js` em amarelo. Uma saída em branco inclui "Conexão estabelecida com sucesso!" e "Quartos disponíveis:". Dentro de colchetes, dois objetos JSON estão listados, cada um com três pares chave-valor: id, numero e disponivel. Os números associados às chaves id, numero e disponivel estão em amarelo. Fim da descrição.

Se tudo estiver configurado corretamente, você visualizará uma mensagem no terminal confirmando a conexão bem-sucedida e exibindo os dados dos quartos disponíveis, conforme ilustrado na Figura 4.

Com esse exemplo prático, podemos compreender como integrar uma aplicação Node.js com um banco de dados MySQL utilizando a biblioteca `mysql2`. Foram abordados passos importantes, como a configuração da conexão, a execução de consultas SQL e o encerramento da conexão para garantir eficiência e evitar sobrecargas. Esse processo evidencia como tecnologias modernas podem ser combinadas para criar sistemas dinâmicos e escaláveis.

EM FOCO

Confira a aula referente a este tema. Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.

NOVOS DESAFIOS

Ao longo deste tema, você explorou os principais **conceitos e práticas** da administração de bancos de dados, desde a instalação e a configuração de SGBDs até a implementação de medidas de segurança e a conexão eficiente com sistemas. Essas habilidades são fundamentais para garantir o desempenho, a confiabilidade e a proteção das informações em qualquer organização. Agora, é hora de consolidar esse conhecimento, revisitando o problema inicial e conectando-o às demandas reais do mercado de trabalho.

Voltando ao desafio apresentado no início, imagine que você já configurou o sistema para a startup de tecnologia. Com as etapas aprendidas, foi possível instalar um SGBD adequado, configurar permissões de acesso para proteger os dados e integrar o banco ao sistema principal da empresa. Isso permitiu melhorar a segurança e o desempenho do banco de dados, além de garantir a continuidade das operações mesmo em momentos de alta demanda. As ferramentas e as técnicas discutidas durante este tema demonstram como a teoria e a prática se conectam para solucionar problemas reais, entregando resultados concretos e otimizando o ambiente de trabalho.

No mercado profissional, a administração de bancos de dados é uma habilidade amplamente requisitada em diferentes setores, tais como tecnologia, saúde, finanças e e-commerce. Dominar a instalação e a configuração de SGBDs, implementar boas práticas de segurança e gerenciar conexões com eficiência são **diferenciais** para profissionais que desejam se destacar. Imagine-se em cenários onde o desempenho de um sistema depende diretamente das suas decisões como administrador de banco de dados. Estar preparado para enfrentar esses desafios é crucial para construir uma carreira sólida e contribuir para o sucesso das organizações.

VAMOS PRATICAR

1. Sistemas de Gerenciamento de Bancos de Dados (SGBDs), como o MySQL, desempenham um papel fundamental no armazenamento e no gerenciamento eficiente de dados. Esses sistemas são amplamente adotados em diversas aplicações devido às peculiaridades e compatibilidade com diferentes plataformas.

Com base nas características do MySQL, assinale a alternativa que exhibe corretamente uma característica ou aplicação do MySQL:

- a) O MySQL não suporta sistemas baseados em nuvem devido às limitações de escalabilidade.
- b) O MySQL é adequado apenas para aplicações locais e não é utilizado em sistemas web ou na nuvem.
- c) A compatibilidade com múltiplas plataformas, como Windows e Linux, contribui para a ampla adoção do MySQL.
- d) O MySQL é amplamente utilizado apenas por profissionais experientes devido à complexidade de configuração.
- e) Uma grande comunidade de suporte é um dos fatores que contribuem para a baixa popularidade do MySQL no mercado.

VAMOS PRATICAR

2. Durante a configuração do MySQL no Windows, as configurações de rede desempenham um papel importante para garantir o acesso adequado ao servidor. Essas configurações envolvem o uso de uma porta de comunicação e configurações no firewall. Essas práticas são fáceis para evitar problemas de conectividade e garantir o funcionamento eficiente do sistema.

Considerando as configurações de rede para o MySQL no Windows, analise as afirmativas a seguir:

- I - O método de autenticação elimina a necessidade de configurar o firewall para permitir conexões na porta especificada.
- II - Em um ambiente local, as configurações de rede, como o ajuste do firewall, não impactam o acesso ao servidor.
- III - Configurar o firewall para permitir conexões na porta padrão é essencial para evitar problemas de conectividade.
- IV - A porta padrão é usada para conexões ao MySQL, mas ajustes podem ser feitos se houver necessidade específica.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

VAMOS PRATICAR

3. A segurança dos bancos de dados é fundamental para proteger informações confidenciais contra ameaças externas e internas. Além de ser uma necessidade técnica, a proteção dos dados também atende aos requisitos legais em diversas jurisdições. Violações de segurança podem gerar consequências graves, como danos financeiros e a perda de confiança dos clientes.

Considerando as informações sobre a segurança em bancos de dados, analise as afirmativas a seguir:

- I - A injeção de SQL é um exemplo de ameaça externa que pode comprometer a segurança de um banco de dados.
- II - Garantir a segurança de um banco de dados é exclusivamente uma questão técnica, sem relação com requisitos legais.
- III - A proteção de bancos de dados contra ameaças, como injeção de SQL e uso indevido por funcionários, requer monitoramento contínuo, atualizações regulares e auditorias de segurança.
- IV - Garantir a segurança de bancos de dados envolve tanto medidas técnicas e uso de autenticação e criptografia quanto práticas organizacionais, como políticas de acesso e treinamento de funcionários.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, III e IV.

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2019.

LEITE JÚNIOR, J. C. **Aprendendo Banco de Dados MYSQL**: do básico ao avançado. [S. l.]: Amazon Press, 2021.

LUIZTOOLS. **Programação Web com Node.js**: Edição MySQL. 3. ed. [S. l.]: LuizTools, 2020.

MACHADO, F. N. R. **Banco de Dados**: projetos e implementação. 4. ed. São Paulo: Érica, 2020.

MYSQL2. **Sidorares**, c2025. Disponível em: <https://sidorares.github.io/node-mysql2/pt-BR/docs>. Acesso em: 17 fev. 2025.

RICARDO, C. M.; URBAN, S. D.; DAVIS, K. C. **Databases Illuminated**. 4. ed. Burlington: Jones & Bartlett Learning, 2022.

ROMANOWSKI, J. The Most Popular Databases in 2020. **Learn SQL**, 18 ago. 2020. Disponível em: <https://learnsql.com/blog/most-popular-sql-databases-2020/>. Acesso em: 17 fev. 2025.

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistemas de Banco de Dados**. 7. ed. Rio de Janeiro: Elsevier, 2020.

CONFIRA SUAS RESPOSTAS

1. Opção C.

A compatibilidade com diferentes plataformas é uma das razões que tornam o MySQL amplamente utilizado.

A. Incorreta. O MySQL é amplamente utilizado em sistemas baseados em nuvem e é conhecido por sua escalabilidade.

B. Incorreta. O MySQL é amplamente utilizado em sistemas web, como e-commerce, e em aplicativos baseados em nuvem.

D. O MySQL é conhecido pela simplicidade, sendo uma escolha popular tanto para iniciantes quanto para profissionais experientes.

E. A grande comunidade de suporte é um dos principais fatores que impulsionaram a popularidade do MySQL no mercado.

2. Opção C.

I. Incorreta. O método de autenticação não elimina a necessidade de configurar o firewall para permitir conexões na porta especificada.

II. Incorreta. Mesmo em um ambiente local, as configurações de rede, como o ajuste do firewall, podem impactar o acesso ao servidor.

III. Correta. Configurar o firewall para permitir conexões na porta padrão é essencial para evitar problemas de conectividade.

IV. Correto. A porta padrão é usada para conexões ao MySQL, mas ajustes podem ser feitos se houver necessidade específica.

3. Opção E.

I. Correta. A injeção de SQL é uma ameaça externa que pode permitir o acesso não autorizado a informações confidenciais.

II. Incorreta. A segurança dos bancos de dados também envolve atender aos requisitos legais, além de ser uma prática técnica essencial.

III. Correta. Medidas contínuas, como monitoramento, auditorias e atualizações, são essenciais para identificar e prevenir ameaças à segurança de bancos de dados.

IV. Correta. A segurança de bancos de dados exige uma combinação de soluções técnicas e práticas organizacionais para mitigar riscos internos e externos.



TEMA DE APRENDIZAGEM 9

ESTUDOS DE CASO E EXEMPLOS PRÁTICOS

MINHAS METAS

- Compreender a análise de requisitos e modelagem conceitual.
- Desenvolver e implementar o Modelo Entidade-Relacionamento (DER).
- Criar scripts DDL para tabelas, constraints e índices.
- Popular o banco com dados reais usando comandos INSERT.
- Implementar visões e procedimentos armazenados no banco.
- Elaborar consultas complexas para relatórios acadêmicos.
- Analisar e otimizar a performance de consultas SQL.

INICIE SUA JORNADA

Imagine que você está desenvolvendo um **sistema de gestão acadêmica** para uma universidade. Esse sistema precisa gerenciar informações de alunos, professores, disciplinas e turmas, além de gerar relatórios detalhados, como listas de alunos matriculados e desempenho acadêmico por curso. No entanto, criar um banco de dados eficiente e implementar consultas avançadas que atendam a essas necessidades não são tarefas triviais. Como estruturar um banco de dados que seja **funcional, escalável** e atenda aos **requisitos** da instituição?

Dominar a criação e a implementação de sistemas baseados em bancos de dados é uma habilidade indispensável no mercado de trabalho. As empresas procuram profissionais capazes de transformar **necessidades reais** em **soluções práticas** e otimizadas. Saber modelar um sistema, implementar bancos de dados e criar consultas eficientes para gerar relatórios impacta diretamente a produtividade e a tomada de decisões organizacionais.

Ao longo deste tema, você terá a oportunidade de colocar em prática todo o conhecimento adquirido até agora. Vamos começar pela análise de requisitos e modelagem do sistema e passar pela criação do banco de dados com todos os detalhes, como tabelas e *constraints*. Além disso, você aprenderá a **popular** o banco com dados. Por fim, exploraremos a geração de **consultas complexas** para relatórios acadêmicos e a otimização do desempenho do banco.

Pense no impacto que um **sistema de gestão acadêmica** bem estruturado pode ter para uma **universidade**. Ele não apenas facilita o trabalho de diferentes departamentos, como também melhora a experiência de alunos e professores, ao proporcionar acesso rápido e preciso às informações. Esses conceitos também são aplicáveis em outros setores, ampliando suas possibilidades no mercado de trabalho. Ao finalizar este tema, reflita sobre como as habilidades desenvolvidas podem ajudar a resolver problemas reais e a criar soluções tecnológicas que transformam dados em decisões **estratégicas**.

**PLAY NO CONHECIMENTO**

Já imaginou como as regras de negócio podem transformar o seu banco de dados? No episódio de hoje, vamos explorar como essas regras são essenciais para garantir integridade, automação e até mesmo segurança no design de sistemas de bancos de dados. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

VAMOS RECORDAR?

Que tal reforçar os seus conhecimentos em SQL? Relembre os conceitos básicos e se aprofunde na linguagem assistindo ao vídeo SQL: Saia do ZERO em APENAS UMA AULA, do canal Jerry Strazzeri.

<https://youtu.be/OFLMhFuArXQ?si=hPGXyGlrZKS1wo2Q>



DESENVOLVA SEU POTENCIAL

DESENVOLVIMENTO DE UM SISTEMA DE GESTÃO ACADÊMICA

Neste tema de aprendizagem veremos um estudo de caso completo para um **sistema de gestão acadêmica**, demonstrando desde a análise inicial até a implementação final do banco de dados. O sistema será desenvolvido para atender às necessidades de uma instituição de ensino superior.

Análise de requisitos e modelagem conceitual

Antes de começar a implementação de um banco de dados, é essencial entender quais problemas ele deve resolver (Machado, 2020). No caso do **sistema de gestão acadêmica**, o objetivo é criar uma solução para **gerenciar as informações** de alunos, professores, cursos, disciplinas, turmas e matrículas.

VOCÊ SABE RESPONDER?

Você sabe por onde começar? A primeira etapa do desenvolvimento desse sistema é a análise de requisitos, em que identificamos as necessidades específicas da aplicação e documentamos as funcionalidades esperadas.

Imagine que você foi contratado para criar esse sistema para uma universidade. Durante a coleta de informações, foram identificadas diversas necessidades. Primeiramente, é necessário **gerenciar usuários**, classificando-os como alunos ou professores. Para cada usuário, o sistema deve salvar dados, como nome, e-mail e telefone. No caso dos professores, é importante que eles estejam **vinculados** às disciplinas que lecionam, enquanto os alunos devem ser **associados** aos respectivos cursos.



Os **cursos** oferecidos têm características específicas, como nome, carga horária e modalidade (presencial ou a distância). Cada curso é composto por **várias disciplinas**, as quais devem conter informações, como carga horária e possíveis **pré-requisitos**, para serem cursadas. Além disso, as disciplinas são ofertadas por meio de **turmas**, cada uma vinculada a um **professor** responsável, com um semestre de oferta, horários de aula e uma capacidade máxima de alunos. O sistema deve controlar o preenchimento dessas vagas, garantindo que a capacidade não seja ultrapassada.

Outro ponto identificado foi o **gerenciamento de matrículas**. O sistema precisa registrar o vínculo entre **alunos** e **turmas**, incluindo informações, como data da matrícula e o **status** dessa relação, que pode ser ativo, cancelado ou concluído. Para complementar o acompanhamento acadêmico, as turmas devem registrar as **notas** obtidas pelos alunos em diferentes avaliações, como provas e trabalhos, permitindo o cálculo da média final de cada aluno. Além disso, o sistema deve controlar a **frequência** dos alunos em cada turma, registrando o percentual de presença para avaliar a situação acadêmica.

Durante a análise, foram estabelecidas algumas **regras de negócio**. Por exemplo, um aluno só pode se matricular em disciplinas associadas ao próprio curso. Para ser aprovado, o estudante precisa atingir uma média mínima de 6.0 nas avaliações e ter, pelo menos, 75% de frequência (Machado, 2020). Também, o sistema deve impedir que alunos se matriculem em turmas em que a capacidade máxima já foi atingida. No caso de disciplinas com pré-requisitos, é necessário validar se o aluno cumpriu as exigências antes de efetivar a matrícula.

Com base nesses requisitos, foi possível identificar as entidades principais que compõem o sistema, como Usuário, Aluno, Professor, Curso, Disciplina, Turma, Matrícula, Nota e Frequência. Essas entidades refletem as estruturas fundamentais necessárias para o funcionamento do sistema.

Criação do Modelo Entidade-Relacionamento (MER)

Após a análise de requisitos exibida na seção anterior, o próximo passo no desenvolvimento do sistema de gestão acadêmica é a construção do MER (Ricardo; Urban; Davis, 2022). O MER é uma representação gráfica que descreve as entidades do sistema, os respectivos atributos e os relacionamentos entre elas.

No estudo de caso, as principais entidades identificadas foram:

- **Usuário**, que representa qualquer pessoa cadastrada no sistema, sejam alunos ou professores.
- **Aluno**, uma especialização de Usuário que inclui atributos, como matrícula e curso.
- **Professor**, que também herda de Usuário e possui informações sobre as disciplinas que leciona.
- **Curso**, que organiza as disciplinas e é vinculado a uma modalidade (presencial ou a distância).
- **Disciplina**, que define os componentes curriculares de um curso e pode ter pré-requisitos.

- **Turma**, que oferece disciplinas em um determinado semestre e possui informações, como professor responsável, horário e capacidade máxima de alunos.
- **Matrícula**, que vincula alunos a turmas e registra o status da inscrição.
- **Nota**, que representa o desempenho acadêmico dos alunos.
- **Frequência**, que armazena os percentuais de presença.

O **MER**, apresentado na Figura 1, destaca os relacionamentos entre essas entidades (Ricardo; Urban; Davis, 2022). Por exemplo, cada disciplina está associada a um curso, mas pode ser ofertada em várias turmas. Uma turma, por sua vez, é ministrada por um único professor e pode ter múltiplos alunos matriculados. Para registrar o vínculo entre alunos e turmas, a entidade **Matrícula** foi criada, armazenando informações, como data da matrícula e status. Além disso, os registros de notas e frequência são vinculados diretamente à matrícula, o que possibilita uma associação precisa entre desempenho e presença.

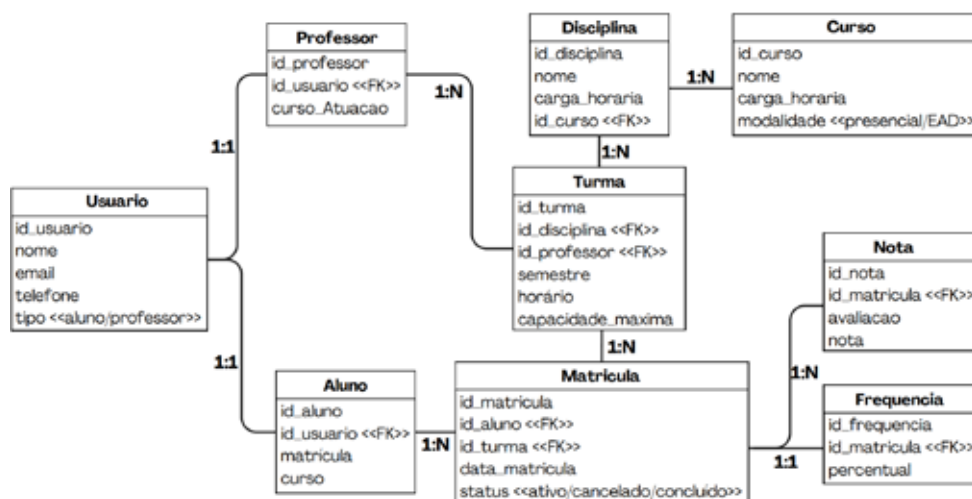


Figura 1 - MER do sistema de gestão acadêmica / Fonte: o autor

Descrição da Imagem: é exibido um Modelo de Entidade-Relacionamento em preto e branco que apresenta tabelas de um banco de dados acadêmico com campos, relacionamentos e cardinalidades. Inclui entidades, como Usuario, Aluno, Professor, Curso, Disciplina, Turma, Matrícula, Nota e Frequência, conectadas por linhas representando relações, como 1:1 e 1:N. Fim da descrição.

A Figura 1 ilustra os **atributos** de cada entidade e as **cardinalidades** dos relacionamentos. É possível observar que um aluno pode estar matriculado em várias turmas, mas cada matrícula está vinculada a somente uma turma. De forma semelhante, uma disciplina pode ser ofertada em várias turmas, enquanto cada turma pertence a uma única disciplina. Esses relacionamentos garantem que as regras de negócio identificadas na análise de requisitos sejam devidamente representadas no modelo (Elmasri; Navathe, 2019).

O **MER** é uma ferramenta essencial para o desenvolvimento do banco de dados, pois facilita a visualização da estrutura lógica do sistema antes da implementação física (Elmasri; Navathe, 2019). Ele também serve como um guia para a próxima etapa, momento em que transformaremos esse modelo conceitual em um **Diagrama Entidade-Relacionamento (DER)**, que é mais detalhado e próximo da implementação no banco de dados.

Implementação do Diagrama Entidade-Relacionamento (DER)

Após a criação do MER, o próximo passo é a construção do **DER** (Machado, 2020). Essa etapa traduz o modelo conceitual para um formato mais próximo da implementação física no banco de dados. O **DER** não apenas detalha as entidades e os respectivos atributos, mas também define os **tipos de dados** e as **restrições** que serão aplicadas durante a criação das tabelas no banco.



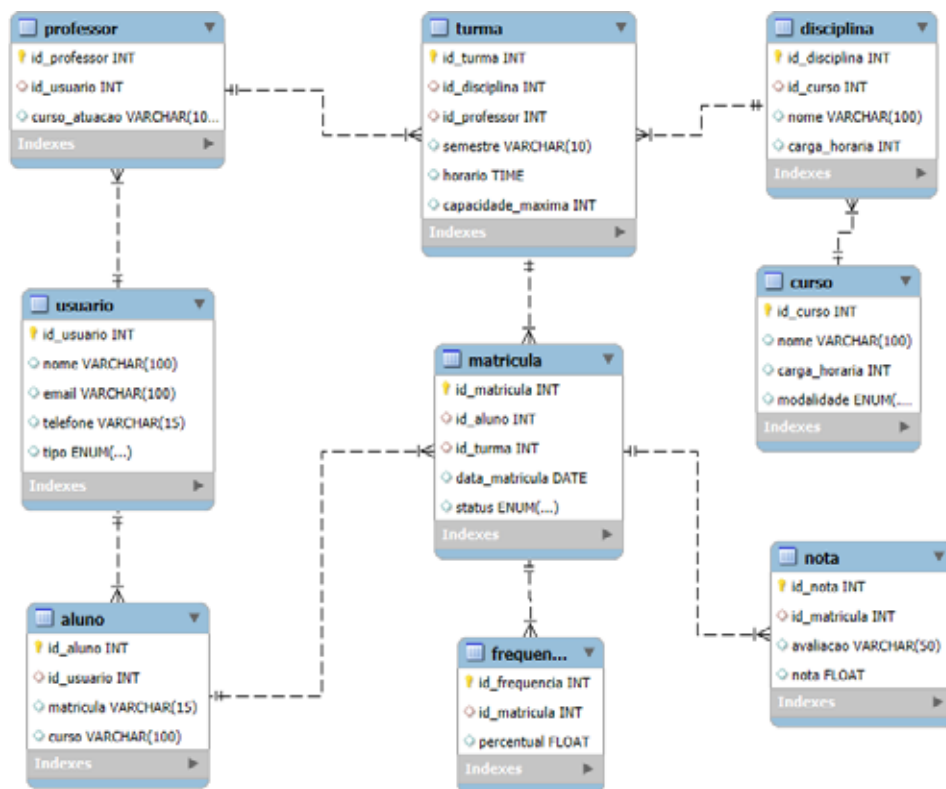


Figura 2 - DER do sistema de gestão acadêmica / Fonte: o autor.

Descrição da Imagem: é exibido um Diagrama Entidade-Relacionamento representando tabelas de um banco de dados, como Usuario, Aluno, Professor, Curso, Disciplina, Turma, Matrícula, Nota e Frequência, com conexões indicando relacionamentos e chaves estrangeiras. Os campos de cada tabela e as respectivas relações são detalhados no esquema. Fim da descrição

O DER apresentado na Figura 2 foi desenvolvido no MySQL Workbench e reflete as necessidades do **sistema de gestão acadêmica** descritas. Cada entidade foi estruturada com os respectivos atributos e tipos de dados. Por exemplo, a entidade Usuario contém atributos, como nome (VARCHAR 100), email (VARCHAR 100), telefone (VARCHAR 15) e o campo tipo, que utiliza o tipo **ENUM** para indicar se o usuário é um aluno ou professor. A entidade Aluno é associada a Usuario e inclui atributos adicionais, como matricula e curso. Os relacionamentos entre as entidades estão representados graficamente com as devidas cardinalidades (Machado, 2020).

Além dos relacionamentos, o DER define restrições importantes, como chaves primárias e estrangeiras. As chaves primárias são usadas para garantir a unicidade de registros em cada entidade. Por exemplo, `id_usuario` é a chave primária da entidade `Usuario`, enquanto `id_matricula` é a chave primária de `Matrícula`. Já as chaves estrangeiras, como `id_curso` em `Disciplina` ou `id_turma` em `Matrícula`, garantem a **integridade referencial**, conectando os registros entre as tabelas relacionadas (Machado, 2020).

Em resumo, a **Figura 2** apresenta como cada entidade e os respectivos relacionamentos estão estruturados no DER. Além de documentar a lógica do banco de dados, essa representação é essencial para a implementação prática do banco, uma vez que fornece um guia para a escrita do script **Data Definition Language (DDL)** que será usado para criar o banco no MySQL.

IMPLEMENTAÇÃO DO BANCO DE DADOS

A seguir, abordaremos a implementação prática do banco de dados, transformando o modelo conceitual e o diagrama desenvolvidos em um banco de dados funcional.

Criação do script DDL completo

A implementação do banco de dados começa com a criação das estruturas fundamentais por meio de comandos DDL (Machado, 2020). O script a seguir foi elaborado com base no DER apresentado. Ele define todas as tabelas necessárias para suportar o sistema de gestão acadêmica, incluindo atributos, chaves primárias, chaves estrangeiras e restrições de integridade. O script apresentado a seguir pode ser executado no **MySQL Workbench** para criar o banco de dados e as respectivas tabelas.

```
-- Criação do banco de dados
CREATE DATABASE sistema_academico;
USE sistema_academico;

-- Tabela Usuario
CREATE TABLE Usuario (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    telefone VARCHAR(15),
    tipo ENUM('aluno', 'professor', 'administrador') NOT NULL
);

-- Tabela Aluno
CREATE TABLE Aluno (
    id_aluno INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    matricula VARCHAR(15) UNIQUE NOT NULL,
    curso VARCHAR(100),
    FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE CASCADE
);

-- Tabela Professor
CREATE TABLE Professor (
    id_professor INT AUTO_INCREMENT PRIMARY KEY,
    id_usuario INT NOT NULL,
    curso_atuacao VARCHAR(100),
    FOREIGN KEY (id_usuario) REFERENCES Usuario(id_usuario) ON DELETE CASCADE
);
```


-- Tabela Curso

CREATE TABLE Curso (

id_curso INT AUTO_INCREMENT PRIMARY KEY,

nome VARCHAR(100) NOT NULL,

carga_horaria INT,

modalidade ENUM('Presencial', 'EAD') NOT NULL

);

-- Tabela Disciplina

CREATE TABLE Disciplina (

id_disciplina INT AUTO_INCREMENT PRIMARY KEY,

nome VARCHAR(100) NOT NULL,

carga_horaria INT,

id_curso INT NOT NULL,

FOREIGN KEY (id_curso) REFERENCES Curso(id_curso) ON DELETE CASCADE

);

-- Tabela Prerequisito (Auto-relacionamento)

CREATE TABLE Prerequisito (

id_disciplina INT NOT NULL,

id_prerequisito INT NOT NULL,

PRIMARY KEY (id_disciplina, id_prerequisito),

FOREIGN KEY (id_disciplina) REFERENCES Disciplina(id_disciplina) ON DELETE CASCADE,

FOREIGN KEY (id_prerequisito) REFERENCES Disciplina(id_disciplina) ON DELETE CASCADE

);

-- Tabela Turma

```
CREATE TABLE Turma (  
    id_turma INT AUTO_INCREMENT PRIMARY KEY,  
    id_disciplina INT NOT NULL,  
    id_professor INT NOT NULL,  
    semestre VARCHAR(10) NOT NULL,  
    horario TIME NOT NULL,  
    capacidade_maxima INT NOT NULL,  
  
    FOREIGN KEY (id_disciplina) REFERENCES Disciplina(id_disciplina) ON DELETE CASCADE,  
  
    FOREIGN KEY (id_professor) REFERENCES Professor(id_professor) ON DELETE CASCADE  
);
```

-- Tabela Matricula

```
CREATE TABLE Matricula (  
    id_matricula INT AUTO_INCREMENT PRIMARY KEY,  
    id_aluno INT NOT NULL,  
    id_turma INT NOT NULL,  
    data_matricula DATE NOT NULL,  
    status ENUM('Ativo', 'Cancelado', 'Concluido') NOT NULL,  
  
    FOREIGN KEY (id_aluno) REFERENCES Aluno(id_aluno) ON DELETE CASCADE,  
    FOREIGN KEY (id_turma) REFERENCES Turma(id_turma) ON DELETE CASCADE  
);
```

-- Tabela Nota

```
CREATE TABLE Nota (  
    id_nota INT AUTO_INCREMENT PRIMARY KEY,  
    id_matricula INT NOT NULL,  
    avaliacao VARCHAR(50),  
    nota FLOAT NOT NULL,
```

```

    FOREIGN KEY (id_matricula) REFERENCES Matricula(id_matricula) ON DELETE
    CASCADE

);

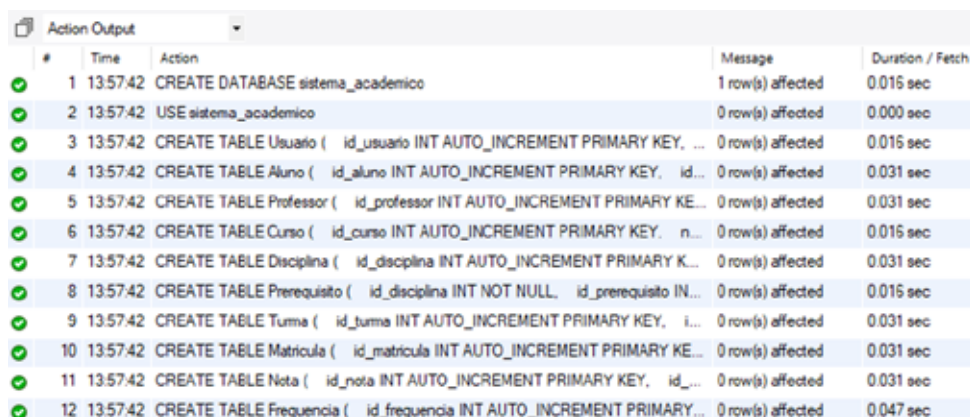
-- Tabela Frequência
CREATE TABLE Frequencia (
    id_frequencia INT AUTO_INCREMENT PRIMARY KEY,
    id_matricula INT NOT NULL,
    percentual FLOAT NOT NULL,
    FOREIGN KEY (id_matricula) REFERENCES Matricula(id_matricula) ON DELETE
    CASCADE

);

```

O script começa com a criação do banco de dados **sistema_academico** e a definição de tabelas fundamentais, como Usuario, Aluno, Professor, Curso, Disciplina, Turma, Matrícula, Nota e Frequência. Cada tabela foi estruturada para refletir os relacionamentos e regras de negócio estabelecidos no modelo conceitual. Por exemplo, o campo tipo na tabela Usuario usa o tipo **ENUM** para limitar os valores a “aluno” ou “professor”, garantindo a consistência nos dados.





#	Time	Action	Message	Duration / Fetch
1	13:57:42	CREATE DATABASE sistema_academico	1 row(s) affected	0.016 sec
2	13:57:42	USE sistema_academico	0 row(s) affected	0.000 sec
3	13:57:42	CREATE TABLE Usuario (id_usuario INT AUTO_INCREMENT PRIMARY KEY, ...	0 row(s) affected	0.016 sec
4	13:57:42	CREATE TABLE Aluno (id_aluno INT AUTO_INCREMENT PRIMARY KEY, id_...	0 row(s) affected	0.031 sec
5	13:57:42	CREATE TABLE Professor (id_professor INT AUTO_INCREMENT PRIMARY KE...	0 row(s) affected	0.031 sec
6	13:57:42	CREATE TABLE Curso (id_curso INT AUTO_INCREMENT PRIMARY KEY, n...	0 row(s) affected	0.016 sec
7	13:57:42	CREATE TABLE Disciplina (id_disciplina INT AUTO_INCREMENT PRIMARY K...	0 row(s) affected	0.031 sec
8	13:57:42	CREATE TABLE Prerequisito (id_disciplina INT NOT NULL, id_prerequisito IN...	0 row(s) affected	0.016 sec
9	13:57:42	CREATE TABLE Turma (id_turma INT AUTO_INCREMENT PRIMARY KEY, i...	0 row(s) affected	0.031 sec
10	13:57:42	CREATE TABLE Matricula (id_matricula INT AUTO_INCREMENT PRIMARY KE...	0 row(s) affected	0.031 sec
11	13:57:42	CREATE TABLE Nota (id_nota INT AUTO_INCREMENT PRIMARY KEY, id_...	0 row(s) affected	0.031 sec
12	13:57:42	CREATE TABLE Frequencia (id_frequencia INT AUTO_INCREMENT PRIMARY...	0 row(s) affected	0.047 sec

Figura 3 - Resultado da aplicação do script DDL para a criação de tabelas/ Fonte: o autor.

Descrição da Imagem: é exibido o log de execução de comandos SQL no MySQL Workbench, listando a criação do banco de dados sistema_academico e das tabelas principais. Cada comando foi executado com sucesso, conforme indicado pelo status "affected" em verde. Fim da descrição

Além disso, o uso de chaves primárias (**Primary Key**) assegura a unicidade dos registros, enquanto as chaves estrangeiras (**Foreign Key**) mantêm a integridade referencial entre as tabelas. O mecanismo **ON DELETE CASCADE** foi aplicado em várias relações para garantir que, ao excluir um registro em uma tabela principal, todos os registros relacionados sejam automaticamente removidos, evitando registros órfãos (Ricardo; Urban; Davis, 2022).

Ao aplicar o script no MySQL Workbench, o resultado esperado será o modelo físico ilustrado na **Figura 3** (Elmasri; Navathe, 2019).

População do banco de dados com dados de exemplo

Com o banco de dados criado e as tabelas devidamente estruturadas, o próximo passo é a **população do banco de dados** com dados de exemplo. Esses dados permitem testar e validar as funcionalidades do sistema, além de fornecerem uma base para consultas e análises futuras. O script a seguir insere informações nas tabelas de acordo com os requisitos e a modelagem apresentada (Elmasri; Navathe, 2019) (Machado, 2020).

```

--          Populando          a          tabela          Usuario
INSERT INTO Usuario (nome, email, telefone, tipo) VALUES
('Ana Silva', 'ana.silva@email.com', '123456789', 'aluno'),
('Carlos Pereira', 'carlos.pereira@email.com', '987654321', 'aluno'),
('Maria Souza', 'maria.souza@email.com', '456789123', 'professor'),
('João Oliveira', 'joao.oliveira@email.com', '321654987', 'professor');

-- Populando a tabela Aluno
INSERT INTO Aluno (id_usuario, matricula, curso) VALUES
(1, '20230001', 'Engenharia de Computação'),
(2, '20230002', 'Engenharia de Computação');

-- Populando a tabela Professor
INSERT INTO Professor (id_usuario, curso_atuacao) VALUES
(3, 'Engenharia de Computação'),
(4, 'Engenharia de Computação');

-- Populando a tabela Curso
INSERT INTO Curso (nome, carga_horaria, modalidade) VALUES
('Engenharia de Computação', 3600, 'Presencial');

-- Populando a tabela Disciplina
INSERT INTO Disciplina (nome, carga_horaria, id_curso) VALUES
('Algoritmos e Programação', 80, 1),
('Cálculo Diferencial', 60, 1),
('Estruturas de Dados', 80, 1);

```

-- Populando a tabela Prerequisito

```
INSERT INTO Prerequisito (id_disciplina, id_prerequisito) VALUES
```

```
(3, 1); -- Estruturas de Dados depende de Algoritmos e Programação
```

-- Populando a tabela Turma

```
INSERT INTO Turma (id_disciplina, id_professor, semestre, horario, capacidade_maxima) VALUES
```

```
(1, 1, '2025/2', '08:00:00', 30),
```

```
(2, 1, '2025/2', '10:00:00', 25),
```

```
(3, 2, '2025/2', '14:00:00', 20);
```

-- Populando a tabela Matrícula

```
INSERT INTO Matricula (id_aluno, id_turma, data_matricula, status) VALUES
```

```
(1, 1, '2025-08-01', 'Ativo'),
```

```
(1, 2, '2025-08-01', 'Ativo'),
```

```
(2, 3, '2025-08-01', 'Ativo');
```

-- Populando a tabela Nota

```
INSERT INTO Nota (id_matricula, avaliacao, nota) VALUES
```

```
(1, 'Prova 1', 8.5),
```

```
(1, 'Trabalho 1', 9.0),
```

```
(2, 'Prova 1', 7.0),
```

```
(2, 'Trabalho 1', 8.0),
```

```
(3, 'Prova 1', 6.5),
```

```
(3, 'Trabalho 1', 7.0);
```

```
-- Populando a tabela Frequência

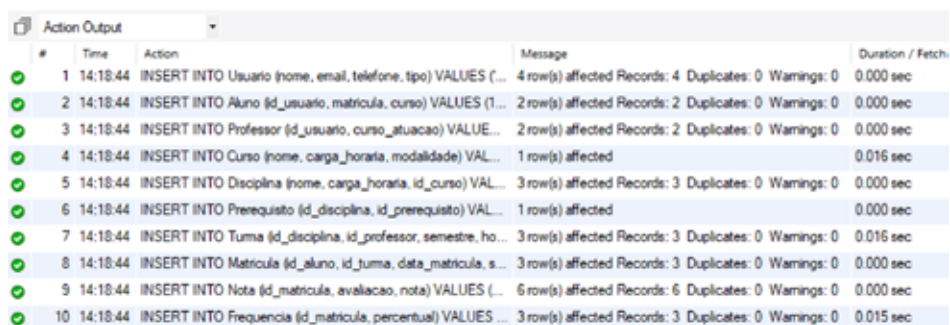
INSERT INTO Frequencia (id_matricula, percentual) VALUES

(1, 90.0),

(2, 85.0),

(3, 70.0);
```

Esse script cobre desde o cadastro inicial de usuários até a gestão de matrículas, o desempenho acadêmico e a frequência dos alunos. Por exemplo, os alunos “Ana Silva” e “Carlos Pereira” foram **matriculados em disciplinas** ofertadas no semestre “2025/2”, com os respectivos **status, notas e frequência** registrados.



#	Time	Action	Message	Duration / Fetch
1	14:18:44	INSERT INTO Usuario (nome, email, telefone, tipo) VALUES (...)	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
2	14:18:44	INSERT INTO Aluno (id_usuario, matricula, curso) VALUES (1...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
3	14:18:44	INSERT INTO Professor (id_usuario, curso_atuacao) VALUE...	2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
4	14:18:44	INSERT INTO Curso (nome, carga_horaria, modalidade) VAL...	1 row(s) affected	0.016 sec
5	14:18:44	INSERT INTO Disciplina (nome, carga_horaria, id_curso) VAL...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
6	14:18:44	INSERT INTO Prerequisite (id_disciplina, id_prerequisite) VAL...	1 row(s) affected	0.000 sec
7	14:18:44	INSERT INTO Turma (id_disciplina, id_professor, semestre, ho...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.016 sec
8	14:18:44	INSERT INTO Matricula (id_aluno, id_turma, data_matricula, s...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.000 sec
9	14:18:44	INSERT INTO Nota (id_matricula, avaliacao, nota) VALUES (...)	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec
10	14:18:44	INSERT INTO Frequencia (id_matricula, percentual) VALUES ...	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.015 sec

Figura 4 - Resultado da inserção de registros nas tabelas / Fonte: o autor.

Descrição da Imagem: a imagem exibe o log de execução de comandos INSERT no MySQL Workbench, que populam tabelas do banco de dados sistema_academico com dados de exemplo. Todos os comandos foram executados com sucesso, conforme indicado pelo status “affected” em verde. Fim da descrição

Ao aplicar esse script no MySQL Workbench, o banco de dados é **populado**, e o resultado pode ser visualizado na Figura 4.

CONSULTAS E ANÁLISES PRÁTICAS

A seguir, veremos exemplos práticos de consultas e análises que podem ser realizadas no banco de dados acadêmico, demonstrando como extrair informações relevantes para a gestão acadêmica e a tomada de decisões.

Listar alunos com as respectivas disciplinas e professores

Uma das consultas mais úteis para o sistema é listar todos os alunos matriculados, as disciplinas que estão cursando e os professores responsáveis. Essa consulta organiza os dados de forma clara e ordenada, exibindo informações, como a matrícula do aluno, o nome do estudante, a disciplina, o professor responsável e o semestre de oferta (Leite Júnior, 2021).

```
SELECT
    A.matricula AS Matricula_Aluno,
    U.nome AS Nome_Aluno,
    D.nome AS Disciplina,
    P.nome AS Professor,
    T.semestre AS Semestre
FROM Matricula M
JOIN Aluno A ON M.id_aluno = A.id_aluno
JOIN Usuario U ON A.id_usuario = U.id_usuario
JOIN Turma T ON M.id_turma = T.id_turma
JOIN Disciplina D ON T.id_disciplina = D.id_disciplina
JOIN Professor Pr ON T.id_professor = Pr.id_professor
JOIN Usuario P ON Pr.id_usuario = P.id_usuario
ORDER BY Nome_Aluno, Semestre;
```


Essa consulta utiliza múltiplos JOINS para combinar informações de diferentes tabelas, como Matricula, Aluno, Usuario, Turma, Disciplina e Professor (Leite Júnior, 2021). O resultado exibe Matrícula do Aluno, Nome do Aluno, Disciplina, Professor e Semestre, como ilustrado na Figura 5.

	Matricula_Aluno	Nome_Aluno	Disciplina	Professor	Semestre
►	20230001	Ana Silva	Algoritmos e Programação	Maria Souza	2023/2
	20230001	Ana Silva	Cálculo Diferencial	Maria Souza	2023/2
	20230002	Carlos Pereira	Estruturas de Dados	João Oliveira	2023/2

Figura 5 – Saída da consulta: alunos matriculados, disciplinas e professores
Fonte: o autor.

Descrição da Imagem: é exibido o resultado de uma consulta SQL. Assim, são expostas informações sobre alunos matriculados em disciplinas, incluindo matrícula do aluno, nome, disciplina, professor responsável e semestre.
Fim da descrição.

Média de notas por disciplina

A seguir, apresentamos uma consulta que pode ser usada para analisar o desempenho acadêmico médio dos alunos em cada disciplina. Essa análise permite identificar padrões, destacando as disciplinas com melhores resultados e aquelas que podem demandar **atenção pedagógica**. Utilizando a função **ROUND(AVG(N.nota), 2)**, as médias são calculadas e arredondadas para duas casas decimais, com os resultados exibidos em ordem decrescente, priorizando as disciplinas com melhor desempenho (Leite Júnior, 2021).

SELECT

D.nome AS Disciplina,

ROUND(AVG(N.nota), 2) AS Media_Notas

FROM

Nota N

JOIN Matricula M ON N.id_matricula = M.id_matricula

```

JOIN Turma T ON M.id_turma = T.id_turma
JOIN Disciplina D ON T.id_disciplina = D.id_disciplina
GROUP BY D.nome
ORDER BY Media_Notas DESC;

```

A Figura 6 ilustra o resultado da consulta, destacando as disciplinas com as melhores médias obtidas pelos alunos (Leite Júnior, 2021).

	Disciplina	Media_Notas
▶	Algoritmos e Programação	8.75
	Cálculo Diferencial	7.5
	Estruturas de Dados	6.75

Figura 6 – Resultado de consulta: média de notas por disciplina / Fonte: o autor.

Descrição da Imagem: é exibido o resultado de uma consulta SQL que calcula a média de notas para cada disciplina. Os dados apresentados incluem o nome da disciplina e a média das notas arredondadas para duas casas decimais. Fim da descrição

Essa análise fornece uma base sólida para avaliar a eficácia dos métodos de ensino e identificar áreas de melhoria.

Lista de alunos aprovados e reprovados

A seguir, apresentamos uma consulta que analisa o status acadêmico de cada aluno, classificando-os como aprovados ou reprovados com base no **desempenho acadêmico** e na **frequência**. Para aprovação, o estudante deve atingir uma média mínima de 6.0 e ter uma frequência igual ou superior a 75%.

```
SELECT
    A.matricula AS Matricula_Aluno,
    U.nome AS Nome_Aluno,
    D.nome AS Disciplina,
    ROUND(AVG(N.nota), 2) AS Media_Final,
    F.percentual AS Frequencia,
    CASE
        WHEN AVG(N.nota) >= 6.0 AND F.percentual >= 75 THEN 'Aprovado'
        ELSE 'Reprovado'
    END AS Status_Academico
FROM Nota N
JOIN Matricula M ON N.id_matricula = M.id_matricula
JOIN Frequencia F ON M.id_matricula = F.id_matricula
JOIN Aluno A ON M.id_aluno = A.id_aluno
JOIN Usuario U ON A.id_usuario = U.id_usuario
JOIN Turma T ON M.id_turma = T.id_turma
JOIN Disciplina D ON T.id_disciplina = D.id_disciplina
GROUP BY A.matricula, U.nome, D.nome, F.percentual
ORDER BY Nome_Aluno, Disciplina;
```

A consulta utiliza múltiplos **JOINS** para unir as tabelas envolvidas, aplicando uma lógica condicional com **CASE** para determinar o status acadêmico de cada aluno em cada disciplina (Leite Júnior, 2021). Essa abordagem facilita os acompanhamentos individual e coletivo, permitindo identificar padrões e alunos em risco de reprovação.

O resultado da consulta pode ser visualizado na Figura 7, que mostra o desempenho acadêmico de cada aluno em suas respectivas disciplinas.

	Matricula_Aluno	Nome_Aluno	Disciplina	Media_Final	Frequencia	Status_Academico
▶	20230001	Ana Silva	Algoritmos e Programação	8.75	90	Aprovado
	20230001	Ana Silva	Cálculo Diferencial	7.5	85	Aprovado
	20230002	Carlos Pereira	Estruturas de Dados	6.75	70	Reprovado

Figura 7 - Resultado de consulta: lista de alunos aprovados e reprovados / Fonte: o autor.

Descrição da Imagem: é exibida uma tabela com os resultados de uma consulta SQL que exibe o status acadêmico dos alunos. Para cada aluno, são exibidos os campos "Matricula_Aluno", "Nome_Aluno", "Disciplina", "Media_Final", "Frequencia" e "Status_Academico". Fim da descrição.

Relatório completo de um aluno específico

A seguir, apresentamos uma consulta que gera um **relatório detalhado** para um aluno específico, o que permite analisar todas as disciplinas cursadas, a média das notas, a frequência em cada uma delas e o status de aprovação. O filtro aplicado pela cláusula **WHERE** seleciona os dados de um aluno específico com base na matrícula (Leite Júnior, 2021).

```
SELECT
    A.matricula AS Matricula_Aluno,
    U.nome AS Nome_Aluno,
    C.nome AS Curso,
    D.nome AS Disciplina,
    ROUND(AVG(N.nota), 2) AS Media_Final,
    F.percentual AS Frequencia,
    CASE
        WHEN AVG(N.nota) >= 6.0 AND F.percentual >= 75 THEN 'Aprovado'
        ELSE 'Reprovado'
    END AS Status_Academico
```

```

FROM Nota N

JOIN Matricula M ON N.id_matricula = M.id_matricula

JOIN Frequencia F ON M.id_matricula = F.id_matricula

JOIN Aluno A ON M.id_aluno = A.id_aluno

JOIN Usuario U ON A.id_usuario = U.id_usuario

JOIN Curso C ON A.curso = C.nome

JOIN Turma T ON M.id_turma = T.id_turma

JOIN Disciplina D ON T.id_disciplina = D.id_disciplina

WHERE A.matricula = '20230001'

GROUP BY A.matricula, U.nome, C.nome, D.nome, F.percentual

ORDER BY D.nome;

```

O resultado da consulta exibe informações organizadas do estudante, como matrícula, nome, curso, disciplinas cursadas, médias das notas, frequência e status acadêmico (Machado, 2020). Essa abordagem é essencial para uma análise detalhada e personalizada do progresso acadêmico de cada estudante.

	Matricula_Aluno	Nome_Aluno	Curso	Disciplina	Media_Final	Frequencia	Status_Academico
▶	20230001	Ana Silva	Engenharia de Computação	Algoritmos e Programação	8.75	90	Aprovado
	20230001	Ana Silva	Engenharia de Computação	Cálculo Diferencial	7.5	85	Aprovado

Figura 8 - Resultado de consulta: relatório completo de um aluno específico
Fonte: o autor.

Descrição da Imagem: trata-se de uma tabela com os resultados de uma consulta SQL para um aluno específico (exemplo: matrícula 20230001). A tabela exibe as colunas "Matrícula_Aluno", "Nome_Aluno", "Curso", "Disciplina", "Media_Final", "Frequencia" e "Status_Academico". Fim da descrição

O relatório completo auxilia na identificação de pontos fortes e fracos no desempenho do aluno, o que fomenta o planejamento de estratégias para melhorar o rendimento acadêmico.

**EM FOCO**

Acesse seu ambiente virtual de aprendizagem e confira a aula referente a este tema.. **Recursos de mídia disponíveis no conteúdo digital do ambiente virtual de aprendizagem.**

NOVOS DESAFIOS

Neste tema de aprendizagem, aprendemos a transformar a teoria em prática no desenvolvimento de sistemas baseados em bancos de dados. Agora é o momento de conectar o conhecimento adquirido às exigências do mercado de trabalho e refletir sobre os impactos dessas habilidades em sua carreira.

No início do tema, apresentamos o desafio de desenvolver um sistema de gestão acadêmica eficiente e funcional. Você aprendeu a analisar requisitos, criar um modelo conceitual bem estruturado, transformar esse modelo em um Diagrama Entidade-Relacionamento (DER) e implementá-lo por meio de scripts DDL. Ao longo do caminho, você também entendeu como popular o banco de dados com dados reais e implementar consultas avançadas para gerar relatórios detalhados. Essas práticas simulam demandas reais enfrentadas em projetos corporativos e te preparam para atuar em cenários desafiadores.

No mercado de trabalho, profissionais capazes de construir bancos de dados eficientes e realizar análises práticas de desempenho são altamente valorizados. Imagine, por exemplo, um analista de dados otimizando o fluxo de informações em uma grande empresa ou um desenvolvedor projetando sistemas críticos que exigem alto desempenho. Esses são os cenários em que o conhecimento adquirido neste tema faz a diferença.

Utilize esses conhecimentos como um diferencial em sua carreira

Por fim, reflita sobre o impacto desse aprendizado em sua trajetória profissional. Você está agora mais preparado para enfrentar desafios, como o desenvolvimento de sistemas complexos, a análise de performance de consultas e a implementação de soluções escaláveis. Utilize esses conhecimentos como um diferencial em sua carreira e esteja pronto para aplicar essas habilidades em diferentes setores. O próximo passo é aprofundar ainda mais e explorar novas tecnologias para se destacar no mercado.



VAMOS PRATICAR

1. Ao estruturar um banco de dados relacional, é necessário definir a forma como as tabelas se relacionam para manter a organização e a integridade dos dados. Durante esse processo, certos elementos precisam ser implementados em uma ordem específica para garantir o correto funcionamento do sistema e evitar inconsistências.

Com base na integração entre chave primária e chave estrangeira, assinale a alternativa correta:

- a) Após criar a chave estrangeira, é possível excluir a chave primária da tabela referenciada sem impactar a estrutura do banco.
- b) Uma chave estrangeira pode referenciar qualquer campo da tabela pai, independentemente de ser uma chave primária ou não.
- c) A chave estrangeira pode ser criada antes da chave primária, pois não há dependência entre essas restrições no banco de dados
- d) Uma chave estrangeira impede qualquer exclusão ou alteração na chave primária da tabela referenciada, mesmo quando há configurações específicas para permitir isso.
- e) A chave primária deve ser criada antes da chave estrangeira, pois é necessário garantir que uma referência esteja disponível antes de estabelecer o vínculo entre as tabelas.

VAMOS PRATICAR

2. As relações entre tabelas em bancos de dados relacionais são fundamentais para garantir a organização e a integridade das informações. Determinados mecanismos, como chaves estrangeiras e regras de exclusão, desempenham um papel muito importante no controle e na manipulação dos dados ao longo do uso do sistema.

Com base nos conceitos relacionados à FOREIGN KEY e à cláusula ON DELETE CASCADE, analise as afirmativas a seguir:

- I - Uma CHAVE ESTRANGEIRA garante que os dados da tabela filha existam na tabela pai.
- II - FOREIGN KEY sempre permite que uma tabela filha contenha valores que não estejam relacionados à tabela pai.
- III - A cláusula ON DELETE CASCADE impede exclusões na tabela pai, mesmo que exista uma relação com registros na tabela filha.
- IV - ON DELETE CASCADE pode ser usado para permitir exclusões automáticas de registros na tabela pai, independentemente de sua relação com a tabela filha.

É correto o que se afirma em:

- a) I, apenas.
- b) II e IV, apenas.
- c) III e IV, apenas.
- d) I, II e III, apenas.
- e) I, II, III e IV.

VAMOS PRATICAR

3. Ao projetar um banco de dados relacional, é fundamental considerar a maneira como as tabelas interagem entre si para garantir a consistência dos dados. Mecanismos específicos podem ser configurados para automatizar determinadas operações em registros relacionados, o que influencia diretamente o comportamento do sistema e a integridade das informações armazenadas.

Com base nas informações apresentadas, avalie as asserções a seguir e a relação proposta entre elas:

I - A cláusula ON DELETE CASCADE permite que os registros na tabela sejam excluídos automaticamente quando o registro correspondente na tabela pai for excluído, evitando inconsistências.

PORQUE

II - O uso de chaves estrangeiras com a cláusula ON DELETE CASCADE elimina a necessidade de validações manuais durante a exclusão de dados relacionados.

A respeito dessas asserções, assinale a opção correta:

- a) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
- b) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
- c) A asserção I é uma proposição verdadeira e a II é uma proposição falsa.
- d) A asserção I é uma proposição falsa e a II é uma proposição verdadeira.
- e) As asserções I e II são falsas.

REFERÊNCIAS

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7. ed. São Paulo: Pearson, 2019.

LEITE JÚNIOR, J. C. **Aprendendo Banco de Dados MYSQL**: do Básico ao Avançado. [S. l.]: Amazon Press, 2021.

MACHADO, F. N. R. **Banco de Dados**: Projetos e Implementação. 4. ed. São Paulo: Érica, 2020.

RICARDO, C. M.; URBAN, S. D.; DAVIS, K. C. **Databases Illuminated**. 4. ed. Burlington: Jones & Bartlett Learning, 2022.

CONFIRA SUAS RESPOSTAS

1. Opção E.

- A. Incorreta. A exclusão da chave primária quebraria a relação de integridade referencial, podendo gerar erros na integridade dos dados.
- B. Incorreta. A chave estrangeira deve referenciar obrigatoriamente uma chave primária ou um campo com restrição de unicidade na tabela pai.
- C. Incorreta. A chave estrangeira depende da existência da chave primária na tabela referenciada para garantir a integridade dos dados.
- D. Incorreta. Dependendo das regras de integridade referencial definidas, algumas ações, como exclusão em cascata, podem ser permitidas.
- E. Correta. A chave primária deve existir primeiro para que a chave estrangeira possa estabelecer uma relação com ela, garantindo a consistência dos dados.

2. Opção A.

- I. Correta. A FOREIGN KEY mantém a integridade referencial entre as tabelas.
- II. Incorreta. Uma CHAVE ESTRANGEIRA impede a inserção de valores na tabela filha que não estejam presentes na tabela pai, garantindo a integridade referencial.
- III. Incorreta. Essa cláusula não impede exclusões, mas garante que elas sejam propagadas para a tabela filha.
- IV. Incorreta. Essa cláusula não exclui registros da tabela pai automaticamente, mas exclui os registros relacionados à tabela filha quando um registro na tabela pai for apagado.

3. Opção A.

- I. Correta. A cláusula ON DELETE CASCADE exclui automaticamente os registros na tabela filha relacionada ao registro excluído na tabela pai. Isso é essencial para evitar inconsistências, como registros órfãos.
- II. Correta. Ao usar a cláusula ON DELETE CASCADE, elimina-se a necessidade de programar validações ou exclusões manuais para manter a integridade referencial. Além disso, a asserção II justifica a I, já que a funcionalidade mencionada em I é um exemplo de como a exclusão em cascata automatiza esse processo.